`hacs`: the command line and the OC console

A user manual

Fabian Januszewski
Universität Paderborn
`fabian.januszewski@math.uni-paderborn.de`

Version 1.5.13, 2026-11-09 (restructured by subject; not binding; history in Appendix F)

Abstract

In memoriam Prof. Dr. Günter Harder (1938–2025).

`hacs` computes the cohomology of $\mathrm{SL}(2,\mathbb{Z})$ and of its congruence subgroups $\Gamma_0(N)$ with coefficients in the symmetric powers Sym^n of the standard representation, over a finite field $\mathbb{F}_\ell$, over $\mathbb{Z}/\ell^k$ and, by a descent, over $\mathbb{Z}$; the Hecke operators on it; their primary decompositions; and an extension layer of Hecke-module invariants (maximal ideals, local Hecke algebras, Loewy structure, the Dickson θ map, Hida theory). This manual explains what these objects are, how the program computes them, and how to drive it: the command line, the interactive and scripted console with its language OC, campaigns, the residue atlas, benchmarks and the engine switches. Every example is a real output of the program — a frozen console transcript or a command run by a generator script — and the reference tables of the 165 console verbs, 31 object types, 43 settings and 289 validations are generated from the binary. It is written for a mathematician who wants to compute, not for someone changing the code.

Contents

1 Getting started **5**

1.1 What the program computes . . . 5
1.2 Quick start . . . 6
1.3 Building the program . . . 7
1.4 The command line . . . 8
1.5 The validation harness . . . 10
1.6 How the examples in this manual are made . . . 11

2 The mathematical objects **12**

2.1 The group, the orbifold and the model . . . 12
2.2 Local systems: the symmetric powers . . . 12
2.3 Coefficient rings . . . 13
2.4 Levels and characters: $\Gamma_0(N)$ and nebentypus . . . 13

3 Cohomology over a field **15**

3.1 Dimensions and the golden reference . . . 15
3.2 Local systems away from the modular curve . . . 15
3.3 Coefficients $\mathbb{Z}/\ell^k$ . . . 16
3.4 A second model at $\ell = 2, 3$: Moerdijk–Pronk . . . 16

4 Hecke operators **18**

4.1 The operators and their certificates . . . 18
4.2 The flagship: T_2 on $(10, 691)$ . . . 18
4.3 Linear algebra on Hecke operators . . . 19

5 Integral cohomology: the descent over $\mathbb{Z}$ **20**

5.1 What is computed . . . 20
5.2 Switching it on . . . 21
5.3 The algorithm, step by step . . . 21
5.4 Reading the record . . . 23
5.5 Worked examples . . . 24
5.6 From the command line and in campaigns . . . 26

6 Extensions, Hida theory and the forms algebra **27**

6.1 The extension layer . . . 27
6.2 Sturm bounds and the U_ℓ budget . . . 27

6.3 The extension record of $(10, 691)$ . . . 28
6.4 The Dickson sequence, cone, boundary and ι . . . 29
6.5 Hida theory, Λ-adic objects and the forms algebra . . . 29
6.6 The residue atlas: `--atlas` . . . 30

7 The console and its language **32**
7.1 The console . . . 32
7.2 The OC language . . . 33
7.3 A tour of the small verbs . . . 37
7.4 Settings . . . 38
7.5 Errors on purpose: `expect_error` . . . 39
7.6 Settings, the fingerprint and `resume` . . . 40
7.7 The harness from the console . . . 40
7.8 The served console: `--serve` . . . 41
7.9 The atlas server: `--serve-atlas` . . . 42
7.10 The scripts, by subject . . . 43

8 Campaigns: `--campaign` **45**
8.1 What a campaign is . . . 45
8.2 The spec format . . . 46
8.3 What `certify` samples . . . 48
8.4 Output . . . 48
8.5 Campaigns over $\mathbb{Z}$ . . . 49
8.6 The orbifold census at $\ell = 2, 3$ . . . 50
8.7 Long census runs . . . 52

9 Performance and engine switches **54**
9.1 Benchmarks: `--bench` . . . 54
9.2 Micro-benchmarks . . . 56
9.3 Environment variables: overview . . . 57
9.4 Threads, substrate and the tape engine . . . 57
9.5 CPU kernels and native small fields . . . 58
9.6 The device: `ENGINE_DEVICE` and the CUDA layer . . . 59
9.7 Native $\mathbb{F}_2$ arithmetic and the GPU elimination . . . 60
9.8 The integral descent: routes, threads and the GPU . . . 61
9.9 The Hecke module and forms routes . . . 62
9.10 The orbifold census routes (`L23_*`) . . . 62
9.11 Diagnostics and test-only variables . . . 63
9.12 Owner-gated emitters, interlocks and strict mode . . . 64
9.13 Read-only validation hooks . . . 64
9.14 Hooks of the level-N curve and the Λ-adic tower . . . 65

10 Glossary and troubleshooting **68**
10.1 Glossary . . . 68
10.2 Troubleshooting . . . 69

A Verb reference **71**
A.1 Group `core` . . . 71
A.2 Group `arithmetic` . . . 72
A.3 Group `pipeline` . . . 75
A.4 Group `reproducibility` . . . 76
A.5 Group `harness` . . . 77
A.6 Group `reference` . . . 78
A.7 Group `extension` . . . 78

B Type reference **82**
`Fp` . . . 82
`Poly` . . . 82
`Matrix` . . . 82
`Model` . . . 82
`Complex` . . . 83
`LocalSystem` . . . 83
`Cohomology` . . . 83
`HeckeOperator` . . . 84
`Family` . . . 84
`ContragredientLocalSystem` . . . 85
`Pairing` . . . 85
`Block` . . . 86
`Decomposition` . . . 86
`Report` . . . 87
`CampaignSpec` . . . 87
`Sturm` . . . 87
`Atlas` . . . 88
`GF` . . . 88
`GFElt` . . . 88
`Settings` . . . 89
`ExtRecord` . . . 89
`MaximalIdeal` . . . 90
`HeckeAlgebra` . . . 90
`LoewyProfile` . . . 91
`ExtensionClass` . . . 91
`Presentation` . . . 92
`Involution` . . . 92
`BoundaryCohomology` . . . 93
`ConeCohomology` . . . 93
`DicksonLES` . . . 94
`DescentRecord` . . . 94

C Settings reference **97**

D Validation roster **101**

E Regenerating this manual **104**

F Versions **105**

Index **109**

Chapter 1

Getting started

1.1 What the program computes

What the program computes. Let $\Gamma = \mathrm{SL}(2,\mathbb{Z})$ and let $M_n = \mathrm{Sym}^n(\mathbb{F}_\ell^2)$ be the n-th symmetric power of the standard representation, with the action fixed by `[C, def:rhon]`. `hacs` computes the equivariant cohomology $H^q(\Gamma; M_n)$, $q \in \{0,1,2\}$, on a seven-vertex model of the orbifold $\Gamma\backslash\mathfrak{H}$; for $6 \in \mathbb{F}_\ell^\times$ this is group cohomology (`[C, rmk:orbifold]`). On top it computes the Hecke operators T_p, U_ℓ (the same code path at $p = \ell$), T_{p^r} and $T(p,p)$ on cohomology (`[C, def:hecke]`, `[C, thm:relations]`, `[C, prop:diamond]`), the certified primary decomposition of H^q over a commuting Hecke family, and the extension layer of [E]: the maximal ideals of the Hecke algebra, local Hecke algebras $\mathbb{T}_\mathfrak{m}$, Loewy and socle series, the cone and boundary cohomology, the involution ι^* (`[C, def:iota]`), the Dickson long exact sequence (`[E, sec:thetaobj]`) and the $\mathbb{Z}/\ell^k$ coefficient route (`[E, sec:ring]`). All arithmetic is exact: residues modulo a prime ℓ (up to 2^{64} in places), or modulo ℓ^k.

One binary, several front ends. Everything is one executable, `hacs`, whose first argument selects a front end (Section 1.4):

- the **validation harness** (`--list`, `--all`, `--all-full`, validation names) – the project's test suite, which is also the most reliable way to reproduce a published number;
- the **benchmark driver** `--bench`: one pipeline point with timings;
- the **campaign driver** `--campaign`: a parameter sweep that writes the mathematics to JSON and TSV;
- the **atlas builder** `--atlas`: the residue atlas $\Sigma(\ell)$;
- the **console** (`--console`, `--script`, `-c`, `-`): an interactive and scriptable expression language, OC, over the whole pipeline (Chapter 7).

How this manual is organised. By subject, from the mathematics to the machinery. Chapter 2 introduces the objects: the group, the model, the local systems Sym^n, the coefficient rings $\mathbb{F}_\ell$, $\mathbb{Z}/\ell^k$, $\mathbb{Z}$, and the levels $\Gamma_0(N)$. Chapters 3–6 compute with them in increasing depth: cohomology over a field, Hecke operators, integral cohomology by descent, and the extension layer with Hida theory. Chapter 7 is the console and its language; Chapter 8 the campaign driver for sweeps; Chapter 9 benchmarks and every environment variable. Chapter 10 has a glossary and troubleshooting, and the appendices are the generated reference tables. Command-line tokens, console verbs, types, settings and environment variables are indexed by name.

Sources of truth. This manual restates; it does not legislate. The mathematics is the binding contract [C] (`docs/latex/gl2_contract.tex`) and the extension specification [E] (`docs/latex/hecke_extensions.tex`), cited by label. The console design is [P] (`docs/console/0`

0-console-interface-plan.md); docs/console/console.md is the console's own generated Markdown reference; the campaign format is docs/matrix-engine/campaign.md; the per-validation encyclopaedia is docs/validation-catalogue.md; benchmark reference figures live in docs/performance-reference.md. Where this manual and the code disagree, the code wins and CliManualCoverageValidation should have failed (Appendix E).

Note

Generated parts. The usage texts in boxes, the token tables, and all of Appendices A–D are generated from the binary by docs/latex/tools/gen_cli_manual_tables.sh; the counts in the prose (165 verbs, 289 validations, ...) are generated too. They describe the build the manual was generated from.

1.2 Quick start

Build once (Section 1.3 has the details), then from the build directory:

```
./hacs --help                     # the top-level usage
./hacs --list | head              # registered validations and tiers
./hacs mod_arithmetic_validation  # run one validation; exit code 0 = pass
./hacs --bench hecke 48 134217757 2      # T_2 on H^*(SL(2,Z); Sym^48), one BENCH line
./hacs -c 'print(cohomology(symmetric_power(sl2z_model(), 10, 691)).dims)'
./hacs --console                  # interactive session, prompt "oc> "
```

The build also creates cohomology_app, a symbolic link to hacs (the program's former name), so older scripts keep working; this manual uses hacs throughout.

The one-liner above computes $\dim H^q(\Gamma; \mathrm{Sym}^{10}\mathbb{F}_{691}^2)$ for $q = 0, 1, 2$; the frozen transcript of script 001_dimensions.oc (Section 3.1) shows the answer [0, 3, 0]. A typical console session builds the flagship cell $(n, \ell) = (10, 691)$ step by step:

From Console/scripts/006_flagship.oc:

```
L = symmetric_power(MODEL, N_SYM, ELL);
H = cohomology(L);
print(H.dims);
T2 = hecke(H, P_HECKE);
A = T2.A(1);
print(charpoly(A));
D = decompose(T2, 1);
print(D.num_blocks);
```

(MODEL, N_SYM, ELL, P_HECKE are bound by the shared prelude.oc to sl2z_model(), 10, 691, 2.) Section 4.2 quotes the full transcript.

Odd n. On this model $H^\bullet(\Gamma; \mathrm{Sym}^n)$ vanishes for odd n, because $-I$ acts by $(-1)^n$ ([C, rmk:negI]). Use an even n for anything you want to look at; the bench usage text says so too.

Limits. --bench parses ℓ (and n, p, r) as 32-bit unsigned integers; a larger value is refused (with the misleading message "must be non-negative integers"). A campaign spec takes a 64-bit ℓ (but refuses coefficient_ring = Z/ell^k with $\ell^k \geq 2^{64}$ together with ext = true). --atlas needs $5 \leq \ell \leq 100000$. The U_ℓ operator needs $\ell \leq 23170$ ([C, lem:bounds]). coefficient_ring = Z/ell^<k> needs $2 \leq k \leq 4096$.

1.3 Building the program

The build needs CMake $\geq$ 3.18 ($\geq$ 3.21 for the presets), a C++17 compiler and, unless it is configured CPU-only (below), the CUDA toolkit (`nvcc`); the CUDA layer is optional at run time and never used by default (Section 9.4).

```
cmake -B build -S . -DCMAKE_BUILD_TYPE=RelWithDebInfo   # out-of-source (the only
↪  kind)
make -C build -j8
./build/hacs --list
```

The sources live under `src/`. Builds are out-of-source only: an in-source configure (`cmake .` in the repository root) is refused with a one-line hint. A build directory's `CMakeCache.txt` pins the configuration it was first made with, so start a fresh build directory (or remove its `CMakeCache.txt` and `CMakeFiles/`) before changing it.

`CMAKE_BUILD_TYPE`	**Flags**	**Use**
`RelWithDebInfo`	`-O2 -g -DNDEBUG`	the default; everyday runs and validations
`Debug`	`-O0 -g`	debugging (much slower)
`Release`	`-O3 -DNDEBUG`	–
`Bench`	`-O3 -march=native -DNDEBUG`	timing runs
`Check`	`-O1 -g -fsanitize=address,undefined`	sanitizer runs (debug checks live)
`CheckThread`	`-O1 -g -fsanitize=thread`	ThreadSanitizer (run with `ENGINE_CUDA_DISABLE=1`)

CMake options. None of them changes a published value: `-DHECKE_PURE_INTEGER=ON` disables the certified floating-point filter of the Hecke sign predicates (bit-identical results, slower); `-DENGINE_SHADOW_LEGACY=ON` runs engine and legacy routes side by side and throws on the first differing entry; `-DENGINE_VERIFY_DEVICE=ON` re-executes every GPU node on the CPU and compares; `-DENGINE_COUNT_OBJECTS=ON` adds object counters to the `BENCH` line; `-DOC_CUDA_ARCHITECTURES=...` overrides the GPU architecture (default `120`; `87` on a Jetson Orin).

Build switches and presets. Three options are decided before anything is compiled; a component switched off is neither required nor probed. `-DOC_ENABLE_CUDA=OFF` (default `ON`) builds CPU-only: no CUDA toolkit is needed, the device layer is a stub that reports itself unavailable, and the device validations print a `[SKIP]` line. `-DOC_BUILD_DOCS=ON` (default `OFF`; needs pdflatex, bibtex, makeindex) adds the targets `make docs` and `make docs_<name>`, which build this manual and the contracts into `<build>/docs-pdf`; they are never part of `all`. `-DOC_BUILD_NOTEBOOK=ON` (default `OFF`) compiles the served console `--serve` (Section 7.8), and `cmake --install` puts the notebook gateway and page under `<prefix>/share/oc-notebook/`. No switch changes a value, and the `--list` count never depends on one: a validation excluded by a switch keeps its name and prints `[SKIP] <name>: OC_BUILD_<X>=OFF` (exit 0). `CMakePresets.json` holds `minimal` (a plain configure), `cpu-only`, `reference` (CUDA `sm_120`, docs and notebook on), `jetson-orin` and `jetson-orin-cpu` (NVIDIA Jetson Orin, aarch64), and `termux` / `aarch64-cpu` (CPU only, `Release -O2`, stripped: Android under Termux and other small aarch64 devices): `cmake --preset cpu-only -B <dir>`. The release's `BUILDING.md` gives the exact commands for the four targets (CUDA, CPU only, Jetson Orin, Termux) and what was tested on each.

The source path is baked in. The binary records the source directory it was configured from (`OC_SOURCE_DIR`); validations that read tracked files (the golden reference, the console scripts, this manual) read them there. Do not delete or move that checkout while a binary built from it is running.

1.4 The command line

`hacs` looks only at its *first* argument to decide what to do; everything after it belongs to the selected front end. The printed top-level usage (`--help`, and the same text when no argument is given) is frozen text; it names every front end, `--atlas` and the console forms included:

```
Usage:
  ./hacs                 With no argument: print this usage text; exit 1.
  ./hacs --help          Print this usage text; exit 0.
  ./hacs --version       Print the version line (`hacs <version> (commit ...; contract ...)`);
  ↪  exit 0.
  ./hacs --list          List all registered validation names with their tier.
  ./hacs --all           Run the fast + standard tiers; exit code = number of failures.
  ./hacs --all-full      Run every tier including slow; exit code = number of failures.
  ./hacs <Name>          Run one registered validation by name; exit code = its own return code.
  ./hacs --bench <mode> <args...>
                                 Run the benchmark CLI (`--bench --help` for modes).
  ./hacs --campaign <spec-file> [options]
                                 Run a parameter sweep (`--campaign --help`;
                                 ↪  docs/matrix-engine/campaign.md).
  ./hacs --atlas <ell> [options]
                                 Build, close, verify or index a residue atlas (`--atlas
                                 ↪  --help`).
  ./hacs --console | -i | --script <file.oc> | -c '<code>' | - [modifiers]
                                 The OC script console (`--console --help`;
                                 ↪  docs/console/console.md).
  ./hacs --serve [options]
                                 The served read-only console, NDJSON on stdio
                                 ↪  (OC_BUILD_NOTEBOOK=ON builds only).
```

The complete dispatch table of `main.cpp`, in the order it is tested:

Token	Arguments	Meaning
`(none)`	–	Prints the top-level usage text (as `--help`) and exits 1, the usage-error code; builds no validation registry. (Until S24 CL-7 it ran the legacy restriction-morphism benchmark.)
`--version`	–	Prints the one-line version `hacs <version> (commit <sha12>; contract [C] <x>, [E] <y>, [U] <z>)` (the `VERSION` file, the configure-time commit, the contract versions) and exits 0; reads nothing else, builds no registry. Metadata only: no golden object, record hash or job id reads it.
`--list`	–	Prints every registered validation as `name<TAB>[tier]`, in registration order; exit 0.
`--all`	`[--shard <i>/<N>] [--list]`	Runs the `fast` and `standard` tiers; exit code = number of failures; `slow` names are skipped and counted.
`--all-full`	`[--shard <i>/<N>] [--list]`	Runs every tier, `slow` included; exit code = number of failures.

Token	Arguments	Meaning
--shard	<i>/<N>	(after --all / --all-full; S24 SHARD) runs shard i of N (1-based, N <= 64) of that tier's roster in this process: a deterministic partition balanced by expected wall time, every device name in one shard, registry order within a shard; the union over i is the roster, each name once. Prints SHARD i/N names=<k> of <total> ... first; exit code = its failures; with --list prints the shard's names instead (exit 0); a malformed i/N is a usage error (exit 1). Driver: docs/matrix-engine/tools/run_all_sharded.sh. Not in the frozen help text.
--calibrate, --calibrate=thorough	[--repeat <R>] [--threads <T>] [--taskset <cpus>] [--profile <path> | --no-write] [--emit-reference]	(NBCONFIRM S3) The per-device speed calibration of the console cost estimates: runs the fixed probe set of the embedded calib rows (six verb anchors inside today's silent envelopes, four fingerprints) as child processes at 1 thread and at min(16, CPUs) threads (=thorough: 3 repeats; default 1, about 20 s on the reference box), prints one CALIB row per probe, NBCONFIRM_CALIB ... verdict=OK and the drift line COST_MODEL_DRIFT ... verdict=OK|DRIFT (meaningful on the reference box only), and writes the device profile ${HACS_DEVICE_PROFILE:-${XDG_CONFIG_HOME:-~/.config}/ (--emit-reference: prints calib rows instead). Advisory only: never a value, a record or a hash. Not in the frozen help text.
<Name>	[<Name> ...]	Runs the named validations in this process, in order; exit code = number of failures (an unknown name is a failure).
--bench	<mode> <args...>	The benchmark CLI: one pipeline point, phase timings, peak RSS and one BENCH line.
--campaign	<spec-file> [options]	The campaign driver: a parameter sweep of Hecke jobs with streamed JSON output.
--atlas	<ell> [options] | --close <f> | --verify <f> | --index <f>	Builds, closes, verifies or indexes a residue atlas Sigma(ell) outside the suite (--atlas --help for its options).
--help, -h	–	Prints the top-level usage text; exit 0.
--console, -i	[modifiers]	Interactive OC session (prompt oc>).
--script	<file.oc> [name=value ...] [modifiers]	Runs an OC script file.
-c	'<code>' [modifiers]	Runs one line of OC script.
-	[modifiers]	Runs the OC script read from standard input (stdin is the script, so no REPL is ever opened).
--serve	[--set key=value ...] [--rss-cap-mib N] [--max-reply-bytes N]	The served console (protocol oc-serve/1): one read-only session driven by NDJSON on stdin / stdout; built only with OC_BUILD_NOTEBOOK=ON (otherwise an unknown token, exit 1).

Token	Arguments	Meaning
`--serve-atlas`	`--store <store.json> [--index <f>] [--expect-fnv 0x<hex>] [--max-reply-bytes N] [--rss-cap-mib N]`	Read-only queries on a `hecke_atlas/v1` store (protocol `oc-serve-atlas/1`) by NDJSON on stdin / stdout: the header, cells, one system, filtered pages of systems; built only with `OC_BUILD_NOTEBOOK=ON` (otherwise an unknown token, exit 1).

Rules that follow from the dispatch.

- With *no* argument the program prints the usage text above and exits with code 1 (the usage error); it builds no validation registry. (Until slot 24 it ran a fixed legacy benchmark.)
- Any first argument that is not one of the tokens above is taken as a *validation name*, and so is every further argument: `./hacs NameA NameB` runs both in one process. An unknown name counts as one failure.
- The console modifiers (`--set`, `--log`, `--quiet`, `--verbose`, `name=value`) are only recognised *after* a console form: `./hacs --script f.oc --set u_ell=off` works, `./hacs --set u_ell=off --script f.oc` looks for a validation called `--set`.
- `--bench micro ...` and a GPU device policy in the environment (`ENGINE_DEVICE=gpu|hybrid`, or `ENGINE_CUDA_PROBE`) install the CUDA hooks; every other run never creates a CUDA context.

Exit codes by front end.

Front end	Exit code
validations, `--all`, `--all-full`	the number of failing validations (the shell sees it modulo 256)
`--list`, `--help`	0
`--bench`	0 success; 1 usage error; 2 an exception, or a `micro` op whose backend is unavailable
`--campaign`	0 success; 1 usage or spec error; 2 at least one job failed
`--atlas`	0 success; 1 usage; 2 `--verify` mismatch; 3 an exception; 5 a non-field coefficient ring in force
console forms	0 ok; 1 usage; 2 parse error; 3 runtime error; 4 failed `assert` / `expect_error`; 5 a legal setting this build cannot execute (Section 7.2.7)

1.5 The validation harness

There is no external test framework. A *validation* is a registered function returning 0 on success; it prints its own `[PASS]`/`[FAIL]` lines. This build registers 289 of them, each with a *tier*:

Tier	Count	Run by
`fast`	75	`--all`, `--all-full`, by name
`standard`	166	`--all`, `--all-full`, by name
`slow`	48	`--all-full`, by name

So `--all` runs 241 validations and `--all-full` all 289. The full roster, sorted by name, is Appendix D; what each one checks is in `docs/validation-catalogue.md`.

- `--list` prints `name<TAB>[tier]` in registration order – the same list the console verbs `validations()` and `validations("fast")` return.

- An exception escaping a validation (an `ENGINE_CHECK`, an out-of-range argument, ...) is reported as that validation's failure with an `[EXCEPTION] <name>: ...` line, and the run continues.
- Validations armed by an environment variable print one `EFFECTIVE <name> arm=<X> cells=<k>` line when the arm actually runs. **Read that line** before trusting a sharded or cluster run: an environment variable that was silently ignored shows up as a missing or vacuous `EFFECTIVE` line.
- Some validations are *interlocked*: they print `[SKIP]` even under `--all-full` unless their variable is set (Section 9.12).
- The Hecke acceptance gate (`HeckeEndToEndValidation`, `HeckeBadPrimeValidation`, `HeckeBenchmarkSmoke`) needs `--all-full` or the three names, since `HeckeBenchmarkSmoke` is `slow`.

From the console, `run_validation("Name")` runs one validation in process and returns its code; the owner-gated emitters refuse there (Section 7.7).

1.6 How the examples in this manual are made

Two kinds of example appear, and neither is ever typed by hand. A *console example* is part of one of the scripts of `src/hacs/Console/scripts/`, which `ConsoleScriptValidation` runs on every `--all`; the output shown is the script's frozen transcript, input from the `.expected` file itself, and a quoted line of the script is checked against the script by `CliManualCoverageValidation`. A *command-line example* ("Real output") is written by `docs/latex/tools/gen_cli_manual_runs.sh`, which runs the command shown on its first line (`$ ...`) against a built binary and appends its exit code; timings and memory figures in it are those of the reference machine and are never part of a result. To run a console script yourself:

```
cd src/hacs/Console/scripts
../../../../build/hacs --script 006_flagship.oc
```

Most scripts begin with `include("prelude.oc")`, which binds the flagship cell $(n, \ell) = (10, 691)$ and checks that the mathematical settings are at their defaults:

From `Console/scripts/prelude.oc`:

```
N_SYM = 10;
ELL = 691;
P_HECKE = 2;
POINT_COH = "n=10 ell=691";
POINT_HECKE = "n=10 ell=691 p=2 r=1";
assert(fingerprint() == "0x0000000000000000",
"the smoke scripts pin the DEFAULT mathematical configuration");
MODEL = sl2z_model();
```

Chapter 2

The mathematical objects

This chapter fixes the vocabulary of the rest of the manual. The definitions are the binding contract's [C]; what follows is a reader's guide to them, with the labels to look up.

2.1 The group, the orbifold and the model

Let $\Gamma = \mathrm{SL}(2,\mathbb{Z})$. It acts on the upper half plane $\mathfrak{H}$ with finite stabilisers: $\pm I$ everywhere, a cyclic group of order 4 at i (generated by $S = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$) and of order 6 at $e^{2\pi i/3}$ (generated by $U = ST$). The quotient $\Gamma\backslash\mathfrak{H}$ is an *orbifold*: a disc with two cone points and a cusp. `hacs` works on one fixed triangulation of it, the *seven-vertex model* (`sl2z_model()`, `[C, rmk:orbifold]`), on which a local system is a coefficient module at each simplex together with the monodromy relations between them. Two facts govern everything else:

- **Away from 2 and 3 the model computes group cohomology.** The finite stabilisers have orders 2, 4 and 6; once 6 is invertible in the coefficients, the equivariant cohomology of the model is $H^\bullet(\Gamma; M)$ (`[C, rmk:orbifold]`, `[C, prop:bredon]`). Over $\mathbb{F}_2$ and $\mathbb{F}_3$ the same complex computes the *Bredon* cohomology of the model, a different object; the group cohomology at $\ell = 2, 3$ is reached by the integral descent (Chapter 5) and by the orbifold census (Section 8.6).
- **The model is finite.** Its cochain complex lives in degrees $0, 1, 2$, while the group cohomology over $\mathbb{Z}$ continues forever in degrees ≥ 2 (periodically, with period 2, through the cup product with an Euler class). Every object therefore records its degrees, its top degree $q_{\max}$ and whether it is truncated.

2.2 Local systems: the symmetric powers

The coefficient module is Sym^n of the standard representation of Γ on R^2: homogeneous polynomials of degree n in two variables, $\gamma \in \Gamma$ acting by substitution, with the convention fixed in `[C, def:rhon]` (ρ_n). Its rank is $d_V = n + 1$. The weight of the corresponding modular forms is $k = n + 2$: by the Eichler–Shimura isomorphism, over a field $\mathbb{F}_\ell$ with $\ell > n + 1$,

$$\dim H^1(\Gamma; \mathrm{Sym}^n \mathbb{F}_\ell^2) \;=\; 2 \dim S_{n+2}(\mathrm{SL}_2(\mathbb{Z})) + 1 \qquad (n > 0 \text{ even}),$$

the cusp forms counted twice and the Eisenstein series once. Because $-I$ acts on Sym^n by $(-1)^n$, **everything vanishes for odd** n (`[C, rmk:negI]`) over a field of characteristic $\neq 2$; use an even n. In the console, `symmetric_power(sl2z_model(), n, ell)` builds the local system; the presets of Section 3.2 put other local systems on other spaces.

2.3 Coefficient rings

One mathematical setting, `coefficient_ring` (Section 7.4.1), chooses where the coefficients live. It is a property of the whole session or campaign, recorded in every result:

Value	What is computed	Where
`F_ell` (default)	vector spaces over the prime field $\mathbb{F}_\ell$ of the verb's ℓ: dimensions, Hecke matrices, decompositions	every verb; Chapters 3, 4, 6
`Z/ell^<k>`	finite $\mathbb{Z}/\ell^k$-modules: lengths and invariant factors, Hecke operators over $\mathbb{Z}/\ell^k$ (`[E, sec:ring]`)	the pipeline verbs and most extension verbs; Section 3.3
`Z`	finitely generated abelian groups $\mathbb{Z}^r \oplus$ tors (`[C, def:ringsemantics]`(3))	the verb `descent` and descent campaigns only; Chapter 5

A verb that does not honour the ring in force stops with exit code 5 and names itself; it never silently computes over another ring. A non-default ring moves the settings fingerprint, so results over different rings never compare equal.

2.4 Levels and characters: $\Gamma_0(N)$ and nebentypus

For a level N, $\Gamma_0(N) \subset \Gamma$ is the subgroup of matrices with lower-left entry divisible by N, of index $\psi(N) = N\prod_{p\mid N}(1+1/p)$. Its cohomology is computed on the *same* seven-vertex model, by Shapiro's lemma: $H^\bullet(\Gamma_0(N); \mathrm{Sym}^n) = H^\bullet(\Gamma; \mathrm{Ind}\,\mathrm{Sym}^n)$, the induced module being the permutation module on the $\psi(N)$ cosets tensored with Sym^n (`[C, sec:levelN]`). A *nebentypus* twists the induced module by a Dirichlet character χ modulo N, given by its Conrey label $N.m$ and reduced into $\mathbb{F}_\ell$. The verbs `level_n(N, n, ell)` and `nebentypus(N, m, n, ell)` return H^0, H^1 and the Hecke operators T_p (and U_p for $p \mid N$) of the level; the descent of Chapter 5 takes a level too. At level 1 the dimension is the Eichler–Shimura count above; at level 11 and weight 2 the two classes of the elliptic curve $X_0(11)$ appear beside one Eisenstein class (the two cusps give $c-1=1$ Eisenstein class in weight 2, so $h^1 = 3 = 2g + (c-1)$):

From `Console/scripts/052_s22_level_n_nebentypus.oc`:

```
A = level_n(1, 10, 7);
print("level 1, n = 10, ell = 7: h1 =", A.h1, " dim S_12 =", A.dim_cusp_forms, "
↪  charpoly T_2 =", A.ops[0].charpoly);
...
B = level_n(11, 0, 7);
print("level 11, n = 0, ell = 7: h0 =", B.h0, " h1 =", B.h1, " index =", B.index, "
↪  cusps =", B.cusps);
...
C = nebentypus(11, 1, 0, 7);
...
D = nebentypus(7, 2, 2, 13);
print("7.2, n = 2, ell = 13: order =", D.order, " h0 =", D.h0, " h1 =", D.h1, "
↪  charpoly T_2 =", D.ops[0].charpoly);
...
E = nebentypus(13, 8, 4, 17);
print("13.8, n = 4, ell = 17: parity_ok =", E.parity_ok, " h1 =", E.h1);
```

Frozen transcript `Console/scripts/052_s22_level_n_nebentypus.expected`:

```
#! oc-format 1
level 1, n = 10, ell = 7: h1 = 3  dim S_12 = 1  charpoly T_2 = [4, 0, 1, 1]
```

```
level 11, n = 0, ell = 7: h0 = 1  h1 = 3  index = 12  cusps = 2
trivial character at (11, 0, 7) agrees with level_n: charpoly T_2 = [2, 6, 1, 1]
7.2, n = 2, ell = 13: order = 3  h0 = 0  h1 = 4  charpoly T_2 = [8, 3, 4, 2, 1]
13.8, n = 4, ell = 17: parity_ok = false  h1 = 0
```

The characteristic polynomials of these verbs are coefficient lists *from the constant term up*: `[4, 0, 1, 1]` is $x^3 + x^2 + 4$ over $\mathbb{F}_7$, i.e. $(x-4)^2(x-5)$ with $4 \equiv \tau(2) = -24$ and $5 \equiv 1 + 2^{11} = 2049$; the trivial character reproduces $\Gamma_0(11)$ operator by operator; a character of the wrong parity ($\chi(-1) \neq (-1)^n$) gives zero.

Chapter 3

Cohomology over a field

The basic computation: build a local system, ask for its cohomology. A pipeline verb returns a lazy *handle*; the work runs on the first access to a computed member (Section 7.2.4).

3.1 Dimensions and the golden reference

The scripts never compare with a copied number: every pin is read from the frozen golden reference with `golden(point, object)`, which refuses at a non-default fingerprint.

From `Console/scripts/001_dimensions.oc`:

```
L = symmetric_power(MODEL, N_SYM, ELL);
H = cohomology(L);
assert(H.degrees == [0, 1, 2], "degrees = [0 .. q_max]");
for q in H.degrees {
assert(H.dims[q] == golden(POINT_COH, "dim_H^" + str(q)).value, "dim H^" + str(q));
assert(H.dim_C(q) == golden(POINT_COH, "dim_C_eq^" + str(q)).value, "dim C^" + str(q));
}
...
print(H.dims, H.q_max, H.truncated, H.q_max_source);
```

Frozen transcript `Console/scripts/001_dimensions.expected`:

```
#! oc-format 1
[0, 3, 0] 2 false derived
```

So $\dim H^\bullet(\Gamma; \mathrm{Sym}^{10}\mathbb{F}_{691}^2) = (0, 3, 0)$: weight $k = 12$, where H^1 is three-dimensional (the Eisenstein class and the two-dimensional cuspidal part of Δ).

3.2 Local systems away from the modular curve

The presets put the equivariant machinery on S^1, the cylinder and the Möbius band, where the answers are theorems of topology:

From `Console/scripts/013_presets.oc`:

```
S1 = s1_scalar(1, ELL_P);
H1 = cohomology(S1);
assert(H1.dims == [1, 1], "H^*(S^1; F_5) with trivial monodromy");
print("s1_scalar(1)", H1.dims);
```

```
...
CY = cohomology(cylinder_scalar(1, ELL_P));
print("cylinder_scalar(1)", CY.dims);
```

Frozen transcript `Console/scripts/013_presets.expected`:

```
#! oc-format 1
s1_scalar(1) [1, 1]
s1_scalar(2) [0, 0]
s1_unipotent [1, 1]
cylinder_scalar(1) [1, 1, 0]
cylinder_unipotent [1, 1, 0]
moebius_scalar(1) [1, 1, 0]
moebius_unipotent [1, 1, 0]
```

Reading it: on S^1 the scalar system with monodromy 1 has the cohomology $(1, 1)$ of the circle, monodromy 2 (a non-trivial scalar over $\mathbb{F}_5$) has none, and the rank-2 unipotent system keeps one invariant and one coinvariant line; the cylinder and the Möbius band retract onto a circle, hence the same answers with a vanishing H^2. `055_s24_language_tour.oc` (Section 7.3) adds the simplicial complexes themselves (`sphere`, `moebius`, `projective_plane`, `complex`).

3.3 Coefficients $\mathbb{Z}/\ell^k$

Under `set coefficient_ring = "Z/ell^2"` (the token `ell` is literal: the verb's own ℓ) the same complex is taken over $\mathbb{Z}/\ell^k$, and H^q becomes a finite $\mathbb{Z}/\ell^k$-module, $H^q \cong \bigoplus_i \mathbb{Z}/\ell^{e_i}$ (`[E, sec:ring]`). The `Cohomology` handle then reports the *lengths* $\sum_i e_i$ (`.lengths`), the exponents (`.invariant_factors(q)`), the reduction map to $\mathbb{F}_\ell$ (`.red(q)`), the Bockstein rank and, when the precision suffices, the $\mathbb{Z}_{(\ell)}$-type of the integral lattice (`.ladic_type(q)`, certified when `.ladic_type_certified` is true). The computation is an echelon form over the local ring $\mathbb{Z}/\ell^k$: a Smith form where the invariant factors are wanted, a Howell form (the canonical row echelon form of a submodule over $\mathbb{Z}/\ell^k$) where a span must be compared. The script `024_e8_ring_points.oc` prints, per point, the lengths of H^0, H^1, H^2, whether the ℓ-adic type is certified, the Bockstein rank and the higher-torsion count:

Frozen transcript `Console/scripts/024_e8_ring_points.expected`:

```
#! oc-format 1
E8 ring=Z/691^2 n=10 ell=691 [0, 6, 0] true 0 0
E8 ring=Z/5^2 n=6 ell=5 [1, 3, 0] true 1 0
E8 ring=Z/5^2 n=16 ell=5 [0, 6, 0] true 0 0
E8 ring=Z/5^2 n=30 ell=5 [2, 12, 0] false 0 1
E8 ring=Z/5^3 n=30 ell=5 [2, 17, 0] true 0 1
E8 ring=Z/5^2 n=60 ell=5 [3, 21, 0] false 1 1
E8 ring=Z/5^3 n=60 ell=5 [3, 30, 0] true 1 1
```

At $(30, 5)$ the precision $\mathbb{Z}/5^2$ is not enough to certify the type (`false`); $\mathbb{Z}/5^3$ is. Hecke operators over $\mathbb{Z}/\ell^k$ are in `025_e8_console_surface.oc` and `026_c5_leftovers_ring_members.oc`, ℓ^k beyond 2^{32} in `028_wide_band.oc`, and the involution ι^* over $\mathbb{Z}/\ell^k$ in `049_lv3_iota_ring.oc`.

3.4 A second model at $\ell = 2, 3$: Moerdijk–Pronk

A second, independent model of the orbifold cohomology of $\mathrm{SL}_2(\mathbb{Z})$ at the primes $\ell = 2, 3$ that divide the orders of the finite subgroups of $\mathrm{SL}_2(\mathbb{Z})$: the Moerdijk–Pronk (MP) simplicial model of

[C, sec:mpmodel]. It is reached *only through validations*; there is no verb, setting or record for it. Ten OrbifoldMP... names, all gating a C++ computation against a frozen independent oracle (a Python script and its recorded output under docs/implementation/slot22/mp/):

Validation	Tier	Checks
OrbifoldMPRulesValidation	fast	the triangulation descriptor, MP rules and the MP set
OrbifoldMPTeardropValidation	fast	MP's teardrop, 9 cells
OrbifoldMPNerveValidation	standard	nerve cochains, $H^{\leq 2}$ over $\mathbb{F}_\ell$ and $\mathbb{Z}/\ell^k$
OrbifoldMPComparisonValidation, OrbifoldMPRefinementValidation	standard	comparison and refinement maps
OrbifoldMPAllDegreesValidation	standard	the all-degrees certificate
OrbifoldMPBoundaryValidation, OrbifoldMPLevelConeValidation	standard	boundary, restriction/corestriction, level pairs
OrbifoldMPHeckeValidation	standard	Hecke T_p on the MP model
OrbifoldMPNerveH3Slow	slow	nerve H^3 at $n \in \{0, 2\}$

Run one by name, e.g. ./hacs OrbifoldMPNerveValidation. Each prints informational lines (MP_RULES, NERVE, INTEGRAL, ORACLE <file> cells=<n> agree=<n>, NC ... for negative controls that must fire) and ends with one summary line

```
ORBIFOLD_MP <Name> checks=<n> failing=0 wall_s=<t>
```

Read failing=0 and the ORACLE ...agree= counts, not only the exit status. OrbifoldMPAllDegreesValidation additionally prints, per model (M7, R_Z), a certificate line

```
MP_ALL_DEGREES model=<M7|R_Z> ring=F_ell nerve_to=2 periodic_on=C_I from=2
↪  via=thm:mpcomparison certified=4/4 scope=bulk verdict=OK
```

Its scope is $\Gamma = \mathrm{SL}_2(\mathbb{Z})$, the bulk only: above degree 2 the cohomology is *transported* by the periodicity theorem, not computed. No MP validation writes a golden line, a record or a setting. The two hooks are ORBIFOLD_MP_CELLS (Section 9.13) and the scratch emitter ORBIFOLD_MP_DUMP (Section 9.12).

Chapter 4

Hecke operators

4.1 The operators and their certificates

For a prime p the Hecke operator T_p is the double coset $\Gamma\,\mathrm{diag}(1,p)\,\Gamma$ acting on cohomology (`[C, def:hecke]`); at $p = \ell$ the same code path gives U_ℓ, for $\ell \le 23170$ in the narrow tier (`[C, lem:bounds]`). The family adds T_{p^r} by the algebra recursion and $T(p,p)$ (`[C, thm:relations]`, `[C, prop:diamond]`). The program computes an operator in three layers, which reappear as the report levels (a)/(b)/(c) of a campaign: (a) the cochain-level matrix $\Theta^{(q)}$, (b) its matrix $A^{(q)}(T_p)$ on H^q, (c) the primary decomposition of H^q over a commuting family. Each layer is certified in process: $\partial\tau = \tau\partial$ for the cochain map, two independent routes agree entry by entry, and a decomposition block is certified *local* (it has exactly one maximal ideal) by the locality certificate (setting `locality`). The verbs are `hecke(H, p)` (one operator), `family(H)` (the commuting family $T_2, T_3, \dots$), `decompose(T, q)` and `report(D)`; relations between operators are checked in `004_family_identities.oc`, certificates in `011_decompose_certify.oc`, the record schema in `012_report_schema.oc`, the strict locality mode in `015_strict_locality.oc`, and the golden pins of $A(T_p)$ in `002_hecke_pins.oc` and `003_bases_and_projectors.oc`. The grid scripts `007_grid_classical.oc` to `010_grid_edge_n.oc` walk the four regimes of the golden grid (classical, Dickson-exceptional, $\ell \in \{2,3\}$, edge cases in n).

4.2 The flagship: T_2 on $(10, 691)$

From `Console/scripts/006_flagship.oc`:

```
T2 = hecke(H, P_HECKE);
A = T2.A(1);
print(charpoly(A));
D = decompose(T2, 1);
print(D.num_blocks);
for b in D.blocks {
print(b.dim, b.residue_degree, b.min_poly(2), b.jordan(2));
}
...
print(D.locality.complete, D.locality.splits, D.locality.phase_reached);
R = report(D);
print(R.q_max, R.truncated, R.q_max_source, R.degrees);
```

Frozen transcript `Console/scripts/006_flagship.expected`:

```
#! oc-format 1
[0, 3, 0]
```

```
X^3 + 72*X^2 + 346*X + 4 (mod 691)
1
3 1 X^2 + 48*X + 576 (mod 691) [2, 1]
true 0 linear
2 false derived [0, 1, 2]
```

Reading it: the characteristic polynomial of T_2 on H^1 is printed over $\mathbb{F}_{691}$; the decomposition of H^1 over $\mathbb{F}_{691}[T_2]$ has one primary block of dimension 3 and residue degree 1, on which T_2 has minimal polynomial $(X+24)^2$ and Jordan type $(2,1)$ – the Ramanujan congruence $\tau(2) \equiv 1 + 2^{11} \pmod{691}$ seen as a non-semisimple Hecke module. The block is certified local by the linear phase without a split, and `report(D)` produces the `hecke_report/v2` record, which covers every degree (`R.degrees` prints `[0, 1, 2]`) and carries the degree-1 decomposition.

4.3 Linear algebra on Hecke operators

From `Console/scripts/005_linear_algebra.oc`:

```
H = cohomology(symmetric_power(MODEL, N_SYM, ELL));
F = family(H);
T2 = F.T(2)[1];
T3 = F.T(3)[1];
d = H.dims[1];
A = algebra([T2, T3]);
assert(in_span(A.basis, mul(T2, T3)), "the algebra is closed under multiplication");
C = commutant([T2, T3]);
...
N = matrix(7, [[0, 1, 0], [0, 0, 1], [0, 0, 0]]);
assert(jordan_type(N) == [3], "one Jordan block of size 3");
expect_error "not nilpotent" { jordan_type(identity(7, 2)); }
...
print(d, A.dim);
```

Frozen transcript `Console/scripts/005_linear_algebra.expected`:

```
#! oc-format 1
3 2
```

The algebra $\mathbb{F}_{691}[T_2, T_3]$ acting on the three-dimensional H^1 is two-dimensional, consistent with the single block of Jordan type $(2,1)$ above.

Chapter 5

Integral cohomology: the descent over $\mathbb{Z}$

Over a field the answer is a list of dimensions. Over $\mathbb{Z}$ it is a list of finitely generated abelian groups, and the interesting information is in their torsion: at 2 and 3, where Γ has finite subgroups, and at finitely many "bad" primes $\ell \geq 5$ tied to congruences. This chapter explains what the program computes over $\mathbb{Z}$, how it does it, how to read the record, and what it does not (yet) do. The mathematics is [C]'s Part on the integral line (`[C, sec:integral]`), the descent (`[C, sec:descent]`, `[C, sec:descentN]`), its Hecke operators (`[C, sec:descenthecke]`) and the boundary pair (`[C, sec:descentpair]`); the one statement that collects what a record certifies is `[C, thm:descentcert]`, and for the boundary pair `[C, prop:descentpair]` (the pair is outside `[C, thm:descentcert]`(7)(c), a separate statement).

5.1 What is computed

For a level N and $n \geq 0$ let $A = \Gamma_0(N)$ and $V = \mathrm{Sym}^n(\mathbb{Z}^2)$. The group cohomology $H^q(A; V)$ is a finitely generated abelian group, so by the structure theorem

$$H^q(A;V) \;\cong\; \mathbb{Z}^{b_q} \;\oplus\; \bigoplus_{\ell} \operatorname{tors} H^q(A;V)_{(\ell)}, \qquad \operatorname{tors} H^q(A;V)_{(\ell)} \cong \bigoplus_i \mathbb{Z}/\ell^{a_i},$$

a free part and, for each prime, an ℓ-primary torsion part. Four facts make this computable (`[C, thm:descentcert]`(1), (6)):

1. **Ranks.** b_q is the dimension over any *good* prime field $\mathbb{F}_{\ell'}$, $\ell' \geq 5$ with no ℓ'-torsion; for $q \geq 2$ it is 0 ($\Gamma_0(N)$ has virtual cohomological dimension 1).
2. **Which primes.** The torsion lives at 2, 3 and at the *bad primes* $\mathcal{B}$, a closed-form finite list: at level 1 and even $n > 0$ these are the primes $5 \leq \ell \leq n-1$ with $\#\{(a,b) \in \mathbb{Z}^2_{\geq 0} : a(\ell+1) + b(\ell^2-\ell) = n\} > 0$, and that count is the number of cyclic summands (`[C, prop:badprimes]`; at level N, `[C, prop:badprimesN]`). At $\ell \geq 5$ the torsion sits in H^1 only. For odd n, 2 kills everything.
3. **Which degrees.** For $q \geq 2$, 24 kills H^q, and the cup product with an Euler class is an isomorphism $H^q \cong H^{q+2}$ of Hecke modules: the degrees $0 \leq q \leq 3$ determine all of them ($q_{\max} = 3$, `truncated = true`, `q_max_source = derived`).
4. **No glueing problem.** The primes do not interact: the arithmetic fracture square $\mathbb{Z} = \mathbb{Z}[1/6] \times_{\mathbb{Q}_2 \times \mathbb{Q}_3} (\mathbb{Z}_2 \times \mathbb{Z}_3)$ gives, degree by degree, H^q as a fibre product of its localisations, with vanishing connecting maps (`[C, prop:fracture]`, `[C, prop:cohglue]`, `[C, prop:descent]`). So the group is assembled from a rank and one ℓ-primary part per prime, each read at that prime alone.

On top of the groups the record carries the Hecke operators T_p, $p \nmid N$: the characteristic polynomial over $\mathbb{Z}$ of T_p on the free quotient H^1/tors, and on each torsion group the isomorphism class of

$(\mathrm{tors}H^q_{(\ell)}, T_p)$ as a module, given by invariants that do not depend on any choice; and the boundary pair (bulk, boundary, compact support).

5.2 Switching it on

The descent is the one consumer of `coefficient_ring = Z`. In the console:

```
set coefficient_ring = "Z"
D = descent(N, n)            # or descent(N, n, [p1, p2, ...]): the Hecke primes
```

or from the command line, the modifier after the console form:

Real output `docs/latex/cli_manual_runs/descent_cli.txt`:

```
$ hacs -c 'print(descent(1, 12).groups)' --set coefficient_ring=Z
["0", "Z + Z/2 + Z/2^2 + Z/2^3 + Z/3 + Z/3^2 + Z/5 + Z/11", "Z/2 + Z/2 + Z/2^2 + Z/3", "Z/2 +
↪  Z/2 + Z/2 + Z/2"]
[exit 0]
```

Under `Z` every other pipeline verb refuses, by name and with exit code 5, rather than compute over another ring:

Real output `docs/latex/cli_manual_runs/descent_refused.txt`:

```
$ hacs -c 'print(cohomology(symmetric_power(sl2z_model(), 12, 5)).dims)' --set
↪  coefficient_ring=Z
<string>:1:18: not implemented in this build: symmetric_power(...) failed
  caused by: hecke::NotImplementedInThisBuild: not_implemented_in_this_build: console verb
  ↪  symmetric_power: coefficient_ring=Z is not honoured by this object (coefficient_ring=Z is
  ↪  honoured by the integral descent only: the descent record, the console verb descent, the
  ↪  campaign's descent cells; [C] thm:descentcert)
  print(cohomology(symmetric_power(sl2z_model(), 12, 5)).dims)
[exit 5]
```

The verb's limits keep a console call interactive: $N \le 300$, $n \le 60$ and $\psi(N)\,(n+1) \le 300$ (at the bound about 40 s and 1.4 GiB); the Hecke primes are 1 to 6 primes ≤ 97 not dividing N, sum ≤ 100 (default: the first three of $2, 3, 5, 7, 11, 13$ not dividing N). A call outside the limits is a type error (exit 3) naming the limit. Larger cells go through a campaign (Section 8.5); their cost is discussed in Section 9.8.

5.3 The algorithm, step by step

The record is assembled from independent pieces, one per prime, each *certified*: a piece either proves that what it read is the true integral answer, or it is refused. Nothing is guessed, and the code uses no arbitrary-precision integers: every computation is in machine words modulo a prime power.

1. The free rank, at a good prime. At a prime $\ell' \ge 5$ with no ℓ'-torsion (by default $\ell' = 10007$) the model computes group cohomology, and $b_q = \dim H^q(A; \mathrm{Sym}^n \mathbb{F}_{\ell'}^2)$ (`[C, prop:certifiede]`). At level N the program does not build the induced module on the model; it uses the *cokernel presentation* of H^1: generators indexed by the cosets, relations from the two finite subgroups $\langle S \rangle$, $\langle U \rangle$, a sparse matrix over $\mathbb{F}_{\ell'}$ (route `sparse`).

2. Torsion at a bad prime $\ell \geq 5$. The same presentation is reduced modulo ℓ^K and brought to *Smith normal form*: invertible row and column operations over $\mathbb{Z}/\ell^K$ make it diagonal, with pivots $u_i\ell^{\sigma_i}$, u_i a unit, $\sigma_i < K$. The cokernel is then visibly $\bigoplus \mathbb{Z}/\ell^{\sigma_i}$ plus a free part. The read is *certified* when the number s of pivots equals the rank of the matrix over $\mathbb{Q}$ (known from step 1): then no pivot of the true integral matrix is hidden in ℓ^K, and $\mathrm{tors}H^1_{(\ell)} = \bigoplus_{\sigma_i>0} \mathbb{Z}/\ell^{\sigma_i}$ exactly (`[C, prop:certifiedeell]`(6), `[C, prop:descentN]`). K is chosen from an a-priori bound and recorded.

3. Torsion at 2 and 3: a free model. Here the finite stabilisers matter and the seven-vertex model computes the wrong object, so the program uses an *ℓ-local free model* of Γ: a complex of free $\mathbb{Z}_{(\ell)}[\Gamma]$-modules whose cohomology is group cohomology. By default it is the *tree model* (`[C, prop:isomodeltree]`): $\mathrm{SL}_2(\mathbb{Z})$ is the amalgam $\mathbb{Z}/4 *_{\mathbb{Z}/2} \mathbb{Z}/6$ of the stabilisers of its tree, so the cochains are the cone of $P^{\langle -I\rangle} \to P^{\langle S\rangle} \oplus P^{\langle U\rangle}$ in closed form, periodic in the higher degrees (route `tree`). Its differentials are brought to Smith form over $\mathbb{Z}/2^{30}$ (re-run in 61-bit words if not certified) and over $\mathbb{Z}/3^{19}$, on two dedicated ring engines, `ring2k` and `ring3k` (`ring2k` also computes Howell forms, the canonical echelon form of a submodule over $\mathbb{Z}/2^k$); the certification rule is the one of step 2 (`[C, prop:certifiedeell]`), and at 3 an uncertified differential is refused. When $\Gamma_0(N)$ has no element of order 3 a shortcut applies at 3 (`[C, prop:descentNthree]`).

4. Hecke operators on the free part: characteristic polynomials by residues. $\chi_p = \det(x - T_p \mid H^1/\mathrm{tors}) \in \mathbb{Z}[x]$ is read modulo several good primes, where it is the characteristic polynomial of the shipped Hecke matrix, and recombined by the Chinese remainder theorem against a certified bound on its coefficients (`[C, prop:crtcharpoly]`, `[C, prop:crtcharpolyN]`). A coefficient that the residues do not determine is reported as `"?"`, never guessed.

5. Hecke operators on torsion: an exact read. On a certified Smith form the generators of the torsion are known modulo $\ell^{K-\sigma_{\max}}$; applying T_p to them gives the module $(\mathrm{tors}H^q_{(\ell)}, T_p)$ exactly provided $2\sigma_{\max} \leq K$, and that condition is necessary (`[C, prop:torsionhecke]`(3)). With a weaker precision only the layer polynomials and the Fitting types are certified (`module_certified = false`). The module is then described by *choice-free invariants* (Section 5.4), never by a matrix, because a matrix depends on the generators chosen (`[C, prop:torsionhecke]`(1), (2)).

6. The boundary pair. The Borel–Serre compactification adds a boundary (one circle per cusp). The record computes, by the same descent, the cohomology of the bulk H^q, of the boundary $H^q(\partial)$ and with compact support H^q_c (the cone of restriction to the boundary), $q \leq 4$, glued in the long exact sequence $\cdots \to H^q_c \to H^q \to H^q(\partial) \to H^{q+1}_c \to \cdots$ (`[C, prop:descentpair]`).

Status at $\ell = 2, 3$, and what is not computed. At 2 and 3 the groups, the Hecke torsion modules and the pair are computed for every level within the limits, and each piece is either certified or refused, never wrong (at 3 the precision cap is $K = 19$ and an uncertified differential is refused, Section 5.3, step 3; a memory prediction can refuse too, Section 5.4); at $\ell \geq 5$ likewise at every bad prime. *Not* in the record (`[C, thm:descentcert]`(7)): U_p for $p \mid N$ (deferred: a prime dividing N is refused), a nebentypus, groups other than $\Gamma_0(N)$, integer Hecke matrices in a basis, the Hecke-module structure of H^1/tors beyond its characteristic polynomial, and the extension class of $0 \to \mathrm{tors} \to H^1 \to H^1/\mathrm{tors} \to 0$.

The second route at level 1: the orbifold census. The descent covers every level N but only $n \leq 60$. For level 1 and large n at $\ell \in \{2, 3\}$ a second route computes the same group cohomology of $\mathrm{SL}_2(\mathbb{Z})$: the orbifold census (Section 8.6), on the isotropy model C_I, cross-checked against the

amalgam resolution $\mathbb{Z}/4*_{\mathbb{Z}/2}\mathbb{Z}/6$. Where both run they agree: `IntegralDescentGroupsValidation` compares the descent at level 1 with an independent C_I computation up to degree 4.

5.4 Reading the record

`descent(N, n, primes)` returns a lazy `DescentRecord` (schema `integral_descent_record/v1`, `[C, sec:recordschema]` item 7; members in Appendix B).

Groups. `.groups` lists $H^0, \ldots, H^3$ as strings: `"0"` for the trivial group, otherwise `Z` (rank 1) or `Z^r` ($r \geq 2$) for the free part followed by the cyclic summands $\mathbb{Z}/\ell^a$ (written `Z/l^a`, `Z/l` for $a = 1$), joined by `+`, ordered by prime and then by exponent; a summand that occurs several times is repeated. `.degrees` is `[0, 1, 2, 3]`; the groups are empty when the record is refused. The same groups in machine form are `.groups_inv` (and `.pair_inv` for the pair): one `{free_rank, torsion}` per degree, `torsion` a list of `[p, k, mult]` meaning `mult` summands $\mathbb{Z}/p^k$; the notebook renders these, scripts may prefer them to parsing the strings.

Primes and precision. `.bad_primes` is $\mathcal{B}$ as a list of `[ell, count]` (the count of cyclic summands at ℓ); `.torsion_primes` the primes whose torsion was read (2, 3 for even n, and $\mathcal{B}$); `.K` one entry per prime, `{ell, route, K, W, certified}`: the route (`tree` at 2 and 3; `shortcut` at 3 when $\nu_3(N) = 0$, i.e. when $\Gamma_0(N)$ has no element of order 3 — the default there, so the worked level 11 reads `shortcut` (`[C, prop:descentNthree]`, Section 5.3, step 3); `sparse` at $\ell \geq 5$; `dense` and `bredon` only under the rollback arms of Section 9.8), the precision K of $\mathbb{Z}/\ell^K$, the word width W in bits, whether the read certified. `.K` is provenance: it is not part of the record's hash.

Characteristic polynomials. `.charpoly` has one entry per Hecke prime: `{p, r, coefficients, exact, determined, digest, refused}`, $r = \operatorname{rank} H^1/\mathrm{tors}$, and the coefficients of χ_p *from the leading one down*, each a string: a decimal integer, `"[sign,D digits]"` for an integer beyond 2^{63} (its sign, and D = the number of mixed-radix digits of its CRT reconstruction, one per word prime, each $< 2^{32}$ — *not* the number of decimal digits), or `"?"` when not determined. `exact` (since 1.5.4) is the same list with *every* coefficient as an exact decimal integer (`"?"` when not determined): the record carries each coefficient beyond 2^{63} exactly (`[C, sec:recordschema]` item 7 (l); in a campaign cell file the entry's list `big` of canonical signed hexadecimal integers, decoded by `docs/integral_census/icensus_exact.py`). (The verbs of Section 2.4 print coefficient lists the other way round, from the constant term up.) `.charpoly_factored` gives the same χ_p factored over $\mathbb{Z}$: per prime `{p, status, expanded, factored, factors, complete, verified, note}`, e.g. `factored = "(X + 24)^2 (X - 2049)"` at $(N, n) = (1, 10)$, $p = 2$, and `factors` a list of `{coefficients, degree, multiplicity, irreducible}`. The factorisation is computed in the program (Yun's squarefree decomposition, then Zassenhaus: Hensel lifting and recombination), exactly, and the product of the factors is checked against χ_p; a coefficient beyond 2^{63} is read from the record's exact value. The factorisation is presentation only: the record's JSON, digest and golden points do not contain it.

The torsion modules. `.torsion` has one entry per (q, ℓ) with $q \in \{1, 2, 3\}$: `{q, ell, exponents, module_certified, ops, refused}`, where `exponents` are the a_i and `ops` has one `{p, invariants}` per Hecke prime. The `invariants` string of $(G, \phi) = (\mathrm{tors}H^q_{(\ell)}, T_p)$ has up to six fields separated by `;`, each defined in `[C, prop:torsionhecke]`(1). The fields `ker=` and `cok=` are *present only* when `module_certified` is true, G has at most 400 cyclic summands and the ring's precision cap is at least $a_{\max} + 2$; otherwise the string is `a=;P=;B=;fit=` (four fields). A `B=` block of more than 300 summands reads $j{:}e{:}$`-` (not computed, never guessed).

Field	Meaning
`a=`$a_1.a_2.\cdots$	the type of G: $G \cong \bigoplus_i \mathbb{Z}/\ell^{a_i}$, exponents ascending
`P=`j`:`$c_0, c_1, \ldots$`|...`	per layer j (the summands of order ℓ^j): the characteristic polynomial P_j of ϕ on the layer $V_j = (\ell^{j-1}G)[\ell]/(\ell^j G)[\ell]$, over $\mathbb{F}_\ell$, coefficients from the leading one down ((a))
`B=`j`:`e`:`$c_0, \ldots$`|...`	per layer j: the characteristic polynomial of the layer's diagonal block modulo ℓ^e, $e = e_j = \min(j, d_j)$ the precision at which it is an invariant (d_j the distance to the nearest other exponent); its reduction mod ℓ is P_j ((e))
`ker=`c`:type|...`	for every $c = 0, \ldots, \ell^{a_{\max}} - 1$: the type of $\ker(\phi - c)$ ((c))
`cok=`c`:type|...`	the same for $\operatorname{coker}(\phi - c)$
`fit=`c`:type|...`	for every $c = 0, \ldots, \ell - 1$: the type of the Fitting component $G_{[c]} = \ker(\phi - c)^{\operatorname{len} G}$, the part on which $\phi \equiv c$ is nilpotent ((b))

A type is written like `a`: exponents joined by `.`, and `-` for the trivial group. When the module is not certified (Section 5.3, step 5) the read certifies only the layer polynomials (`P=`), the Fitting types (`fit=`) and the blocks modulo $\ell^{\min(e_j, K')}$, $K' = K - \sigma_{\max}$ (`[C, prop:torsionhecke]`(3)). The same text is produced by the independent Python oracle `docs/implementation/slot24/d3_hecke_oracle.py`, which is how it is tested.

The pair, status, identity. `.pair` holds `bulk`, `boundary` and `compact`, each with its own `groups` for $q = 0, \ldots, 4$ (`compact` also `derived_from = [bulk, boundary]`). `.status` is `ok` or `refused`, `.refusal` the first refused part's reason (a precision cap or a memory prediction: refused, never wrong); `.digest` is `record_fnv1a64`, the FNV-1a-64 of the record's invariants and the settings' canonical text; `.json` the record as one JSON line. Printing the record itself gives a one-line summary.

5.5 Worked examples

The script `054_s24_descent.oc` reads three cells and four refusals:

From `Console/scripts/054_s24_descent.oc`:

```
set coefficient_ring = "Z";
D = descent(1, 12);
print("descent(1, 12):", D.status);
for q in D.degrees {
print("  H^" + str(q) + " =", D.groups[q]);
}
print("  bad primes [ell, count]:", D.bad_primes, " torsion primes:",
↪  D.torsion_primes);
...
t = D.torsion[0];
print("  tors H^" + str(t.q) + " at ell =", t.ell, ": exponents", t.exponents, " module
↪  certified:", t.module_certified);
print("  T_" + str(t.ops[0].p) + ":", t.ops[0].invariants);
...
E = descent(11, 0);
print("descent(11, 0):", E.groups);
print("  bulk    :", E.pair.bulk.groups);
print("  boundary:", E.pair.boundary.groups);
print("  compact :", E.pair.compact.groups);
...
F = descent(11, 2, [2, 3]);
...
expect_error "n must satisfy 0 <= n <= 60" { descent(1, 400) }
expect_error "must not divide N" { descent(4, 2, [2]) }
```

Frozen transcript `Console/scripts/054_s24_descent.expected`:

```
#! oc-format 1
descent(1, 12): ok
  H^0 = 0
  H^1 = Z + Z/2 + Z/2^2 + Z/2^3 + Z/3 + Z/3^2 + Z/5 + Z/11
  H^2 = Z/2 + Z/2 + Z/2^2 + Z/3
  H^3 = Z/2 + Z/2 + Z/2 + Z/2
  bad primes [ell, count]: [[5, 1], [11, 1]]  torsion primes: [2, 3, 5, 11]
  ell = 2  route = tree  K = 30  certified = true
  ell = 3  route = tree  K = 19  certified = true
  ell = 5  route = sparse  K = 13  certified = true
  ell = 11  route = sparse  K = 8  certified = true
  char_Z(T_2 | H^1/tors) coefficients (high -> low): ["1", "-8193"]
  char_Z(T_3 | H^1/tors) coefficients (high -> low): ["1", "-1594324"]
  char_Z(T_5 | H^1/tors) coefficients (high -> low): ["1", "-1220703126"]
  tors H^1 at ell = 2 : exponents [1, 2, 3]  module certified: true
  T_2: a=1.2.3;P=1:1,0|2:1,0|3:1,0;B=1:1:1,0|2:1:1,0|3:1:1,0;ker=0:1.3|1:-|2:1.3|3:-|
  ↪  4:1.3|5:-|6:1.3|7:-;cok=0:1.1.2|1:-|2:1.1.2|3:-|4:1.1.2|5:-|6:1.1.2|7:-;fit=0:1
  ↪  .2.3|1:-
  record digest: 0x6c841fb033fbd3d1
descent(11, 0): ["Z", "Z^3", "Z/2", "Z/2 + Z/2 + Z/2"]
  bulk    : ["Z", "Z^3", "Z/2", "Z/2 + Z/2 + Z/2", "Z/2"]
  boundary: ["Z^2", "Z^2", "Z/2 + Z/2", "Z/2 + Z/2", "Z/2 + Z/2"]
  compact : ["0", "Z^3", "Z", "Z/2 + Z/2 + Z/2", "Z/2"]
  char_Z(T_2): ["1", "1", "-8", "-12"]
  char_Z(T_3): ["1", "-2", "-7", "-4"]
  char_Z(T_5): ["1", "-8", "13", "-6"]
descent(11, 2, [2, 3]): ["0", "Z^6 + Z/2", "Z/2", "Z/2 + Z/2 + Z/2 + Z/2 + Z/2 + Z/2
↪  + Z/2"]
  char_Z(T_2): ["1", "-22", "153", "-316", "-140", "576", "324"]
  char_Z(T_3): ["1", "-52", "470", "7988", "-57823", "-271096", "1731856"]
refusals: ok
```

The Hecke structure over $\mathbb{Z}$ (`hecke_structure`). The verb `hecke_structure(N, n, algebra = "full")` (slot 25, HZ-REC) returns the record `integral_hecke_structure/v1` of a cell (`[C, sec:recordschema]` item 8): per prime ℓ whether $\mathbb{T} \otimes \mathbb{Z}_\ell$ is certified to be generated by the Hecke operators (and by which route), and the congruence module $C_{\mathbb{T}}$ between the Eisenstein and the cuspidal part, `certified` only where its lower and upper bounds meet. Like `descent` it needs `coefficient_ring = "Z"`. The script `060_s25_hecke_structure.oc` reads $(1, 10)$ ($C_{\mathbb{T}} = \mathbb{Z}/691$), $(1, 22)$ and the anemic algebra at $(11, 0)$; a campaign writes the same records with `structure = hecke` (keys `algebra`, `hz_cert`).

Level 1, $n = 12$ (weight 14). $S_{14}(\mathrm{SL}_2(\mathbb{Z})) = 0$, so $H^1/\mathrm{tors} = \mathbb{Z}$ is spanned by the Eisenstein class, on which T_p acts by $1 + p^{13}$: $\chi_2 = x - 8193$, $\chi_3 = x - 1594324$, $\chi_5 = x - 1220703126$. The bad primes follow from the count of Section 5.1: at $\ell = 5$, $6a + 20b = 12$ has the one solution $(2, 0)$; at $\ell = 11$, $12a + 110b = 12$ has $(1, 0)$; at $\ell = 7$, $8a + 42b = 12$ has none. Hence one summand $\mathbb{Z}/5$ and one $\mathbb{Z}/11$ in H^1: the count is the dimension of the Γ-invariants of $\mathrm{Sym}^{12}\mathbb{F}_\ell^2$ (Dickson invariants of degree 12), and each invariant line becomes a cyclic summand of H^1 through the Bockstein (`[C, prop:badprimes]`). The 2- and 3-parts come from the finite subgroups, and H^2, H^3 are killed by 24. The `K` lines show which route read each prime at which precision.

Reading an invariants string. For T_2 on $G = \mathrm{tors}H^1_{(2)} \cong \mathbb{Z}/2 \oplus \mathbb{Z}/4 \oplus \mathbb{Z}/8$ (`a=1.2.3`): every layer is one-dimensional and `P=1:1,0|2:1,0|3:1,0` says $P_j = x$ on each, so T_2 is nilpotent modulo 2

on every layer; `B` gives the same at precision $e_j = 1$ (the exponents $1, 2, 3$ are at distance 1). `ker=0:1.3|1:-|2:1.3|...` says $\ker(T_2 - c) \cong \mathbb{Z}/2 \oplus \mathbb{Z}/8$ for even c and 0 for odd c, and `cok=` the cokernels $\mathbb{Z}/2 \oplus \mathbb{Z}/2 \oplus \mathbb{Z}/4$ (the orders agree, 2^4, as they must for an endomorphism of a finite group). `fit=0:1.2.3|1:-`: the whole group is the Fitting component at $c \equiv 0$, i.e. T_2 is topologically nilpotent on the 2-torsion, and $T_2 - 1$ is invertible there.

Level 11, $n = 0$ (weight 2). $H^0 = \mathbb{Z}$, $H^1 = \mathbb{Z}^3$: the two classes of the elliptic curve $X_0(11)$ and one Eisenstein class. $\chi_2 = x^3 + x^2 - 8x - 12 = (x+2)^2(x-3)$: the eigenvalue $a_2 = -2$ of the curve (twice) and $1 + 2 = 3$ on the Eisenstein class; likewise $\chi_5 = (x-1)^2(x-6)$. The pair shows the two cusps: $H^0(\partial) = H^1(\partial) = \mathbb{Z}^2$, and in the long exact sequence the restriction $H^1 \to H^1(\partial) = \mathbb{Z}^2$ has rank 1 (the Eisenstein class): its kernel is the rank-2 cuspidal part, $H^1_c = \mathbb{Z}^3$ is that kernel plus the image of $H^0(\partial)/H^0 = \mathbb{Z}$, and $H^2_c = \mathbb{Z}$ is the fundamental class.

Level 11, $n = 2$ (weight 4). $\operatorname{rank} H^1 = 6 = 2 \dim S_4(\Gamma_0(11)) + 2$ (two cusp forms, two cusps); the characteristic polynomials have degree 6, their integer coefficients reconstructed from residues by the CRT step of Section 5.3. The only torsion of H^1 in this cell is one $\mathbb{Z}/2$.

Refusals. The limits and $p \mid N$ are type errors; outside `coefficient_ring = Z` the verb asks for the ring; and under `Z` the field verbs refuse with exit code 5 (here caught by `expect_error`).

5.6 From the command line and in campaigns

A grid of descent cells, with one JSON record per cell and a summary table, is a campaign over $\mathbb{Z}$ (Section 8.5). Its records are the same `integral_descent_record/v1` as the verb's. The golden reference pins nine descent cells (156 tier-A entries), so a change to a record is a re-baseline.

Chapter 6

Extensions, Hida theory and the forms algebra

The extension layer of [E] studies H^1 over $\mathbb{F}_\ell$ as a module over its Hecke algebra $\mathbb{T}$, not just its eigenvalues: congruences between an Eisenstein series and a cusp form, or between cusp forms, show up as *non-split* extensions, i.e. Jordan blocks of the Hecke operators. The verbs return records; the frozen scripts below show each one.

6.1 The extension layer

The chain of objects is: the commuting family $T_2, T_3, \ldots$ (with U_ℓ when the memory model admits it, setting `u_ell`); the *maximal ideals* $\mathfrak{m}$ of the algebra they generate (the mod-ℓ eigensystems, in a canonical splitting field `GF`); for each $\mathfrak{m}$ the *local Hecke algebra* $\mathbb{T}_\mathfrak{m}$ and the localised module $H^1_\mathfrak{m}$ with its Loewy and socle series, its presentation and its extension classes; and the Sturm bound B' that proves a finite family generates $\mathbb{T}$ (`[E, def:bprime]`). `ext_record(n, ell)` assembles all of it into one record per cell, optionally with the cone and boundary cohomology and the ι-sign split (ι^* the involution of `[C, def:iota]`; the plus part contains the Eisenstein side). The local-algebra invariants (`radical`, `socle`, `loewy`, `tangent_space`, `nilpotency_index`, `is_local`, ...) apply to any commutative algebra of operators (`047_c5_local_algebra_verbs.oc`).

6.2 Sturm bounds and the U_ℓ budget

From `Console/scripts/016_extensions.oc`:

```
S = sturm_bound(24, 5);
print(S.k, S.ell, S.B_prime, S.dim_M_k, S.closure_dim, S.primes_done, S.cap);
CUT = sturm_bound(24, 5, 3);
print(CUT.outcome, CUT.closure_dim, CUT.dim_M_k, CUT.primes_done);
CONT = resume(CUT, 36);
print(CONT.B_prime, state(CONT).resumable.progress.budget_history);
V = u_ell_decision(16, 43867, 2048);
print(V.included, V.reason, V.predicted_mb, V.budget_mb, V.model_version);
W = u_ell_decision(10, 691, 2048);
print(W.included, W.reason, W.predicted_mb);
set u_ell = "off";
OFF = u_ell_decision(10, 691);
print(OFF.included, OFF.reason);
```

Frozen transcript `Console/scripts/016_extensions.expected`:

```
#! oc-format 1
24 5 5 3 3 [2, 3, 5] 36
none_at_cap 2 3 [2, 3]
5 [3, 36]
false u_ell_over_budget 8254.76 2048 u_ell_rss/v2-2026-09-17
true within_budget 104.06
false off
```

The computed Sturm bound at weight 24 and $\ell = 5$ is $B' = 5$ (`[E, def:bprime]`): T_2 and T_3 alone generate a proper subalgebra, which is why the cap-3 run ends `none_at_cap`. The U_ℓ decision is a frozen memory model: U_{43867} at $n = 16$ is refused against a 2048 MB budget, U_{691} at $n = 10$ fits, and `u_ell = "off"` refuses without forming a prediction.

6.3 The extension record of $(10, 691)$

From `Console/scripts/040_c5_ext_record_10_691.oc`:

```
R = ext_record(N_SYM, ELL, cone = true, sign_split = true);
print(R);
m = R.ideals[0];
print(m);
T = local_algebra(m);
print(T);
P = module_structure(m);
print(P);
X = P.structure_map(0);
print(X);
Q = presentation(m);
print(Q);
plus = R.ideals_plus;
print(plus[0], R.ideals_minus[0]);
```

Frozen transcript `Console/scripts/040_c5_ext_record_10_691.expected`:

```
#! oc-format 1
ExtRecord (10, 691): 1 maximal ideal(s)
MaximalIdeal #0 (all): dim 3, f = 1, local
HeckeAlgebra: dim 2 over F_691, f = 1, embdim 1, nu = 2, Gorenstein
LoewyProfile on H1: loewy [2, 1], socle [2, 1], not cyclic
ExtensionClass layers (0, 1): rank 1 (non-split)
Presentation: 2 generator(s), 1 relation(s)
MaximalIdeal #0 (plus): dim 2, f = 1, local MaximalIdeal #0 (minus): dim 1, f = 1,
↪  local
```

One maximal ideal $\mathfrak{m} = (T_2 + 24)$, certified local, with $\mathbb{T}_\mathfrak{m} = \mathbb{F}_{691}[\varepsilon]/(\varepsilon^2)$; $H^1_\mathfrak{m}$ has Loewy and socle series $(2, 1)$ and is not cyclic; the extension class between the two layers is non-split; and the ι-sign records put a two-dimensional block in H^1_+ (the Eisenstein-cuspidal congruence) and the cuspidal line in H^1_-. That this block is non-split is not printed for the plus record; it follows because a Jordan block of size 2 cannot lie in the one-dimensional H^1_-.

6.4 The Dickson sequence, cone, boundary and ι

At the *Dickson primes* (those ℓ for which $\mathrm{Sym}^n \mathbb{F}_\ell^2$ has Γ-invariants, the Dickson invariants) H^0 does not vanish, and the Dickson long exact sequence and the map θ relate Sym^n to $\mathrm{Sym}^{n-\ell-1}$ (`[E, sec:thetaobj]`): verbs `dickson_les`, `theta`, `dickson_count`, script `027_dickson_theta.oc`. The cone and boundary cohomology (`cone`, `boundary`) and the involution (`iota`) appear in `042_c5_dickson_class_cone.oc`, `043_c5_boundary_coincidences.oc`, `044_c5_multiplicity_two.oc`, `045_c5_pd_loewy_reversal.oc` and `046_c5_identities.oc`; the Kummer cells of [E]'s Stage 9 in `041_c5_stage9_kummer_cells.oc`; the canonical fields in `047_c5_local_algebra_verbs.oc`; the Hecke algebra over $\mathbb{Z}/\ell^K$ with its certificate (`[C, cor:stablecert]`) in `029_hecke_ring_algebra.oc`; the predictions in `017_predictions.oc`; the phase scripts `018_e0_locality_grid.oc`, `019_e1_stage1_sturm.oc`, `020_e2_ramanujan_10_691.oc` and the duality pairing in `023_e7_duality_10_691.oc`.

6.5 Hida theory, Λ-adic objects and the forms algebra

Hida theory follows the *ordinary* part (where U_ℓ is a unit) across weights $k \equiv k' \pmod{(\ell-1)\ell^{m-1}}$ and up the tower $\Gamma_1(\ell^r)$. `hida_tower(n, ell, m)` reads the ordinary rank and the Hecke polynomials at the levels $\mathbb{Z}/\ell^e$, $e \le m$; `hida_lambda(ell, k, W)` the ordinary part of the forms over $\mathbb{Z}/\ell^W$ and its Eisenstein component; `forms_algebra(k, ell)` the cusp forms S_k over $\mathbb{F}_\ell$ by q-expansions, an independent check of the cohomology; `pow` and `kummer_unit` the matrix powers and the cyclotomic units behind the Kummer criterion $\ell \mid B_i$; and `lfunction` the ℓ-adic L-function of a branch (`[E, sec:lvlfunction]`).

From `Console/scripts/051_s21_hida_tower.oc`:

```
T = hida_tower(10, 11, 2);
print("h =", T.h, " e_max =", T.e_max, " K =", T.K, " exps =", T.exps);
for L in T.levels {
print("e =", L.e, " ord_rank =", L.ord_rank, " readings agree:", L.readings_agree, "
↪  charpoly T_2 =", L.charpoly["2"]);
```

Frozen transcript `Console/scripts/051_s21_hida_tower.expected`:

```
#! oc-format 1
h = 3  e_max = 0  K = 2  exps = [2, 2, 2]
e = 1  ord_rank = 3  readings agree: true  charpoly T_2 = [10, 3, 1, 1]
e = 2  ord_rank = 3  readings agree: true  charpoly T_2 = [10, 113, 56, 1]
HIDA_TOWER ell=11 class=0 m=1 n=10 ord_rank=3 charpoly_T2=[10,3,1,1]
↪  charpoly_T3=[7,4,9,1]
```

The ordinary part of $H^1(\mathrm{Sym}^{10})$ at $\ell = 11$ has rank 3 at both levels, and the polynomial of T_2 over $\mathbb{Z}/11$ lifts to one over $\mathbb{Z}/11^2$ (coefficients from the constant term up).

From `Console/scripts/053_s22_forms_hida_lambda.oc`:

```
F = forms_algebra(12, 7);
print("S_12 over F_7: d =", F.d, " basis =", F.basis[0], " charpoly T_2 =",
↪  F.ops[0].charpoly);
...
H = hida_lambda(11, 12, 3, [2, 3]);
```

Frozen transcript `Console/scripts/053_s22_forms_hida_lambda.expected`:

```
#! oc-format 1
S_12 over F_7: d = 1  basis = [0, 1, 4, 0, 5, 0, 0, 0]  charpoly T_2 = [3, 1]
S_24 over F_7: d = 2  charpoly T_2 = [6, 5, 1]
e S_12 over Z/11^3: rank = 1  charpoly T_2 = [24, 1]  U = [450, 1]  T_2 on Eisenstein
↪  = 718
```

S_{12} is spanned by $\Delta = q - 24q^2 + 252q^3 - \cdots$, whose reduction mod 7 is the printed basis, and T_2 acts by $\tau(2) = -24 \equiv 4$ (the polynomial $x + 3$). Over $\mathbb{Z}/11^3$ the ordinary cusp part has rank 1 and T_2 acts on the Eisenstein part by $1 + 2^{11}$. The Kummer units are in `050_s21_pow_kummer_unit.oc`:

Frozen transcript `Console/scripts/050_s21_pow_kummer_unit.expected`:

```
#! oc-format 1
ell = 37 i = 32: v_pi = 38  ell | B_i: true  oracle agrees: true  not an ell-th
↪  power: true
ell = 37 i = 30: v_pi = 30  ell | B_i: false  oracle agrees: true
```

$37 \mid B_{32}$ (37 is irregular): $v_\pi(\eta^{36} - 1) = 38 > 32$ and η_{32} is not a 37-th power; at $i = 30$, $37 \nmid B_{30}$ and $v_\pi = 30$, as Kummer's criterion predicts. The ℓ-adic L-function is in Section 7.3.

6.6 The residue atlas: `--atlas`

`--atlas` builds the residue atlas $\Sigma(\ell)$ (the anemic level-one eigensystems over weights $\leq \ell + 1$, their θ-twists and a separating prime set) outside the validation suite. It is not listed by the top-level `--help`.

```
usage: hacs --atlas <ell> [--n-max N] [--collect-only] [--quiet]
       hacs --atlas --close <store.json>
       hacs --atlas --verify <store.json>
       hacs --atlas --index <store.json> [--out <path>]
       hacs --atlas --query <store.json> '<request JSON>'   (OC_BUILD_NOTEBOOK=ON)
The store is written only under HECKE_ATLAS_OUT=<dir> (never under
docs/matrix-engine/results/).  --index writes the oc-atlas-index/1
sidecar (default <store.json>.ocidx; never the store itself, never
inside the source tree).  See HeckeExt/HeckeAtlasCli.h.
```

Token	Arguments	Meaning
`<ell>`	–	Builds Sigma(ell) over every even n <= min(ell - 1, N); one ATLAS_CELL line per cell. Requires 5 <= ell <= 100000 (otherwise usage, exit 1).
`--n-max`	`<N>`	Caps the weight range of the build.
`--collect-only`	–	Stops after the collection (no closure, no S_sep) and writes the `.collected.json` store.
`--quiet`	–	Suppresses the per-cell lines.
`--close`	`<store.json>`	Re-derives the closure and S_sep of a store without the pipeline.
`--verify`	`<store.json>`	Re-closes a closed store and compares byte for byte (exit 0 iff identical).

Token	Arguments	Meaning
`--index`	`<store.json>`	Writes the `oc-atlas-index/1` sidecar of a store (default `<store.json>.ocidx`), keyed to the store's size and FNV; the store is opened read-only; an output that is the store itself, inside the source tree or under `docs/matrix-engine/results/` is refused (exit 1); a store that is not indexable exits 3.
`--query`	`<store.json> '<request JSON>'`	One-shot read-only query on a `hecke_atlas/v1` store for scripts (slot-22 VERB-3, owner decision O13): the request handler of `--serve-atlas` (`info`, `cells`, `cell`, `system`, `systems`; same validation and filters) run once, one `atlas_reply` frame on stdout; the store is opened read-only and its path comes from the command line only. Exit 0 reply `ok`, 1 usage or a refused store, 3 an `error` / `rejected` reply; built only with `OC_BUILD_NOTEBOOK=ON` (otherwise exit 5).
`--out`	`<path>`	(`--index`) the sidecar's path instead of `<store.json>.ocidx`.
`--help`, `-h`	–	Prints the atlas usage.

The store file is written only when `HECKE_ATLAS_OUT=<dir>` is set, and never under `docs/matrix-engine/results/`. Every run prints an `ATLAS_SUMMARY` line and an `ATLAS_STORE` line with the FNV-1a-64 of the store text. The atlas is an $\mathbb{F}_\ell$ object: under a non-field `coefficient_ring` the command exits 5. In the console, `atlas(ell)` builds $\Sigma(\ell)$ in process for $\ell \in \{5, 7, 11, 13\}$.

One-shot queries. `--atlas --query <store.json> '<request JSON>'` answers a single read-only request against a `hecke_atlas/v1` store and exits, for scripts that do not want a server: the request is handled exactly as by `--serve-atlas` (`info`, `cells`, `cell`, `system`, `systems`; same validation, same filters) and one `atlas_reply` frame is written to standard output. The store is opened read-only and its path comes from the command line only. Exit 0 for an `ok` reply, 1 for usage or a refused store, 3 for an `error` or `rejected` reply; without `OC_BUILD_NOTEBOOK=ON` it exits 5. A blank or multi-line request is refused.

The index sidecar. `--atlas --index <store.json>` writes the byte-offset index `oc-atlas-index/1` of a store beside it (`<store.json>.ocidx`, or `--out <path>`): one line per cell and per system with its offset, length and the attributes the notebook filters on (r, type, collected, base, twist), the occurrences of the collected systems, the store's size and FNV-1a-64, and a closing hash of the index itself ([P] v3.11 Section 18.16). The store is opened read-only; an output that is the store itself, lies inside the source tree or under `docs/matrix-engine/results/` is refused (exit 1); a file that is not a `hecke_atlas/v1` store in the layout the build writes exits 3. The sidecar is the owner's one-off for the large $\Sigma(691)$ store; `--serve-atlas` (Section 7.9) reads it.

Chapter 7

The console and its language

7.1 The console

The console is a small deterministic expression language, *OC script*, over the whole pipeline. Its binding design is [P]; `docs/console/console.md` is its generated Markdown reference. It has four invocation forms and a few modifiers:

```
OC script console (console phase C1a).
Usage:
  hacs --console | -i          Interactive session (prompt 'oc> ').
  hacs --script <file.oc> [name=value]...
                                       Run a script; parameters are read as args.name.
  hacs -c '<code>'             Run one line of OC script.
  hacs -                       Run the script on standard input.
Modifiers:
  --set key=value                       Apply a session setting before the first statement.
  --log <file>                          Append the transcript (inputs prefixed '> ').
  --quiet | --verbose                   The pipeline's own chatter off / on.
  --script <file> --console             Run the script, then drop into the REPL.
  --help                                This text.
Exit codes: 0 ok, 1 usage, 2 parse error, 3 runtime error, 4 failed assert,
            5 a legal setting this build cannot execute.
Language and verb reference: `help` inside the session, `?name` for one name.
```

Token	Arguments	Meaning
`--console`, `-i`	–	Interactive session; after `--script <f>` it opens the REPL once the script has succeeded.
`--script`	`<file.oc>`	Runs a script file (one of `--script`, `-c`, `-`).
`-c`	`<code>`	Runs one line of OC script.
`-`	–	Runs the script read from standard input (stdin is the script, so no REPL is ever opened).
`--set`	`key=value`	Sets a session setting before the first statement (repeatable); an unknown key or a bad value is exit 1, a legal value this build cannot execute exit 5.
`--log`	`<file>`	Appends the transcript to a file, inputs prefixed `> `.
`--quiet`, `--verbose`	–	The pipeline's own chatter off / on (default: on in the REPL, off in scripts).
`--help`, `-h`	–	Prints the console usage.

Token	Arguments	Meaning
`<name=value>`	–	A script parameter, read as `args.name`: `true` / `false`, a decimal integer, a quoted string, otherwise a String.

Interactive mode. `--console` (or `-i`) prompts `oc>` , and `...` while brackets are open or a line ends in `\`. An expression statement prints its value; a trailing `;` suppresses that. Every error is printed and the session stays usable; a verb-time error binds nothing. `quit` ends the session. Lines are read with plain `getline`; run the console under `rlwrap` for history and line editing. `Ctrl-C` during a statement stops it at its next cancellation point (every statement and call, and the long verbs' loop steps: per Hecke prime, torsion prime, level, stage) and prints `interrupted (at <site>)`; the prompt returns; the interrupted statement's own binding is not made, earlier effects of a compound statement stay as after any error (a lazy `descent` handle keeps its place and retries on the next read). `Ctrl-C` at the idle prompt, or twice in a row, ends the process as before. Scripts and `-c` are not affected. `--script f.oc --console` runs the script and then opens the REPL on its bindings – the natural way to explore a prepared state.

Script mode. `--script f.oc` runs a file; `-c '<code>'` one line; `-` standard input. In a script *every* error is fatal and sets the exit code (Section 7.2.7); a parse error in the script file itself is reported before anything runs, so a script with one has no side effect. The exception is `include("f.oc")`: the included file is parsed when the statement executes, so earlier statements have already run when its parse error (exit 2) is reported. The pipeline's own chatter is off by default in scripts (on in the REPL): a transcript must be a function of the script alone.

Parameters. `name=value` pairs after `--script` become `args.name`: `true` and `false` are Bools, a decimal integer is an Int, a quoted string a String, anything else a String. Reading an `args.name` the command line did not supply is a runtime error (exit 3). For example `Console/scripts/cli_params.oc` starts with

From `Console/scripts/cli_params.oc`:

```
n = args.n;
ell = args.ell;
```

and is run as `./hacs --script Console/scripts/cli_params.oc n=10 ell=691`.

Logging and transcripts. `--log <file>` appends the session to a file, inputs prefixed `>` . A line a script prints with the prefix `#~` (the `annotate` and `bench_line` verbs) is an *annotation*: timings and other machine-dependent text go there, and every transcript comparison ignores it. The frozen transcripts of `Console/scripts/*.expected` are the session output with annotations and trailing blanks removed, under one header line `#! oc-format 1`; they are regenerated only by the owner-gated emitter of Section 9.12.

7.2 The OC language

7.2.1 Lexical structure

A newline ends a statement unless brackets are open or the line ends in `\`; `;` also ends one and, after an expression statement, suppresses its printing. Comments run from `#` or `//` to the end of the line. Identifiers are `[A-Za-z_][A-Za-z0-9_]*`, case-sensitive. Reserved words:

```
for in if else set help quit true false and or not expect_error include args def
```

(`def` is reserved for a future user-function form and has no meaning yet.) Integers are decimal or `0x` hexadecimal with optional `_` separators (`134_217_757`); they are of type `Int` = `int64_t`, a literal outside $[-2^{63}, 2^{63}-1]$ is a parse error and arithmetic overflow is an error, never a wrap. Strings are `"..."` with the escapes `\" \\ \n \t`; there is no interpolation – use `str(x)` and `+`.

7.2.2 Grammar

The normative grammar ([P] Sec. 3.2):

```
program    := { statement }
statement  := assign | exprstmt | for | if | setstmt | include | help | expect | quit
assign     := IDENT "=" expr TERM
quit       := "quit" TERM
exprstmt   := expr TERM
for        := "for" IDENT "in" expr block
if         := "if" expr block [ "else" ( block | if ) ]
setstmt    := "set" IDENT "=" expr TERM
include    := "include" "(" STRING ")" TERM
help       := ( "help" [ IDENT | "?" ] | "?" IDENT ) TERM
expect     := "expect_error" STRING block
block      := "{" { statement } "}"
TERM       := NEWLINE | ";"
expr       := orexpr
orexpr     := andexpr { "or" andexpr }
andexpr    := notexpr { "and" notexpr }
notexpr    := "not" notexpr | cmp
cmp        := arith [ ("==" | "!=" | "<" | "<=" | ">" | ">=") arith ]
arith      := term { ("+" | "-") term }
term       := power { ("*" | "/" | "%") power }
power      := unary [ "^" power ]                          (right-assoc)
unary      := "-" unary | postfix
postfix    := primary { "(" args ")" | "." IDENT | "[" expr "]" }
primary    := INT | STRING | "true" | "false" | IDENT | "args" | "(" expr ")"
            | "[" [ expr { "," expr } ] "]"
args       := [ arg { "," arg } ]          arg := expr | IDENT "=" expr
```

Operators, from loosest to tightest. `or`; `and`; `not`; one comparison (`== != < <= > >=`, not chained); `+ -`; `* / %`; `^` (right-associative); unary `-`; then calls `f(...)`, members `.name` and indexing `[i]`.

Note

Two rules to know. (1) *The unary minus binds inside the base of* `^`: `power := unary [ "^" power ]`, so `-2 ^ 2 == 4` and `0 - 2 ^ 2 == -4`. This is not the school convention. (2) *Strings print bare only at the top of* `print` / `str`: `print("dim")` prints `dim`, while a bare expression statement `"dim"`, and a string inside a list or record, prints quoted.

Semantics. `/` on `Int` is exact and an error on a non-zero remainder; on `Fp` it is field division. `^` on `Int` is exact and overflow-checked. Comparison of objects is mathematical equality (matrices entrywise, polynomials coefficientwise). Arguments are positional or by name (`bench("hecke", n = 48, ell = 134217757, p = 2)`); optional parameters have defaults. Overloads are resolved in registration order: the first overload that accepts the arguments wins.

7.2.3 Statements and control flow

- **Assignment** `x = expr` binds in the single global frame; there are no local scopes and no user functions.
- `if` / `else`: `if cond { ...} else if cond2 { ...} else { ...}`; the condition must be a `Bool`.
- `for`: `for x in list { ...}` iterates a `List` (`range(a, b)` builds one).
- `include("f.oc")` runs another file in the same global frame, resolved relative to the including file and cycle-checked.
- `set key = value` changes a setting for the rest of the session (Section 7.4).
- `expect_error "substring" { ...}` succeeds iff the block raises an error whose message contains the substring; it is the only error-consuming construct (there is no `try`), and a block that does *not* fail is itself an error (exit 4).
- `help`, `help <name>`, `help ?` and `?name` print the registry's documentation of a verb, type or setting; `?x` on a value describes it (type, state, settings snapshot, creation site, members) – the same as `describe(x)`.
- `quit` ends a REPL session or a script (exit 0); `exit(code)` ends a script with a chosen code; `assert(cond, msg)` fails it with exit 4.

7.2.4 Values and types

The core value kinds are `Nil`, `Int`, `Bool`, `String`, `List` (`[a, b]`, heterogeneous, indexed from 0, `len`) and `Record` (printed as `{k: v, ...}`; read-only, members by `.k`, `keys`, `json`). A Record has no literal: `x = {a: 1}` is a parse error (exit 2). Records are produced by verbs and bindings (`settings()`, `state(x)`, `golden(...)`, `args`, the pipeline's report objects). Everything else is an *object* of a registered type: the arithmetic types (`Fp`, `Poly`, `Matrix`, `GF`, `GFElt`) and the pipeline types (`Model`, `LocalSystem`, `Cohomology`, `HeckeOperator`, `Family`, `Decomposition`, `Block`, `Report`, `ExtRecord`, `MaximalIdeal`, `HeckeAlgebra`, `LoewyProfile`, ...). This build registers 31 types; Appendix B lists each with its members.

Lazy handles. A pipeline verb returns at once with a *handle* in state `declared`; the work runs on the first access to a *computed* member, under the settings *snapshot taken when the handle was created*. *Structural* members are available while the handle is still declared. A computation that fails leaves the handle in state `failed`: every later access rethrows the stored error, naming both the access site and the creation site, and the handle is never retried – build a new one. `compute(x)` forces a handle, `state(x)` reports state, creation site, settings snapshot and resumable state, and `resume(x, budget)` continues a computation that a budget cut off (Section 7.6).

7.2.5 Verbs

Verbs are grouped by area; Appendix A lists all 178 overloads of the 165 verbs with their documentation, in registration (= resolution) order:

`core` printing, introspection, lists and records, `assert`, `exit`, settings, `compute` / `state` / `resume`;

`arithmetic` $\mathbb{F}_p$ elements, polynomials (`factor` is always certified), dense matrices over $\mathbb{F}_p$, linear algebra (`rank`, `kernel`, `solve`, `restrict`, ...), commutative algebras generated by commuting operators, characteristic and minimal polynomials, seeded random data;

`pipeline` the $\mathrm{SL}(2,\mathbb{Z})$ model and simplicial presets, local systems, `cohomology`, `hecke`, `family`, `decompose`, `report`, duality (`dual`, `pairing`), cross-checks;

`reproducibility` `hash`, `golden` (read-only access to the frozen reference), `fingerprint`, `math_settings`, `save_state`;

`harness` `validations`, `run_validation`, `bench`, `micro`, the campaign verbs, `golden_subset`, `timed`, `rss`;

`reference` `help_markdown` (the Markdown reference `docs/console/console.md`);
`extension` the extension layer: `sturm_bound`, `u_ell_decision`, `ext_record`, `local_algebra`, `module_structure`, `presentation`, the local-algebra invariants (`radical`, `socle`, `loewy`, ...), `atlas`, the canonical fields `GF`, `iota`, `boundary`, `cone`, `dickson_les`, `theta`, `hecke_ring_algebra`.

Inside a session, `verbs()`, `types()` and `settings()` enumerate the registry, `help` lists everything and `?name` documents one entry.

7.2.6 Printing

Every value has a *canonical* form – what a transcript contains and what `show`, `repr` and an expression statement print – and a *display* form, which differs only in that a top-level String loses its quotes (`print`, `str`). Numbers are canonical residues in $[0, p)$; polynomials print from the highest degree down with their modulus (`X^3 + 72*X^2 + 346*X + 4 (mod 691)`), although `coeffs` and `poly` use the machine order low → high.

`print(a, b, ...)`	display forms on one line, separated by blanks
`show(x)`	the full canonical form, ignoring `print_max_entries`
`display(a, b, ...)`	forces each declared handle, then shows each argument: as `show` in a script or a terminal, as a rendered value in the notebook
`describe(x)`, `?x`	type, state, settings snapshot, creation site, members
`str(x)`, `repr(x)`	the display / canonical form as a String
`json(x)`	JSON for Int, Bool, String, List and Record (objects have their own `.json` members)
`annotate(text)`	a `#~` annotation line, ignored by transcript comparisons

The engine setting `print_max_entries` (default 400) caps what `print` and expression statements render: a larger object prints its summary form (`Matrix 1677 x 1419 over F_134217757 (hash 0x...)`).

`display`: printing for the notebook. A handle such as `cohomology(L)` is *declared*: nothing is computed until a value is needed, and an expression statement prints `(declared, not yet computed)`. `display(x, ...)` first forces every declared handle among its arguments, exactly as `compute(x)` does, and then shows each argument on a line of its own. In a script or at the terminal that line is the full canonical form, as `show` prints it:

From `Console/scripts/056_s24_display.oc`:

```
M = sl2z_model();
H = cohomology(symmetric_power(M, 10, 691));
H
display(H);
display(H.dims, 691, "dim H^1 = 2 dim S_12 + 1");
display(dickson_les(16, 7).dims);
```

prints (the frozen transcript)

Frozen transcript `Console/scripts/056_s24_display.expected`:

```
#! oc-format 1
Cohomology (declared, not yet computed)
Cohomology dims = [0, 3, 0]
[0, 3, 0]
691
"dim H^1 = 2 dim S_12 + 1"
{H0_n: 1, H0_up: 1, H0_Q: 1, H1_n: 4, H1_up: 4, H1_Q: 1}
```

In a served notebook (Section 7.8) `display` sends each argument as a *value*, as an expression statement does, so the page typesets it: $\dim_{\mathbb{F}_{691}} H^1 = 3$, a matrix as a matrix, an integral group as $\mathbb{Z}^3 \oplus \mathbb{Z}/2 \oplus \mathbb{Z}/2^2 \oplus \mathbb{Z}/3$, a character value as an element of the residue field. `print` instead sends plain text, so a notebook cell uses `display` for the quantities and `print` for the prose. In a served session the text line of each argument is its canonical form under `print_max_entries`, the echo an expression statement prints. That line equals `show`'s line, except for a value above the cap and for a type with a long form of its own, such as `Model` and `Complex`. `display` is served (`read_only`) and changes no value.

7.2.7 Errors and exit codes

Exit	Meaning
0	success
1	usage: an unrecognised console argument, a script file not found, an undeclared `--set` key, an invalid `--set` value, or a `--log` file that cannot be opened
2	parse error – in the script file itself reported before anything runs (an `included` file is parsed when the statement executes)
3	runtime error: a type or arity error, a pipeline contract violation, every `ENGINE_CHECK`, `hecke::HeckeOverflow`, out of memory, a failed handle
4	a failed `assert(cond, msg)` or `expect_error`
5	a *legal* setting this build cannot execute (`hecke::NotImplementedInThisBuild`): on a `set` statement, as a `--set` argument, or at a verb that does not honour the setting in force; never reported as a bad value and never silently degraded to the default. Exceptions: `coefficient_ring = Z/<m>` (reserved) is refused with exit 5 already at `set` / `--set`, whereas `coefficient_ring = Z` is *accepted* there (stored; honoured by the integral descent only, `[C, sec:recordschema]`, `[C, def:ringsemantics]`(3)) and exits 5 at every verb other than `descent`, naming the verb; and `field_polynomial = "conway"` is *accepted* there (the fingerprint moves) and only a verb that needs the field exits 5

Every error is printed as `file:line:col: kind: message` followed by the offending source line; a C++ exception names its class (`caused by: hecke::HeckeOverflow: ...`), and an `ENGINE_CHECK` keeps its own `file:line`. Section 7.5 shows `expect_error` over the documented error messages.

7.3 A tour of the small verbs

The script `055_s24_language_tour.oc` runs once through the core, arithmetic and reproducibility verbs that the larger examples take for granted: values and records, $\mathbb{F}_p$ elements and polynomials (printed from the highest degree down, with their modulus), matrices, a finite field, the simplicial presets, hashes and the golden reference, and one extension verb.

Frozen transcript `Console/scripts/055_s24_language_tour.expected`:

```
#! oc-format 1
math_settings() is a Record with the keys ["locality", "locality_source",
↪  "is_default", "fingerprint", "canonical_text", "keys"]
abs(-7) = 7  min(3, 5) = 3
repr of a string: "dim"  json of a list: [1, "two", true]
script_args() without parameters: {}
Fp(691, 1000) = 309 mod 691
gcd( X^2 + 2*X + 1 (mod 691) , X^2 + 690 (mod 691) ) = X + 1 (mod 691)
X = X (mod 691)  f(2) = 9  X^691 mod (X^2 + 1) = 690*X (mod 691)
rref: [[1 0 1] [0 1 1] [0 0 0]]
rank 2 column space: [[1 2] [2 4] [1 0]]
in GF(7^2), C_{ell,r} = X^2 + 1 (mod 7) : (1 + 2Y) - (1 + 2Y) is zero: true
```

```
models(): ["sl2z_7vertex"]
1-sphere : dimension 1  simplex counts [3, 3]  Euler characteristic 0
2-sphere : dimension 2  simplex counts [4, 6, 4]  Euler characteristic 2
3-sphere : dimension 3  simplex counts [5, 10, 10, 5]  Euler characteristic 0
Moebius Band : dimension 2  simplex counts [6, 12, 6]  Euler characteristic 0
Projective Plane : dimension 2  simplex counts [6, 15, 10]  Euler characteristic 1
a hand-made circle: triangle [3, 3] 0
golden_has(n=10 ell=691, dim_H^1): true
[0, 3, 0]
type:  List
size:  3
value: [0, 3, 0]
lfunction/v1  k = 12  ell = 691  a = 199  mu = 0  lambda = 1
the first three coefficients of L_a(T): [33498404414, 209903387020, 26227700637]
```

Four verbs have no example by design: `exit` (it ends the script), `save_state` (it writes a file), `help_markdown` (its output is `docs/console/console.md`) and `campaign_spec` (it reads a spec file; the same parser is shown by `campaign_spec_text` in `048_c5c_ring_campaign.oc`).

7.4 Settings

The console has one settings mechanism, `set key = value` (or `--set key=value` on the command line); `settings()` shows every key with its value, source and family, and `reset_settings()` returns to the session's baseline. There are two *families*, and the difference matters:

- **Engine settings** change how fast a value is produced, never the value: thread counts, the substrate, the device policy, the compiler passes of the tape engine. Their defaults come from the `ENGINE_*` environment variables (Section 9.4); some are *env-only* and cannot be `set`.
- **Mathematical settings** may change what a result *means*: the search budget of the locality certificate, whether U_ℓ is included, the Sturm cap, the coefficient ring, ... The engine owns their defaults and their validation; every record stores the values in force with their source; and a non-default value moves the *fingerprint* (`fingerprint()`, FNV-1a-64 of the non-default entries, `0x0000000000000000` at the defaults). The fingerprint enters every extension-record hash input and every campaign job id, and `golden(...)` refuses to compare at a non-zero fingerprint: the frozen reference pins the defaults only.

A handle captures the settings in force *when it is created*; changing a setting later does not change an existing handle. The complete table (43 keys: 11 mathematical, 32 engine) is Appendix C.

7.4.1 The 11 mathematical settings

The legal values, from `Hecke/HeckeSettings.h`; the first one listed is the default.

Key	Values	Meaning
`locality`	`budget=64`, `budget=<N>`, `strict`	the locality certificate of a decomposition block (`[E, sec:localcert]`): the witness-search budget, or the terminating strict mode (byte-identical wherever budget mode certifies)
`u_ell`	`auto`, `off`, `budget_mb=<M>`	whether the extension layer builds U_ℓ: by a frozen memory model against 2048 MB, never, or against M MB

Key	Values	Meaning
sturm	auto, off, cap=<B>	the computed Sturm bound $B'(k,\ell)$ ([E, def:bprime]): cap $12B(k)$, not attempted, or cap $B \geq 2$
sturm_unavailable	nonfinal, heuristic	an ideal whose bound is not found at the cap stays non-final, or is swept heuristically
commutant_budget	20000000000, <N>	the work budget (modular multiplications) of the commutant certificate
final_mode	closure, sturm_heuristic	final = true only on a proof, or "heuristic" on [E, cert:sturm]'s heuristic range
field_polynomial	least, conway	the defining polynomial of the canonical field ([E, def:canon]); conway is legal but not executable (exit 5)
split_field_report	canonical, own_field	the primary form of reported eigenvalues ([E, sec:field]); both forms are always emitted
presentation	off, on	whether extension records carry module presentations
predictions	annotate, assert	predictions are annotations, or a refuted prediction fails the run (a debug switch)
coefficient_ring	F_ell, Z/ell^<k>	coefficients in $\mathbb{F}_\ell$ or $\mathbb{Z}/\ell^k$, $2 \leq k \leq 4096$, the token ell literal ([E, sec:ring]); Z is stored and honoured by the integral descent only (the verb descent, the campaign's descent cells, the record integral_descent_record/v1; [C, sec:recordschema], [C, def:ringsemantics](3)), every other verb refuses it by name with exit 5; the reserved value Z/<m> is refused with exit 5 at set / --set

In the console a String setting takes a quoted value: `set locality = "budget=8"`; on the command line `--set locality=budget=8` (only the first = splits). A bad value is an ordinary error naming the key (exit 3 on `set`, exit 1 on `--set`); a legal value this build cannot execute is exit 5. In a campaign spec the same keys appear as `key = value` lines.

7.5 Errors on purpose: `expect_error`

From `Console/scripts/014_expect_error.oc`:

```
expect_error "is not a degree of this complex" { decompose(T2, 5) }
expect_error "two operators with the same prime" { decompose([T2, T2], 1) }
expect_error "no SL(2,Z) tessellation model" { hecke(cohomology(s1_scalar(1, 5)),
↪  2).dims }
expect_error "unknown locality value" { set locality = "nonsense" }
expect_error "unknown setting 'no_such_key'" { set no_such_key = 1 }
...
set locality = "strict";
expect_error "the mathematical settings are not at their defaults" {
golden(POINT_COH, "dim_H^1")
}
set locality = "budget=64";
```

Frozen transcript `Console/scripts/014_expect_error.expected`:

```
#! oc-format 1
014_expect_error: every documented throw of the C1b surface
```

The transcript is one line because `expect_error` prints nothing when the expected error occurs. The last block shows the reproducibility rule: under a non-default mathematical setting `golden` refuses.

7.6 Settings, the fingerprint and `resume`

From `Console/scripts/021_settings_resume.oc`:

```
set threads = 1;
H1 = cohomology(symmetric_power(MODEL, N_SYM, ELL));
reset_settings();
print("threads = 1:", H1.dims);
...
set locality = "budget=8";
assert(fingerprint() != "0x0000000000000000", "a non-default key moves the
↪  fingerprint");
print(settings().locality.value, settings().locality.family);
set locality = "budget=64";
...
set field_polynomial = "conway";
expect_error "not_implemented_in_this_build" { canonical_field_polynomial(691, 2) }
set field_polynomial = "least";
print("C_{691,2} =", canonical_field_polynomial(691, 2));
...
CUT = sturm_bound(24, 5, 3);
...
CONT = resume(CUT, 36);
FRESH = sturm_bound(24, 5, 36);
...
print(CONT.B_prime, state(CONT).resumable.progress.budget_history,
state(FRESH).resumable.progress.budget_history);
```

Frozen transcript `Console/scripts/021_settings_resume.expected`:

```
#! oc-format 1
threads = 1: [0, 3, 0]
budget=8 mathematical
C_{691,2} = X^2 + 1 (mod 691)
5 [3, 36] [36]
```

An engine setting (`threads`) changes no value; a mathematical one (`locality`) moves the fingerprint; `conway` is a legal value this build cannot execute (outside `expect_error` that is exit 5); and a Sturm-bound computation cut off at cap 3 is resumed under cap 36 to the same $B' = 5$ a fresh run finds, with the budget history $[3, 36]$ recorded.

7.7 The harness from the console

From `Console/scripts/022_harness.oc`:

```
print("run_validation(mod_arithmetic_validation) =",
↪  run_validation("mod_arithmetic_validation"));
expect_error "no validation is registered" { run_validation("NoSuchValidation") }
expect_error "owner-gated emitter" { run_validation("GoldenEmit") }
```

```
B = bench("hecke", n = 48, ell = 134217757, p = 2);
bench_line(B);
print("bench(hecke, 48, 134217757, 2).dims =", B.dims);
...
G = golden_subset([[N_SYM, ELL]]);
print("golden_subset([[10, 691]]).mismatches =", G.mismatches);
```

Frozen transcript `Console/scripts/022_harness.expected`:

```
#! oc-format 1
run_validation(mod_arithmetic_validation) = 0
bench(hecke, 48, 134217757, 2).dims = [0, 7, 0]
bench(decomp, 10, 691, 2): q = 1 blocks = 1
golden_subset([[10, 691]]).mismatches = 0
jordan_type(N) = [3, 1]
```

`bench_line` prints the timing line as a `#~` annotation, which is why no time appears in the transcript; the owner-gated emitters cannot be run from a session. (The console can still write files, through `save_state(x, path)` and `run_campaign(spec)`; only the emitters of Section 9.12 are out of reach.)

7.8 The served console: `--serve`

`--serve` runs *one* console session driven by newline-delimited JSON on standard input and output – protocol `oc-serve/1`, specified in [P] v3.10 Section 18. It is the back end of the notebook: a gateway (a separate program) starts the process, relays requests and frames, and owns the network side; the binary never opens a socket. It is compiled only with the CMake option `OC_BUILD_NOTEBOOK=ON`; in a default build the token is unknown (exit 1).

Page themes. The notebook page's header carries a theme selector (*Auto*, *Light*, *Dark*, two high-contrast presets, *Custom*) and a *Theme...* editor. They change the page's colours only, in the browser: the choice is kept in the browser's `localStorage`, never sent to the gateway or the binary, and no verb, setting or command-line token is involved. A theme is exchanged as JSON of the format `oc-notebook-theme/1`; see `notebook/page/README.md` for the format and the tokens, and [P] Section 18.17 for the specification.

Token	Arguments	Meaning
`--set`	`key=value`	Sets a SETTABLE key before the `hello` frame (repeatable): `print_max_entries`, `lean`, `program_cache` and the eleven mathematical settings; any other key is exit 1, a legal value this build cannot execute exit 5.
`--rss-cap-mib`	`<N>`	The resident-set cap of the watchdog (polled every 250 ms and at every statement boundary): above it a `fatal` frame `rss_cap` and exit 70; 0 (the default) is off.
`--max-reply-bytes`	`<N>`	The frame cap (default 1048576, at least 4096): longer `out` text is split at line boundaries, a larger value is sent in its truncated form.

Requests and frames. Every request is one JSON object per line with an `op`: `eval` (`src`, optional `args`) runs a cell as one program, parsed in full first; `complete`, `help` and `describe` answer from the registry and the session without evaluating; `interrupt` stops the running cell at its next cancellation point (a statement, a verb call, or a loop step inside a long verb); `close` ends the session. The binary answers with frames: one `hello` first, then per request `out` (what a script would print; annotations go to `progress`), `value` (the JSON envelope `{t, canonical, hash, v}` of each echoed value; no TEX, no floats, integers above $2^{53} - 1$ as decimal strings) and exactly one `done`. The `out` text of a cell is byte for byte what `--script` prints; `ConsoleServeValidation` compares it with every frozen transcript. A `display(x, ...)` call (Section 7.2.6) sends, per argument, the same canonical line and `value` frame that the expression statement `x` would, after forcing a declared handle.

What the page renders. The page turns the typed envelopes into TEX (the binary emits none). It typesets numbers, residues, polynomials, matrices, finite fields, local systems and cohomology, and the parts that render hints mark as polynomials, q-series, abelian groups, Hilbert functions, Loewy layers, Jordan types, character values and the maps of the Dickson long exact sequence. Records become tables. Each type's decision is listed in `src/hacs/Console/serve/render_coverage.tsv`, and `ConsoleReadOnlyTagValidation` keeps that list complete. A declared value carries a *compute* button, which evaluates `display` of its statement. A matrix too large for `print_max_entries` carries a *show all* button, which displays it once under a raised cap (up to 100 000 entries) and then restores the setting. The tour `notebook/examples/notebook/` (ten short files, from the model to the integral Hecke structure) is written for this mode.

Errors and exit codes. A cell's error is a `done` with `status` and the exit code of Section 7.2.7 (2 parse, 3 runtime, 4 assertion, 5 not implemented) and the session survives it; 1 is a refusal of the request (nothing ran: `include`, `quit`, `exit`, a verb that is not served, a malformed request, a full queue). The process exits 0 on end of input, `close` or `SIGTERM`, 1 when its launch is refused and 70 after a `fatal` frame (the RSS cap, a request line over 8 MiB, an internal error).

Read-only. A served session computes and displays; it writes no file and reaches no emitter. Every verb overload carries a `read_only` tag (Appendix A marks it [served] / [not served]): 169 of the 178 overloads, in 159 verbs, are served; the other 9 (the two overloads of `exit`, `report(D, level_a, path)` — the pathless `report(D, level_a)` is served and computes the record in memory —, `save_state(x, path)`, `run_validation`, `bench`, `micro`, `campaign_spec`, `run_campaign`) are refused. Only `print_max_entries`, `lean`, `program_cache` and the eleven mathematical settings may be `set`. The launch is refused (exit 1, nothing on standard output) when the environment holds any `GOLDEN_*`, `CONSOLE_*`, `CLI_MANUAL_*`, `HECKE_*`, `ENGINE_*`, `OC_*` or sanitizer variable other than `ENGINE_THREADS`, `ENGINE_CUDA_DISABLE`, `ENGINE_CUDA_DEVICE` and `ENGINE_HOST_THREADS`; the process then sets `RLIMIT_FSIZE` and `RLIMIT_CORE` to 0 and `PR_SET_NO_NEW_PRIVS`. This is a safety rail against accidents, not a sandbox. The machine-readable twin of the tag column is `docs/latex/cli_manual/read_only_verbs.json`.

7.9 The atlas server: `--serve-atlas`

`--serve-atlas` answers read-only queries on one `hecke_atlas/v1` store – protocol `oc-serve-atlas/1`, [P] v3.11 Section 18.16 – so that the notebook's atlas panel can browse the $\Sigma(691)$ store (451 875 793 bytes) without ever sending it to a browser. Like `--serve` it is compiled only with `OC_BUILD_NOTEBOOK=ON`, speaks newline-delimited JSON on standard input and output, and is started by the gateway (`--atlas-store`); the store's path comes from the command line, never from a request.

Token	Arguments	Meaning
`--store`	`<store.json>`	The `hecke_atlas/v1` store to serve (required): opened `O_RDONLY`, mapped read-only, its FNV-1a-64 computed at open; the path comes from the command line only, never from a request.
`--index`	`<path>`	The `oc-atlas-index/1` sidecar (default `<store.json>.ocidx` when it exists, else an index built in memory); compared field by field with the store at open, a stale or corrupt one refuses the launch (exit 1).
`--expect-fnv`	`0x<hex>`	Refuses the launch unless the store's FNV-1a-64 equals this value (Sigma(691): `0x73c7210047a81b97`).
`--max-reply-bytes`	`<N>`	The frame cap (default 1048576, at least 4096): a page is cut before the frame would exceed it (`next` continues).
`--rss-cap-mib`	`<N>`	The resident-set cap of the watchdog (polled every 250 ms and after every request): above it an `atlas_fatal` frame `rss_cap` and exit 70; 0 (the default) is off.

Requests and frames. After one `atlas_hello` (the store's name, size, FNV and counts, the index source `sidecar` or `memory`) every request – `info` (the store header, the type and residue-degree profile), `cells` and `systems` (pages with `offset`, `limit` ≤ 1000 and, for systems, a `filter` on `r`, `type`, `collected`, `base`, `twist` or the cell `n`), `cell` (one cell and its occurrences), `system` (one system by `sid`) – gets exactly one `atlas_reply` carrying the store's FNV; `close` ends the process. A page is cut before a frame would exceed `--max-reply-bytes`, and `next` says where to continue. Errors (`protocol`, `not_found`, `too_large`) are replies; the process goes on.

Read-only. The launch applies the environment rule and the hardening of `--serve` (Section 7.8) and is refused (exit 1, nothing on standard output) on a forbidden variable, an unknown token, a store that does not open or is not indexable, an FNV that differs from `--expect-fnv`, or a stale or corrupt sidecar: the index is re-derived from the store at every open and compared field by field. The store is opened `O_RDONLY` and mapped without write permission; the process cannot extend any file.

7.10 The scripts, by subject

Every numbered script is a frozen, tested example; read them as recipes.

Subject	Scripts
cohomology	001_dimensions.oc, 013_presets.oc, 055_s24_language_tour.oc
Hecke operators	002_hecke_pins.oc, 003_bases_and_projectors.oc, 004_family_identities.oc, 005_linear_algebra.oc, 006_flagship.oc, 011_decompose_certify.oc, 012_report_schema.oc, 015_strict_locality.oc
the golden grid	007_grid_classical.oc, 008_grid_dickson.oc, 009_grid_small_ell.oc, 010_grid_edge_n.oc
$\mathbb{Z}/\ell^k$	024_e8_ring_points.oc, 025_e8_console_surface.oc, 026_c5_leftovers_ring_members.oc, 028_wide_band.oc, 029_hecke_ring_algebra.oc, 048_c5c_ring_campaign.oc, 049_lv3_iota_ring.oc
over $\mathbb{Z}$	054_s24_descent.oc
levels, Hida, forms	050_s21_pow_kummer_unit.oc, 051_s21_hida_tower.oc, 052_s22_level_n_nebentypus.oc, 053_s22_forms_hida_lambda.oc
extension layer	016_extensions.oc, 017_predictions.oc, 018_e0_locality_grid.oc, 019_e1_stage1_sturm.oc, 020_e2_ramanujan_10_691.oc, 023_e7_duality_10_691.oc, 027_dickson_theta.oc, 040_c5_ext_record_10_691.oc to 047_c5_local_algebra_verbs.oc
the language	014_expect_error.oc, 021_settings_resume.oc, 022_harness.oc, cli_params.oc, prelude.oc

(041_c5_stage9_kummer_cells.oc, 042_c5_dickson_class_cone.oc, 043_c5_boundary_coincidences.oc, 044_c5_multiplicity_two.oc, 045_c5_pd_loewy_reversal.oc and 046_c5_identities.oc complete the range 040–047.)

Chapter 8

Campaigns: `--campaign`

8.1 What a campaign is

A campaign is a sweep over (n, ℓ, p, r, q) described by a small text file, the *spec*. It runs every job through the same public API as `--bench hecke` and writes down the mathematics: one JSON per job with the canonical objects and their hashes, the report levels (a)/(b)/(c), the certificates that ran, timings and peak RSS, plus a campaign-level `index.json` and `summary.tsv`.

```
Usage: hacs --campaign <spec-file> [options]

Runs a parameter sweep of (n_sym, ell, p, r, q) Hecke jobs through the same
public API as `--bench hecke`, writing one streamed JSON per job plus
index.json and summary.tsv. Spec grammar and worked examples:
  docs/matrix-engine/campaign.md

Options:
  --output <dir>   override the spec's `output` directory
  --jobs <n>       override `jobs`: concurrent jobs per shape group
  --threads <n>    override `threads`: engine workers per concurrent job
  --dry-run        print the plan (groups and jobs) and exit
  --quiet          suppress per-job progress lines
  --help           this text

Spec example:
  name    = t2_sweep
  output  = out/t2_sweep
  certify = sampled(every 10)
  lean    = true
  n_sym   = 0..200 step 2
  ell     = 5,13,691,10007,134217757
  p       = 2
  r       = 1
```

Token	Arguments	Meaning
`--output`	`<dir>`	Overrides the spec's `output` directory.
`--jobs`	`<n>`	Overrides `jobs`: concurrent jobs per shape group.
`--threads`	`<n>`	Overrides `threads`: engine workers per concurrent job.
`--dry-run`	–	Prints the plan (groups and jobs) and exits without computing.
`--quiet`	–	Suppresses the per-job progress lines.
`--help`, `-h`	–	Prints the campaign usage.

Token	Arguments	Meaning

8.2 The spec format

The spec is line-oriented:

```
# or //          comment to end of line
---              on a line by itself: start a new SECTION
key = value      value is a comma-separated list whose elements are literals
                 or ranges `a..b [step c]` (step defaults to 1)
```

Grid keys (`n_sym`, `ell`, `p`, `r`, optional `q`) belong to the current section; the campaign is the union of the sections' Cartesian products, and every section must give the four mandatory axes. *Campaign keys* may appear anywhere; the last assignment wins. Duplicate jobs are removed (first occurrence wins). Every key the parser accepts:

Key	Value / default	Meaning
`n_sym`, `n`	list (grid)	Symmetric-power degree n (d_V = n + 1); every section must give it.
`ell`	list (grid)	Coefficient prime; ell >= 2^32 selects the 64-bit pipeline.
`p`, `p_hecke`	list (grid)	Hecke prime; p = ell is U_ell, the same code path.
`r`	list (grid)	Exponent of T_{p^r}; r >= 2 adds the algebra recursion (32-bit ell only).
`q`, `degrees`	list (grid), optional	Degrees to report (default: all of the model's); a non-default list is part of the job id.
`level`	list (grid), coefficient_ring = Z only	(slot-24 D6) The level N of Gamma_0(N) of a descent cell (default 1); admitted only under `coefficient_ring = Z`, where each (level, n_sym) cell writes one integral_descent_record/v1 (campaign.md Sec. 17); a spec error everywhere else.
`parts`	default `all`, coefficient_ring = Z only	(slot-24 ICP2) `groups`, `groups,hecke` or `all`: which parts of the integral descent record a descent cell computes (the Hecke data char_Z and the torsion modules; the boundary pair). A part not run is marked "ran":false in the record; a non-default value is part of the cell id; a spec error outside `coefficient_ring = Z`.
`structure`	default `none`, coefficient_ring = Z only	(slot-25 HZ-REC) `none` or `hecke`: under `coefficient_ring = Z`, `hecke` makes each (level, n_sym, algebra) cell write one integral_hecke_structure/v1 record instead of a descent record (campaign.md Sec. 18); it selects the record schema and is not a mathematical setting; a spec error outside `coefficient_ring = Z`.
`algebra`	list (grid), structure = hecke only	(slot-25 HZ-REC) `full`, `anemic` or both: the Hecke algebra of a hecke-structure cell (default `full` at level 1, `anemic` above; `anemic` at level 1 is a spec error).
`hz_cert`	default off, structure = hecke only	(slot-25 HZ-EMIT) `on` also writes the certificate files of each record into <output>/hzcert/ for lean/tools/hzcert_to_lean.py; an output knob, no value depends on it.

Key	Value / default	Meaning
hz_g8	default off, structure = hecke only	(slot-25 H1) off or hida: the record's settings value heckez.G8. off writes no G8 field (the default records); hida adds H1's Hida block per maximal ideal at level 1 (ordinary, chain_key, g, L_k, v_Bk, structure, I0_generator), a non-default value with its own settings fingerprint and cell ids, never golden.
name	default campaign	Echoed into index.json and every job file.
output, output_dir	default campaign_out	The output directory (created).
model	default sl2z_7vertex	The model descriptor; an unknown name is a spec error.
resume	default false	Skip jobs whose output file exists and carries the completion trailer.
lean, memory_lean	default false	Memory-lean mode, as --bench --lean.
threads	default 0	Engine workers per concurrent job; 0 shares the process pool (ENGINE_THREADS).
jobs, concurrent_jobs	default 1	Jobs of one shape group that run concurrently.
certify	default full	full, sampled(<fraction>), sampled(every <k>), sampled(<k>) or none: which jobs run the external cross-checks (the in-process certificates always run).
level_a	default auto	auto, always or never (off is accepted as an alias of never): whether a job report carries level (a).
level_a_max_entries	default 16000000	The auto size gate of level (a), in matrix entries.
theta_rank_max_dim	default 2048	rank(Theta_flat^q) is computed only below this dimension.
ext	default false	Run the extension engine once per (n_sym, ell) cell; false leaves the output byte-identical to the pre-extension driver.
family	default 2,3,5,7	The good primes of S_0 for the ext pass; the literal token ell admits U_ell through the u_ell budget.
sign	default all	all, plus or minus: which iota-sign record a cell reports.
ext_groups	default all	bulk, bulk+bd or all; bulk leaves the cone unbuilt.
ext_jobs	default 1	Scheduling knob: cells of the ext pass run at once (changes no value, id or file).
ext_memory_mb	default 40000	Scheduling knob: the admission cap of the parallel ext pass, in MB.
ext_gpu	default off	Scheduling knob: off, on or force, one extra device worker; never on by default.
ext_gpu_cells	n:ell pairs	Scheduling knob: the cells the device worker takes; needs ext_gpu on or force.
orbifold	default absent (off)	c_i: write one orbifold_cell/v2 record per (n_sym, ell) cell, ell in {2, 3} – group cohomology on C_I plus the degree-1 amalgam reader – instead of Hecke jobs; no p / r / q axis. Absent or off leaves the campaign byte-identical.
orbifold_primes	default 2,3,5,7	The T_p of an orbifold cell (T_ell is always included); a non-default list changes the cell id (_s<16 hex>) and the record hash input.
orbifold_m_target	default auto	The certified ell-adic precision every cell reaches (auto = max(floor((n+1)/(ell+1)) + 10, 40)); a divisor not certified below it is a fallback. Non-default: part of the cell id.

Key	Value / default	Meaning
`orbifold_k_cap`	default `0`	The largest working precision K the reader may climb to (0 = its instantiated range, 1024 at ell = 2, 312 at ell = 3). Non-default: part of the cell id.

In addition, every registered **mathematical setting** (Section 7.4.1: `locality`, `u_ell`, `sturm`, ..., `coefficient_ring`) is accepted as a campaign-wide key and validated by the engine itself; an unknown key is a spec error. A non-default mathematical key changes every job id (a settings suffix) and the shape signature; at the defaults nothing is appended. `coefficient_ring = Z/ell^<k>` runs the job loop over $\mathbb{Z}/\ell^k$ (`[E, sec:ring]`) and is refused together with `ext = true`, before any job runs.

A complete example, the one the usage text prints:

```
name    = t2_sweep
output  = out/t2_sweep
certify = sampled(every 10)
lean    = true
n_sym   = 0..200 step 2
ell     = 5,13,691,10007,134217757
p       = 2
r       = 1
```

Run `--campaign t2.spec --dry-run` first: it prints the shape groups and jobs and exits.

8.3 What `certify` samples

The in-process certificates are never sampled: $\partial\tau = \tau\partial$, the agreement of the two Θ routes on every entry, the exact Θ_{eq} residual and the decomposition certificate run on every job, and each job file records their executed counts. `certify` only selects the jobs that additionally run the *external* cross-checks (the classical $\alpha/\beta/\omega$ factorisation and, for $r \geq 2$, the direct double-coset route): `full` every job; `sampled(every k)` or `sampled(k)` the jobs with index $\equiv 0 \bmod k$; `sampled(1/4)` the same with $k = \mathrm{round}(1/\mathrm{fraction})$; `none` no job.

8.4 Output

```
<output>/
  index.json            campaign metadata, the shape groups, one row per job
  summary.tsv           one row per job -- the published table
  jobs/<job-id>.json    one streamed JSON per job
  ext/hecke_ext_ell<L>_n<N>.json      (only with ext = true) one record per cell
```

A job id is `n<N>_ell<L>_p<P>_r<R>`, followed by `_q<d1>-<d2>-...` when the spec names a non-default degree list and by a settings suffix when a mathematical key is not at its default. The id is the file stem, the `id` column of `summary.tsv` and the *resume key*: with `resume = true` a job is skipped iff `jobs/<id>.json` exists and carries the completion trailer, so an interrupted campaign is restarted with the same command. Over $\mathbb{Z}/\ell^k$ the job file schema is `hecke_ring_report/v1`; with `ext = true` the cell records are `hecke_ext/v5`. The scheduling knobs (`jobs`, `threads`, `ext_jobs`, `ext_memory_mb`, `ext_gpu`, `ext_gpu_cells`) change only when and where a job runs, never an id, a file or a value.

From the console the same driver is reached by `campaign_spec(path)`, `campaign_spec_text(text)`, `campaign_plan(spec)` and `run_campaign(spec, output)`.

8.5 Campaigns over $\mathbb{Z}$

Under `coefficient_ring = Z` a campaign is a grid of *descent cells*, one per (N, n), each writing one `integral_descent_record/v1` (Chapter 5). The grid keys are `n_sym` (required) and `level` (default 1; a spec error under any other ring); `p` optionally lists the Hecke primes (none may divide a level of its section; default as for the verb). `ell`, `r`, `q`, `orbifold`, `ext = true`, `jobs` > 1 and a non-prime p are refused by name before any cell runs. The spec of the examples below:

```
# A campaign over Z: six descent cells, levels 1 and 11, n = 0, 2, 4 ([CLI] sec:campaign-z; campaign
coefficient_ring = Z
name   = descent_small
n_sym  = 0..4 step 2
level  = 1, 11
```

`--dry-run` lists the cells with their size $D = \psi(N)(n+1)$, the dimension of the induced coefficient module:

Real output `docs/latex/cli_manual_runs/campaign_z_dry.txt`:

```
$ hacs --campaign descent_small.spec --dry-run
descent campaign 'descent_small': 6 cells (integral_descent_record/v1, coefficient_ring = Z,
↪  fingerprint 0x098a1abfd656372a)
    desc_N1_n0_s098a1abfd656372a  N=1 n=0 primes=2,3,5 D=1
    desc_N1_n2_s098a1abfd656372a  N=1 n=2 primes=2,3,5 D=3
    desc_N1_n4_s098a1abfd656372a  N=1 n=4 primes=2,3,5 D=5
    desc_N11_n0_s098a1abfd656372a  N=11 n=0 primes=2,3,5 D=12
    desc_N11_n2_s098a1abfd656372a  N=11 n=2 primes=2,3,5 D=36
    desc_N11_n4_s098a1abfd656372a  N=11 n=4 primes=2,3,5 D=60
  6 cells total
[exit 0]
```

A cell id is `desc_N<N>_n<n>[_p<p1>-<p2>-...]_s<16 hex>`: `coefficient_ring = Z` is a non-default mathematical setting, so its fingerprint is part of every id. The run writes `cells/<id>.json` (one line, `integral_descent_cell/v1`, with the record and a completion trailer), `summary.tsv` (`id N n primes status H0 H1 H2 H3 record_fnv1a64 seconds`) and `index.json`, and ends with the line `EFFECTIVE campaign arm=descent cells= ran= resumed= refused= failed=`:

Real output `docs/latex/cli_manual_runs/campaign_z_run.txt`:

```
$ hacs --campaign descent_small.spec --output descent_small_out --quiet
EFFECTIVE campaign arm=descent cells=6 ran=6 resumed=0 refused=0 failed=0
[exit 0]
$ head -40 descent_small_out/summary.tsv
id  N   n   primes  status  H0  H1  H2  H3  record_fnv1a64  seconds
desc_N1_n0_s098a1abfd656372a    1   0   2,3,5   ok  Z   0   Z/2^2 +
↪  Z/3 0   0x0a716318aeeaa7fe  0.077
desc_N1_n2_s098a1abfd656372a    1   2   2,3,5   ok  0   Z + Z/2 0   Z/2 +
↪  Z/2   0xd747891c23237467  0.051
desc_N1_n4_s098a1abfd656372a    1   4   2,3,5   ok  0   Z + Z/2^2 + Z/3 Z/2^2   Z/2 + Z/2 +
↪  Z/3 0x4ee10c1f0eada2a3  0.056
desc_N11_n0_s098a1abfd656372a   11  0   2,3,5   ok  Z   Z^3 Z/2 Z/2 + Z/2 +
↪  Z/2 0xfaa3569ebd8f4514  0.051
desc_N11_n2_s098a1abfd656372a   11  2   2,3,5   ok  0   Z^6 + Z/2   Z/2 Z/2 + Z/2 + Z/2 + Z/2 +
↪  Z/2 + Z/2 + Z/2 0xa77d93e005bf411d  0.077
desc_N11_n4_s098a1abfd656372a   11  4   2,3,5   ok  0   Z^10 + Z/2^2 + Z/3  Z/2 Z/2 + Z/2 + Z/2
↪  + Z/2 + Z/2 + Z/2 + Z/2 + Z/2 + Z/2 + Z/2 + Z/2 0xc75fa300ff13caaa  0.148
$ cut -c1-400 descent_small_out/cells/desc_N11_n2_s*.json
```

```
{"schema":"integral_descent_cell/v1","id":"desc_N11_n2_s098a1abfd656372a","campaign":"descent_s
↪ mall","record":{"schema":"integral_descent_record/v1","N":11,"n":2,"primes":[2,3,5],"coeffi
↪ cient_ring":"Z","object":"group","q_max":3,"truncated":true,"q_max_source":"derived","degre
↪ es":[0,1,2,3],"periodic_from":2,"groups":["0","Z^6 + Z/2","Z/2","Z/2 + Z/2 + Z/2 + Z/2 +
↪ Z/2 + Z/2 + Z/2"],"bad_primes":[],
```


A refused record (a precision cap or a memory prediction) is written with `status = refused` and counted; a cell that throws is `failed` (exit 2); `resume = true` skips a cell file that ends in the trailer. From the console, `campaign_plan(spec)` of such a spec returns one group with the cell ids and `run_campaign(spec, output)` runs it. Details: `docs/matrix-engine/campaign.md` §17.

8.6 The orbifold census at $\ell = 2, 3$

The pipeline of Chapters 3–6 computes *Bredon* objects at $\ell = 2, 3$ (Section 2.1). To see the ordinary group cohomology of $\mathrm{SL}_2(\mathbb{Z})$ at those primes a campaign has a second mode, switched on by the one key `orbifold = c_i` ("the isotropy model C_I"). Without the key nothing in this section is consulted and every output is unchanged. The mode writes one record per *cell* (n, ℓ) instead of one per Hecke job. Details: `docs/matrix-engine/campaign.md` §16 and `docs/implementation/slot22/L23_1_report.md`.

What a cell holds. The record schema is `orbifold_cell/v2`; its hash is a function of the mathematics only (timings are unhashed fields).[1] For one cell it contains

- $H^q(\mathrm{SL}_2(\mathbb{Z}); \mathrm{Sym}^n \mathbb{F}_\ell^2)$ for $q \le 3$ with the Hecke operators T_p on each, the Jordan type of T_ℓ and the cup product with the Euler class e (computed over $\mathbb{F}_\ell$ on C_I);
- the torsion of $H^q(\mathrm{SL}_2(\mathbb{Z}); \mathrm{Sym}^n \mathbb{Z}^2)$ (Smith invariants over $\mathbb{Z}/\ell^K$);
- the ℓ-adic lattice of $H^1(\mathrm{SL}_2(\mathbb{Z}); \mathrm{Sym}^n \mathbb{Z}^2)$: its rank, its torsion and the elementary divisors of $T_p - (1 + p)$, $T_p - (1 + p^{n+1})$ and T_ℓ;
- a cross-check against a second model of the same group, the amalgam $\mathbb{Z}/4 *_{\mathbb{Z}/2} \mathbb{Z}/6$ (no model code shared with C_I; run for $\ell = 2$, $n \ge 1$ and for $\ell = 3$, even $n \ge 2$);
- the *gates*: every statement the cell checks, each tagged with where it comes from. A `G` gate is a theorem (of [C], of the review, or of the literature) or a consistency check between two readings of the same group (cross-model, UCT, the reader's self-check); a failure is a bug or a refutation; a `G-conj` gate tests a conjecture, and a mismatch is only recorded as a *finding*.

A divisor is printed as `0` (a unit), `inf` (the known kernel, the Eisenstein line) or `>=<cap>` (not certified below the precision cap). The record also lists the precision path the ℓ-adic reader took (`reader_path`) and the provenance of the binary (commit, path, `sha256`); neither enters the hash `record_fnv1a64`.

The spec. A section gives only `n_sym` and `ell` (with $\ell \in \{2, 3\}$); everything else is optional:

Key	Meaning
`orbifold = c_i`	switch the campaign to orbifold cells; `off` or absent: Hecke jobs, unchanged
`orbifold_primes`	the Hecke primes p whose T_p a cell computes; default `2,3,5,7`; T_ℓ is always included
`orbifold_m_target`	the ℓ-adic precision every cell must reach; default `auto`, which is $\max(\lfloor (n+1)/(\ell+1) \rfloor + 10,\ 40)$
`orbifold_k_cap`	the largest working precision the reader may climb to; default `0`, meaning its whole built range (1024 at $\ell = 2$, 312 at $\ell = 3$)

[1] `v1` was the same record with the sum-rule oracle's wall time inside the hashed gate detail; `v2` moves that time to an unhashed field. History: `docs/matrix-engine/campaign.md` §16.

```
name     = l23_first
orbifold = c_i
n_sym    = 0..700
ell      = 2
---
n_sym    = 0..700 step 2
ell      = 3
```

Run `--campaign l23.spec --dry-run` first. *Refused before any cell runs* (exit 1, no output): $\ell \notin \{2,3\}$, a `p`, `r` or `q` axis, `ext = true`, a non-default `coefficient_ring`, `jobs` > 1 (shard by writing several specs instead), a cell whose `m_target` $+ 16$ exceeds the reader's range, an unknown model, a non-prime in `orbifold_primes`. The ordinary Hecke runner refuses an orbifold spec, so a Hecke grid is never run by accident.

Cell ids and output. A cell is named `orb_ell<L>_n<N>`. A non-default `orbifold_primes`, `orbifold_m_target` or `orbifold_k_cap` (each can change a recorded value) appends a settings suffix `_s<16 hex>` to the id, as for the Hecke jobs (Section 8.4). The output directory holds `cells/<id>.json`, `summary.tsv` (one line per cell, 42 columns), `layers.tsv` (one line per degree q, layer and T_ℓ-eigenvalue), and `index.json`.

Reading the `EFFECTIVE` line. The run starts with `EFFECTIVE campaign arm=orbifold model=c_i cells=<k> primes=... m_target=... fingerprint=...` and ends with

```
EFFECTIVE campaign arm=orbifold cells=<k> ran=<r> skipped=<s> failed=<f> refused=0
↪  fallback:<b> findings=<c> stopped=<0|1> wall_s=... peak_rss_mb=...
```

Read it before trusting a shard. A clean census shard has `failed=0`, `refused=0`, `fallback:0` and `stopped=0`. `ran` are the cells computed now, `skipped` those taken from an earlier run (`resume`). `fallback:<b>` counts the cells whose status is `fallback`: a divisor that should be finite stayed `>=cap`, or the reader hit `orbifold_k_cap` unresolved; it is recorded and counted, not a failure, and a census shard must read `fallback:0`. `findings` counts failed `G-conj` gates. In the calibration runs (cells $(2,240)$, $(2,500)$, $(2,700)$, $(3,700)$, $(2,701)$) every cell read `ran=1 failed=0 refused=0 fallback:0 findings=0 stopped=0`.

Resume, and when a campaign stops. With `resume = true` a cell is skipped iff its file ends with the trailer `"orbifold_cell_complete":true` and its status is `ok` or `fallback`. A complete record whose status is `failed` *stops the campaign there*: the program prints `STOP (resume on a failed cell; [P3] R7): <id>: <first failure> -- remove or rename cells/<id>.json to re-run it`, nothing after that cell runs, `stopped=1` and `failed=1` appear in the `EFFECTIVE` line, and the exit code is 2. A failed `G` gate means a bug or a refutation, so it is to be reported, not re-run until it passes. A cell killed while writing leaves no trailer and is simply recomputed on the next start.

Memory and time. The orbifold driver has no memory limit of its own, deliberately: a hard limit inside the process would turn a failed allocation into a false `failed` cell. Limit memory from outside, one shard at a time. The calibration measured, with one cell at a time (peak resident memory, wall time): $(2,240)$ 1.7 GiB, 2.7 min; $(2,500)$ 6.8 GiB, 25 min; $(2,700)$ 13.2 GiB, 63 min; $(3,700)$ 12.2 GiB, 32 min; the odd cell $(2,701)$ 13.5 GiB, 74 min. Memory grows like $n^{1.95}$, time like $n^{2.7}$ to $n^{3.1}$. For many cells at once, plan $13.5\,\text{GiB} \cdot (n/701)^{1.95}$ per cell and keep the cells with $n \geq 600$ to a few at a time (`docs/implementation/slot22/L23_4_calib_report.md`).

A small run. The spec `docs/latex/cli_manual_runs/orbifold_small.spec` asks for the cells $(n, \ell) = (10, 2)$, $(11, 2)$, $(10, 3)$:

Real output `docs/latex/cli_manual_runs/orbifold_dry.txt`:

```
$ hacs --campaign orbifold_small.spec --dry-run
orbifold campaign 'orbifold_small': 3 cells (orbifold_cell/v2, model c_i, fingerprint
↪  0000000000000000)
    orb_ell2_n10  ell=2 n=10 m_target=40
    orb_ell2_n11  ell=2 n=11 m_target=40
    orb_ell3_n10  ell=3 n=10 m_target=40
  3 cells total
[exit 0]
```

Real output `docs/latex/cli_manual_runs/orbifold_run.txt`:

```
$ hacs --campaign orbifold_small.spec --output orbifold_small_out
=== orbifold campaign 'orbifold_small' ===
EFFECTIVE campaign arm=orbifold model=c_i cells=3 primes=2,3,5,7 m_target=auto k_cap=0
↪  fingerprint=0000000000000000 resume=0 out=orbifold_small_out
EFFECTIVE l23_ternary_f3=on source=default (h1_mod ell=3 k=1: Ops3F/TritMatrix when on, Ops3<1>
↪  when off)
EFFECTIVE l23_ring2k=on source=default (orbifold (Z) phase at ell=2: engine::ring2k smith /
↪  kernel_module when on, elim::invariant_factors / kernel_module when off)
EFFECTIVE l23_mod3k=on source=default (orbifold (Z) phase at ell=3: engine::ring3k smith /
↪  kernel_module when on, elim::invariant_factors / kernel_module when off)
EFFECTIVE l23_f3l=on source=default rho_n=recurrence ell3_f3_elim=ternary (orbifold F_3 field
↪  half: rho_n by the O(d^2) column recurrence and, at ell=3, TritMatrix RREF when on; the
↪  O(d^3) sum and the engine PLU when off)
EFFECTIVE l23_bin3o1=on source=default (amalgam reader: O(d^2) relation matrix, parity-split A,
↪  vectorised Taylor shifts; h1_mod ell=2 k=1 on F_2/BitMatrix)
  [ok  ] orbifold ell=2 n=10 h=(2,6,6,6) rankL=3 T1=[2,1] T_ell|H1=0:[1,1,1,1,1] 1:[1] K=64
  ↪  m=61 gates=61 G_fail=0 conj=0 fbk=0 0.03 s 30 MB
  [ok  ] orbifold ell=2 n=11 h=(2,6,6,6) rankL=0 T1=[1,1] T_ell|H1=0:[1,1,1,1,1] 1:[1] K=64
  ↪  m=62 gates=42 G_fail=0 conj=0 fbk=0 0.02 s 33 MB
  [ok  ] orbifold ell=3 n=10 h=(1,4,1,1) rankL=3 T1=[1] T_ell|H1=0:[1,1,1] 1:[1] K=78 m=77
  ↪  gates=80 G_fail=0 conj=0 fbk=0 0.01 s 32 MB
BIN3O1_ROUTE route=on split=2/2 odd=1 rankq=9 f2=2 reused=3 split_fallback=0 rankq_fallback=0
↪  parity_fallback=0 fallback=0
EFFECTIVE campaign arm=orbifold cells=3 ran=3 skipped=0 failed=0 refused=0 fallback:0
↪  findings=0 stopped=0 wall_s=2.43 peak_rss_mb=32.5
[exit 0]
$ cut -f1-16 orbifold_small_out/summary.tsv
cell_id ell n   weight  model   formal  status  h0  h1  h2  h3  t1  rank_lambda t2  t3  torsion1
orb_ell2_n10    2   10  12  c_i 0   ok  2   6   6   6   2   3   1   5   2;1
orb_ell2_n11    2   11  13  c_i 1   ok  2   6   6   6   2   0   4   2   1;1
orb_ell3_n10    3   10  12  c_i 0   ok  1   4   1   1   1   3   0   1   1
```

Each cell line prints the dimensions h^q over $\mathbb{F}_\ell$, the rank of the lattice, the type of torsH^1, the Jordan type of T_ℓ on H^1, the precisions K, m and the gate counts; the `EFFECTIVE` lines name the routes in force (the L23 levers of Section 9.10); the summary has 42 columns, of which the first 16 are shown.

8.7 Long census runs

The large census cells (the campaign specs with `ext = true`, and the saturation-defect cases) need no other front end than the ones above: a cell is a `--campaign` spec (Section 8.1), and a saturation-defect case is a run of `HeckeSaturationDefectValidation` with its cell list in the

environment (Section 9.14). Batch drivers that schedule many such cells over one or more machines are site-specific and are not part of this release; what any such driver should do is the following.

- **One build, one roster.** Build the binary once (CPU-only is enough for every census value; the CUDA layer changes no value), record its `sha256` and its `--list` count, and refuse to resume a run with a different binary. Records computed by two machines are comparable when the binaries come from the same source commit; recompute a few cells already known (*anchor* cells) on a new machine first and compare their records byte for byte before the bulk run.
- **Predict, then cap memory.** Predict every cell's peak resident memory before it starts (from the calibrations of Section 8.6, or the `predicted_bytes` line where a validation prints one); keep the sum of the predictions of the running cells below the free memory, and limit memory from outside the process, never inside it.
- **Threads.** `ENGINE_THREADS` per cell; one thread per cell for the saturation-defect cases.
- **Resume, never re-run blindly.** A finished cell keeps its record; a resumed run starts only the unfinished ones. A cell that fails twice with the same symptom is a finding, not a reason to change its parameters.
- **Read the `EFFECTIVE` lines** of every cell (Section 1.5) before trusting a run: they show the cells, the device route and any fallback.

For the saturation-defect target the per-cell environment is `ENGINE_THREADS=1 HECKE_SAT_DEFECT_CELLS=n:ell HECKE_SAT_DEFECT_STRICT=1`. `HECKE_SAT_DEFECT_STRICT=1` makes the validation fail every block whose reading was cut short by `kmax` (`not_stable_at_KMAX`) as well, so a too-small `kmax` is loud; without it a cell fails only when it has no `ok` block and some block is not a torsion refusal.

Chapter 9

Performance and engine switches

Every value the program prints is a function of the mathematics and the mathematical settings only. How fast and on which hardware it is produced is governed separately: by the build type (Section 1.3), the engine settings of the console (Section 7.4) and the environment variables of this chapter. The rule that makes this safe: **no environment variable of Sections 9.4–9.10 can change a value**; each is gated by a validation that compares the routes entry by entry, and most print an `EFFECTIVE` line naming the route in force.

9.1 Benchmarks: `--bench`

`--bench` drives the shipped pipeline through its public API – the same code paths as the validations – for one point, and prints phase-split timings, peak RSS and one machine-readable line

```
BENCH mode=...  n=...  ell=...  [p=...]  [r=...]  total_s=...  rss_mb=...
                                <mode-specific keys>
```

(`rss_mb` is the peak resident set of the whole process). The usage text:

```
Usage: hacs --bench <mode> <args...> [--repeat k] [--cross-checks]

Modes:
  cohom  <n_sym> <ell>            EquivariantCohomologyOnK on the SL(2,Z) model, all q
  hecke  <n_sym> <ell> <p>        T_p end to end (cosets -> tau -> Theta_g -> A^(q))
  family <n_sym> <ell> <p> <r>    T_{p^r} by recursion AND by the direct double-coset route
  decomp <n_sym> <ell> <p>        + primary module decomposition of the largest H^q
  ext    <n_sym> <ell> [p,q,...]  + the extension layer, Stages 1-5 (default family 2,3,5)
  micro  <op> <args...>           Phase-2 arithmetic micro-benchmarks; `micro --help`
                                  lists the ops (gemm/gemv/rref/kernel/transpose/suite)

Options:
  --repeat k          run the whole pipeline k times; print min/median of the totals
  --backend b         (mode `micro`) legacy | engine; default engine
                      (`legacy` = the pre-Phase-2 substrate; ENGINE_BACKEND
                      sets the default)
  --cross-checks      (mode `hecke`) additionally run the alpha/beta/omega route
                      (off by default: it roughly doubles the measured time)
  --lean              memory-lean configuration: drop the cohomology objects no
                      production consumer reads and skip the Theta_eq certification
                      (A^(q) unchanged; reported as lean=1 on the BENCH line)
  --count-allocs      report heap allocations (count and requested bytes) made
                      during each measured repeat, via a counting replacement
                      global operator new; adds allocs=/alloc_bytes= to the
                      BENCH line. With -DENGINE_COUNT_OBJECTS=ON it also reports
                      FlatMatrix / BlockFlatMatrix constructions.
```

```
Every run ends with one machine-readable line:
  BENCH mode=... n=... ell=... [p=...] [r=...] total_s=... rss_mb=... <mode-specific keys>
(rss_mb is peak RSS over the whole process, from /proc/self/status VmHWM.)

NOTE: on this model H^*(K, R; Sym^n) vanishes for ODD n (the -I / odd-n base-model
gap); use an even n_sym for a meaningful measurement.
```

All modes and options the parser accepts, including the flags that are deliberately kept out of the frozen usage text:

Token	Arguments	Meaning
`cohom`	`<n_sym> <ell>`	EquivariantCohomologyOnK on the SL(2,Z) model, all degrees.
`hecke`	`<n_sym> <ell> <p>`	T_p end to end: cosets, tau, Theta_g, A^(q).
`family`	`<n_sym> <ell> <p> <r>`	T_{p^r} by the algebra recursion and by the independent direct double-coset route.
`decomp`	`<n_sym> <ell> <p>`	`hecke` plus the primary module decomposition of the largest H^q.
`ext`	`<n_sym> <ell> [p,q,...]`	`hecke` plus the extension layer, Stages 1-5; the family defaults to 2,3,5 (2,3,5,7 under `--campaign-cell`).
`micro`	`<op> <args...>`	Phase-2 arithmetic micro-benchmarks (the `micro` table).
`--repeat`	`<k>`	Runs the whole pipeline k >= 1 times and prints min / median of the totals.
`--backend`	`legacy|engine|gpu`	The substrate of `micro` (default `engine`; `ENGINE_BACKEND` sets the default).
`--cross-checks`, `--no-cross-checks`	–	(`hecke`) also run, or explicitly skip, the classical alpha/beta/omega route; off by default (it roughly doubles the time).
`--lean`	–	Memory-lean configuration: drop objects no production consumer reads and skip the Theta_eq certification; A^(q) unchanged, `lean=1` on the BENCH line.
`--count-allocs`	–	Reports the heap allocations of each measured repeat (`allocs=`, `alloc_bytes=` on the BENCH line).
`--campaign-cell`	–	(`ext`) the campaign cell's own driver options: cone and sign split built, default family 2,3,5,7; not in the frozen help text.
`--u-ell`	`auto|off|budget_mb=<M>`	(`ext`) scopes the MATHEMATICAL setting `u_ell` for the run: it decides whether U_ell enters the record, so it can change a value; not in the frozen help text.
`--help`, `-h`	–	Prints the bench usage (after `micro`: the micro usage).

Value-preserving and value-changing flags. `--repeat`, `--lean`, `--count-allocs`, `--cross-checks`, `--backend` and the three lever pairs never change a computed value; equal checksums across them are part of what the validations assert. The one exception is `--u-ell`, which sets the *mathematical* setting `u_ell` (Section 7.4) for an `ext` run and so decides whether U_ℓ enters the extension record.

The `--backend` values.

Token	Arguments	Meaning
`engine`	–	The Phase-2 ModCtx / DenseBuf / Kernels layer; the default and the shipped substrate.
`legacy`	–	The pre-Phase-2 substrate end to end (an engine::elim::ScopedLegacy guard); equal checksums against `engine` are a bit-identity statement.
`gpu`	–	The Phase-5 device kernels (gemm / gemv / transpose); rref / kernel and every op without a usable device report `not available`, exit 2.

A run on the reference machine:

Real output `docs/latex/cli_manual_runs/bench_hecke.txt`:

```
$ hacs --bench hecke 48 134217757 2
=== bench hecke (n=48 ell=134217757 p=2, repeat=1) ===
    [A^(0)=0x0 A^(1)=7x7 A^(2)=0x0]
    setup (K + model + relations)           0.0007 s
    cohomology (all q)                      0.0204 s
    coset tables                            0.0000 s
    tau table (+ exact certification)       0.0004 s
    operator assembly (Theta + A^(q))       0.0084 s
    cohomology-level extraction             0.0000 s
    TOTAL                                   0.0299 s
BENCH mode=hecke n=48 ell=134217757 p=2 total_s=0.0299 rss_mb=31.5 dims=0,7,0 xcheck_rc=skipped
[exit 0]
```

9.2 Micro-benchmarks

`--bench micro <op> <args...>` measures single arithmetic kernels on fixed-seed inputs; the reported FNV-1a-64 checksum of the result is a bit-identity check between backends.

```
Usage: hacs --bench micro <op> <args...> [--repeat k] [--backend legacy|engine|gpu]

Ops (every <ell> defaults to 134217757, the benchmark prime of doc 03):
  gemm      <n> [<k>] [<ell>]                 C = A*B, A: n x k, B: k x n; work = n*n*k
  gemv      <n> [<ell>]                       y = A*x, A: n x n; work = n*n
  rref      <rows> <cols> [<rank>] [<ell>]    RREF of a matrix of prescribed rank
  kernel    <rows> <cols> [<rank>] [<ell>]    kernel() of the same matrix family
  transpose <n> [<ell>]                       transpose of an n x n matrix
  suite                                       the Phase-2 exit-criterion set + floors
  cuda                                        the Phase-5 GPU throughput table (needs a device)

<rank> omitted, 0, or >= min(rows,cols) means FULL rank (a plain random matrix,
which is what doc 03 Sec. 7.3's reference elimination uses); a smaller <rank>
builds L(rows x rank) * R(rank x cols) so rank-deficient inputs are covered.

All inputs are fixed-seed (splitmix64 over the shape), so two runs -- and the two
backends -- see byte-identical inputs; the reported FNV-1a-64 checksum of the
result therefore doubles as a bit-identity check between backends.

--backend engine selects the Phase-2 ModCtx/DenseBuf/Kernels layer (the default).
--backend gpu selects the Phase-5 engine::cuda device kernels (gemm / gemv /
transpose; rref / kernel stay CPU-driven and report "not available", exit 2).
Without a usable CUDA device every gpu op reports "not available", exit 2.
```

Token	Arguments	Meaning
`gemm`	`<n> [<k>] [<ell>]`	C = A*B with A: n x k, B: k x n; work n*n*k.
`gemv`	`<n> [<ell>]`	y = A*x with A: n x n.
`rref`	`<rows> <cols> [<rank>] [<ell>]`	RREF of a fixed-seed matrix of prescribed rank (omitted, 0 or >= min(rows, cols): full rank).
`kernel`	`<rows> <cols> [<rank>] [<ell>]`	kernel() of the same matrix family.
`transpose`	`<n> [<ell>]`	Transpose of an n x n matrix.
`suite`	–	The Phase-2 exit-criterion set with its floors.
`cuda`	–	The Phase-5 GPU throughput table (needs a device).
`--help`, `-h`, `help`	–	Prints the micro usage.

Reference figures. The standing performance gates (`--bench hecke 256 134217757 2`, `--bench decomp 128 134217757 2`) and the measured tables live in `docs/performance-reference.md`; timings depend on the machine and its load and are never part of a result.

The standing gates. Two points guard the pipeline's speed from slot to slot: `--bench hecke 256 134217757 2` must stay at or below 0.470 s and `--bench decomp 128 134217757 2` at or below 0.109 s on the reference machine, and the median of 13 alternating rounds against the previous slot's binary must not be slower by more than 4 % (`docs/matrix-engine/tools/standing_gate_h2h.sh`); the fitted exponent of the total time in n over $\{64, 96, 128\}$ stays ≤ 3.2. Compare two binaries only when they place the global thread pool alike (`docs/matrix-engine/tools/pool_placement_check.sh`).

9.3 Environment variables: overview

Environment variables fall into five groups: the engine switches (this section to Section 9.10), which can never change a value; the owner-gated emitters, which write files; the interlocks and the strict acceptance mode; and the read-only hooks of individual validations. Unless a variable of Section 9.12 is set, `--all` and `--all-full` are read-only. Three conventions hold throughout:

- A *rollback arm* is a variable whose value `0` restores an older route verbatim (the new route is the default); any other value prints a `WARNING` and keeps the default. It exists to compare and to recover, not to tune.
- An *opt-in route* is off unless set to exactly `1`.
- Most variables are read once per process; set them on the command line of the run.

9.4 Threads, substrate and the tape engine

Variable	Effect
`ENGINE_THREADS`	worker threads of the engine pool (console: `threads`); the descent's Smith forms use it too (an integer in [1, 256]; default the CPU affinity count, at most 64)
`ENGINE_HOST_THREADS`	the host-side fan-out of independent tasks (one thread per task); `0` runs them serially (env-only)
`ENGINE_BACKEND`	`engine` (default) or `legacy`, the pre-engine substrate (console: `backend`)
`ENGINE_BATCH`, `ENGINE_KSPLIT`, `ENGINE_ARENA`, `ENGINE_CSE`, `ENGINE_FUSE`, `ENGINE_LAYOUT`	the tape compiler's passes (console: `batch`, `ksplit`, `arena`, `cse`, `fuse`, `layout`)

Variable	Effect
ENGINE_TILE_MULT, ENGINE_TILE_COLS, ENGINE_SPIN, ENGINE_ELIM_BASE, ENGINE_ELIM_PANEL, ENGINE_ELIM_TRSM_BASE	tuning of the tiles, the spin-wait and the blocked elimination (panel, base case, triangular-solve base), read once at start-up (env-only settings)
ENGINE_HOST_ELIM_HOOK	0: rollback arm; the cone and ι^* certificates no longer bind the elimination's inner parallel hook to the pool
ENGINE_LADIC_WIDTH	32, 64 or 128: the digit width of the ℓ-adic route for $\ell^k \geq 2^{64}$; the records are width-independent

The console's engine settings of the same names override the environment for a session; the *env-only* ones cannot be changed after start-up (`set tile_mult = "2"` is refused with a message naming ENGINE_TILE_MULT).

9.5 CPU kernels and native small fields

The matrix product over $\mathbb{F}_p$ ("GEMM") is the inner loop of almost everything. It has several exact implementations, chosen at run time from what the CPU offers; all give the same residues.

Variable	Effect
ENGINE_KERNEL	auto (default: AVX-512 if the CPU has it, else AVX2, else portable), avx512, avx2 or portable: the 32-bit GEMM microkernel. avx2 on an AVX-512 machine imitates a CPU without AVX-512 (e.g. Zen 3). An unavailable value warns and stays auto; a set value prints EFFECTIVE ENGINE_KERNEL path= value= cpu=avx2:<0|1>,avx512:<0|1>
ENGINE_AVX2_PAIR	0 (or off): rollback arm of the AVX2 pair route (16-bit products, $p \leq 4096$); the AVX2 path then runs its 64-bit microkernel for every p
ENGINE_VNNI, ENGINE_VNNI_SHADOW	the AVX-512 VNNI route for $p \leq 128$ and its two-limb form for $128 < p \leq 2^{14}$, on by default where the CPU has avx512_vnni; ENGINE_VNNI=0 is the rollback arm, ENGINE_VNNI_SHADOW=1 recomputes every routed product on the 32-bit kernel and stops on a difference
ENGINE_VNNI2_FLOOR	the two-limb VNNI route's work-per-thread floor ($mnk \geq$ floor $\times$ threads, default 2^{18}, threads = the configured pool size); 0 restores the slot-24 rule; a value that is not a string of digits (or overflows 64 bits) warns and keeps the default; a set value prints EFFECTIVE ENGINE_VNNI2_FLOOR floor= value= threads= when the floor is first consulted (under auto, at the first two-limb product that passes the large-p test); it cannot change a value
ENGINE_BINARY_F2	1: opt-in; the dense eliminations over $\mathbb{F}_2$ run bit-packed (Four Russians), Section 9.7
ENGINE_TERNARY_F3	1: opt-in; the dense eliminations over $\mathbb{F}_3$ run bit-sliced (two bit planes per row), proved entrywise by TernaryF3Validation

Example: the AVX2 kernel without the pair route on the reference machine (an AVX-512 CPU):

Real output docs/latex/cli_manual_runs/env_kernel.txt:

```
$ ENGINE_KERNEL=avx2 ENGINE_AVX2_PAIR=0 hacs --bench micro gemm 256
EFFECTIVE ENGINE_KERNEL path=avx2 value=avx2 cpu=avx2:1,avx512:1
=== bench micro gemm (backend=engine, repeat=1) ===
    gemm            0.000749 s (min)
```

```
      rate        = 22.399 Gmodmul/s  (0.0168 Gmodmul of work)
      note        = AVX2 microkernel
      checksum    = 0x5ae13caf41f0a5bf
BENCH mode=micro op=gemm n=256 k=256 ell=134217757 total_s=0.0007 rss_mb=131.8 backend=engine
↪  median_s=0.000749 gmodmul_s=22.3995 checksum=0x5ae13caf41f0a5bf
[exit 0]
```

9.6 The device: ENGINE_DEVICE and the CUDA layer

The default device policy is `cpu`: the golden path, which never creates a CUDA context. A GPU is used only when asked for. The reference machine's card is (temporarily) an NVIDIA RTX PRO 6000 Blackwell with 96 GB (compute capability 12.0, like the RTX 5070 Ti it replaces).

Variable	Effect
ENGINE_DEVICE	`cpu` (default, the golden path), `gpu`, `hybrid` (console: `device`). On the Hecke pipeline's shapes the GPU is slower than the CPU and stays opt-in; for the integral descent's 2-adic Smith form, `gpu` is the fast route (Section 9.8)
ENGINE_CUDA_DISABLE, ENGINE_CUDA_DEVICE, ENGINE_CUDA_PROBE	disable the CUDA layer, pick a device, probe; `ENGINE_CUDA_DISABLE=1` is required for sanitizer runs
ENGINE_CUDA_GRAPH, ENGINE_CUDA_MERGE_LEVELS, ENGINE_CUDA_PROGRAM_CACHE, ENGINE_ELIM_SESSION	GPU tuning knobs, only under a GPU device policy. `ENGINE_CUDA_GRAPH`: `0` never capture a CUDA graph, `1` (default) capture on the second run of a program, `2` on the first. `ENGINE_CUDA_MERGE_LEVELS=0` stops merging consecutive GPU-only wavefronts into one graph. `ENGINE_CUDA_PROGRAM_CACHE=<k>` sets the number of cached programs (default 64). `ENGINE_ELIM_SESSION=0` turned off the device-resident elimination session; retired at the slot-24 close (CL-R2): refused in a default build (`OC_ENABLE_LEGACY=OFF`), a comparison arm only in a legacy build
ENGINE_CUDA_PINNED_MB, ENGINE_CUDA_PINNED_CHUNK_MB, ENGINE_CUDA_COPY_THREADS, ENGINE_CUDA_ARENA_SLACK_MB	the pinned host staging pool (size, chunk), the threads of the copy pool, and the idle bytes the device arena keeps before trimming
ENGINE_GPU_LEAN	by default a program whose GPU nodes are all memory-bound runs on the host, and the arena keeps a sixteenth of the card as slack; `0...` (the former rollback arm) is retired (S25 LEVERS-25): refused in a default build, the old route in a legacy build
ENGINE_GPU_IMMA	the int8 tensor-core GEMM for $p \leq 127$, always on; `0...` (the former rollback arm) is retired (CL-R2): refused in a default build, a comparison arm in a legacy build (Section 9.7)
ENGINE_GPU_ELIM, ENGINE_GPU_ELIM_MIN, ENGINE_GPU_ELIM_VERIFY	the device-resident elimination for $p \leq 127$: `0...` retired (CL-R2; refused in a default build), size floor in matrix entries (default 2^{20}), host cross-check (Section 9.7)
ENGINE_GPU_SMITH2K, ENGINE_GPU_SMITH2K_MIN, ENGINE_GPU_SMITH2K_VRAM_MB, ENGINE_GPU_SMITH2K_HOST_MB, ENGINE_GPU_SMITH2K_VERIFY	the device 2-adic Smith LU of the descent, under `ENGINE_DEVICE=gpu` only (Section 9.8): rollback arm (`0`), size floor $m \cdot n$ (default 2^{24}), device budget (default the free memory less 512 MiB and 1/32 of the card), the host store of the tiled route per phase (default 4096 MiB), and a host re-run of every device LU (`1`)

9.7 Native $\mathbb{F}_2$ arithmetic and the GPU elimination

Three routes sit beside the tape and block code. All three are *value-preserving*: the reduced row echelon form, the rank and the pivots of a matrix over a field are unique functions of the matrix, and each route computes them in exact integer arithmetic, so a record is byte-identical with the route on or off. None is a mathematical setting, none enters a record's hash or a campaign job id, and none is on by default on the golden path (`ENGINE_DEVICE=cpu`). The contract is `ENGINE_API.md`, Sections 8.1, 11.8 and 11.9.

Native $\mathbb{F}_2$ arithmetic: `ENGINE_BINARY_F2=1`. Opt-in, off by default, and read once per process; only the value `1` switches it on. The in-place dense eliminations over a field with $p = 2$ (`uint32_t` or `uint64_t`) – `rref`, `rref_with_transform`, `inverse`, `solve_multi_rhs`, `complement_basis` and `column_rank_profile` – then pack the matrix into one bit per entry and eliminate with the Four-Russians method (M4RI, strips of eight pivots; AVX-512 or AVX2 row XORs chosen at run time, a portable body on other CPUs), single-threaded. The tape elimination and the block model are not routed, and no other modulus is affected. Measured on the reference machine at $N \times N$ with `--bench micro rref`: 12× at $N = 1024$, 46× at 4096, 70× at 8192 (identical checksums). Gate: `BinaryF2Validation` (`standard`); `GOLDEN_STRICT=1 ENGINE_BINARY_F2=1 ./hacs GoldenRegressionValidation` is clean.

GPU routes for $p \leq 127$: `ENGINE_GPU_IMMA`, `ENGINE_GPU_ELIM`. Both are reached only under a GPU device policy (`ENGINE_DEVICE=gpu`, or `hybrid` for the GEMM); the default `cpu` never creates a CUDA context, and without a device of compute capability ≥ 8.0 the program silently keeps the CPU kernels.

- `ENGINE_GPU_IMMA`: modular GEMM on the int8 tensor cores for moduli $2 \leq p \leq 127$ (balanced residues, exact `int32` accumulation). Always on for the device GEMMs; a value starting with `0` was the rollback arm until the slot-24 close (CL-R2) and is now refused in a default build (`OC_ENABLE_LEGACY=OFF`); a legacy build still honours it. Gate: `CudaImmaBitIdentityValidation` (`standard`).
- `ENGINE_GPU_ELIM`: the whole elimination on the device for a `uint32_t` field with $p \leq 127$, replacing the tape elimination and the dense RREFs, and only under `ENGINE_DEVICE=gpu`. A value starting with `0` turns it off in a legacy build only; a default build refuses it (retired, CL-R2), and `ENGINE_DEVICE=cpu` remains the way to stay off the GPU. `ENGINE_GPU_ELIM_MIN=<m*n>` is a size floor (default 1 048 576 entries; tuning only). `ENGINE_GPU_ELIM_VERIFY=1` re-runs each device elimination on the host and compares the result entrywise (a difference raises an error); `ENGINE_TRACE_IMMA_ELIM=1` prints the phase times of each call. Gate: `CudaImmaElimValidation` (`standard`). `Z/ell^k` is not covered.

A device elimination that fails before writing output (for instance a memory allocation) falls back to the host route with the same result. The fallback is never silent: one stderr line `[imma_rref] fallback to the host route (<route>, <m>x<n>): <reason>` per fallback, and one summary line

```
EFFECTIVE DEVICE_ELIM calls=<n> device=<n> fallbacks=<n>
```

on stderr when the process exits, if any device elimination was attempted (`device` counts the eliminations that ran on the GPU, `calls` = `device` + `fallbacks`). Read it after a GPU run: `fallbacks=0` means every elimination ran on the device. A process that never attempted one prints no such line. `ENGINE_GPU_ELIM_SIMULATE_CUDA_ERROR=1` is a test hook that forces the fallback path.

9.8 The integral descent: routes, threads and the GPU

The descent of Chapter 5 spends almost all of its time in Smith forms over $\mathbb{Z}/2^k$ and $\mathbb{Z}/3^k$ of large dense matrices (the 2-adic piece at level N has about $\psi(N)(n+1)\cdot\frac{4}{3}$ rows). Its routes, all value-preserving, are on by default; each lever below is a rollback arm (`0`) gated by the validation named in `docs/validation-catalogue.md`.

Variable	Effect
`HECKE_DESCENT_TREE`, `HECKE_DESCENT_TREE_PAIR`	the tree models of the 2- and 3-adic pieces and of the pair (closed-form cochains); `0` restores the dense isotropy model (same groups and maps)
`HECKE_DESCENT_GOOD_SPARSE`, `HECKE_DESCENT_GOOD_DENSE`	the good-prime step at level 1: the sparse cokernel (rollback: the seven-vertex pipeline), read as dense ranks of the averaging idempotents (rollback: the sparse elimination)
`HECKE_DESCENT_SMITH_MT`	the threaded 2-/3-adic Smith forms on `ENGINE_THREADS` threads; `0` (or `ENGINE_THREADS=1`) the serial code. Prints `EFFECTIVE HECKE_DESCENT_SMITH_MT phase=start|exit ...` when set
`HECKE_DESCENT_SMITH_PANEL`	the LU panel width of the threaded Smith form. **Unset = auto**: the 2-adic host LU takes **512** columns when the matrix it factors has $\geq$**12 000** rows, else 128; the 3-adic LU 128; the device LU always 128. An explicit width in $[1, 4096]$ wins for every size; `128` restores the earlier default (the rollback), `64` the first threaded one
`HECKE_DESCENT_SMITH_F2_MT`	the lazy blocked $\mathbb{F}_2/\mathbb{F}_3$ rank profile of each Smith level and the vectorised level passes
`HECKE_DESCENT_SMITH_BLOCKED`, `HECKE_DESCENT_SMITH_SIMD`	the blocked, threaded integer Smith form of the maps and presentations (rollback: the scalar oracle); `_SIMD=scalar|avx2|avx512` forces its kernel body
`HECKE_DESCENT_PAR_ASM`, `HECKE_DESCENT_PAR_ELL`, `HECKE_DESCENT_PAR_LN`, `HECKE_DESCENT_EMBED_MAP`	concurrency of the assembly: the pair's complex assembly, the $\ell \geq 5$ pieces, the level-N stages (each under a joint memory prediction), and the cone maps without a dense cochain map

The GPU route. Under `ENGINE_DEVICE=gpu` the 2-adic Smith LU runs device-resident (tiled when the matrix does not fit) above the size floor `ENGINE_GPU_SMITH2K_MIN`; the result is bit-identical, and the run prints `EFFECTIVE GPU_SMITH2K ... device=<calls> resident= tiled= fallbacks=` at exit. On the 96 GB card every LU of the largest cells is resident: $\Gamma_0(1000)$, $n = 20$ takes 142.8 s on the GPU against 903.9 s on 16 CPU threads, with a predicted peak of about 36.7 GiB. Below the floor nothing changes; with the floor at 0 even a small cell uses the device:

Real output `docs/latex/cli_manual_runs/env_descent_gpu.txt`:

```
$ ENGINE_DEVICE=gpu ENGINE_GPU_SMITH2K_MIN=0 hacs -c 'print(descent(1, 30).groups[1])' --set
↪  coefficient_ring=Z
EFFECTIVE GPU_SMITH2K mode=on policy_gpu_calls=7 device=7 resident=7 tiled=0 slabs=0 replays=0
↪  fallbacks=0 declined_vram=0 declined_host=0 panels=7 lookahead=0 verified=0 min_entries=0
↪  t_s=open:0.01,fetch:0.00,panel:0.00,step:0.00,host:0.00,close:0.00
Z^5 + Z/2 + Z/2 + Z/2 + Z/2^2 + Z/2^2 + Z/2^2 + Z/3 + Z/3 + Z/3^2 + Z/5^2 + Z/29
[exit 0]
```

And the panel lever in action (an explicit width; the start and exit lines show the width in force and how many LU calls took each width):

Real output `docs/latex/cli_manual_runs/env_descent_panel.txt`:

```
$ HECKE_DESCENT_SMITH_PANEL=128 hacs -c 'print(descent(1, 30).groups[1])' --set
↪  coefficient_ring=Z
EFFECTIVE HECKE_DESCENT_SMITH_MT phase=start mode=threaded lever=on source=default threads=32
↪  threads_source=affinity panel=128 f2=threaded f2_source=default
```

```
EFFECTIVE HECKE_DESCENT_SMITH_MT phase=exit mode=threaded lever=on source=default threads=32
↪  threads_source=affinity panel=128 f2=threaded f2_source=default max_used=1
↪  regions=parallel:0,serial:0 d2_regions=0 tasks=0 panel_splits=15 scoped_regions=0
↪  lu_panels=128:7,512:0,other:0 f2_lazy=15 f2_par_searches=0
↪  t_s=f2_pack:0.00,f2_ech:0.00,f3_pack:0.00,f3_ech:0.00,level:0.00,lu:0.00,ech_search:0.00,ec
↪  h_flush:0.00,ech_table:0.00,perm:0.00,schur:0.00
Z^5 + Z/2 + Z/2 + Z/2 + Z/2^2 + Z/2^2 + Z/2^2 + Z/3 + Z/3 + Z/3^2 + Z/5^2 + Z/29
[exit 0]
```

9.9 The Hecke module and forms routes

The U_ℓ operator, the $\mathbb{Z}/\ell^k$ module route (saturation and Loewy cells of the census), and the forms algebra each have memory and speed levers. All are value-preserving. Where a lever is still a rollback arm, `0` restores the earlier route; a *retired* arm is refused in a default build (`REFUSED <NAME>=0: compiled out (OC_ENABLE_LEGACY=OFF)`, exit 1) and works as before in a legacy build (`-DOC_ENABLE_LEGACY=ON`); the earlier routes themselves stay, as the comparisons of the validations.

Variable	Effect
`HECKE_U_ELL_LOW_RANK`, `HECKE_COSET_STREAM`, `HECKE_PARALLEL_SETUP`, `HECKE_RANK1_CLOSED_FORM`	the four memory levers of U_ℓ: low-rank factorisation, streamed cosets, parallel set-up, the closed-form rank-1 terms; all four rollback arms retired (S24 CL-R1: refused in a default build)
`HECKE_TAU_WIDE`	the wide tier of U_ℓ for $\ell > 23170$ (`0`: such ℓ are refused as before)
`HECKE_RHO2_STREAM`	the cross-check route of an operator streamed key by key (no per-degree memo); `0` retired (S25 LEVERS-25)
`HECKE_MODULE_STREAM`, `HECKE_MODULE_EVAL`, `HECKE_RHO_SIMD`, `HECKE_ROUTE2_SHARED_MEMO`	the streamed $\mathbb{Z}/\ell^k$ module route: read-offs at unit pivots, fast ρ_n evaluation (SIMD: `0|scalar|avx2|avx512`), one memo per coset for the cross-check; `0` (and `scalar` for `HECKE_RHO_SIMD`) retired (S25 LEVERS-25)
`HECKE_MODULE_PARALLEL`, `HECKE_MODULE_MT`, `HECKE_MODULE_TILE_WORK`, `HECKE_MODULE_LEAN_PEAK`	its parallel assembly, threaded decompositions, the minimum work per parallel tile, and the lean peak memory; `HECKE_MODULE_PARALLEL=0` retired (S25 LEVERS-25)
`HECKE_PRECISION_REUSE`	one module run per cell with the lower precisions derived (`0`, one run per level: retired, S25 LEVERS-25)
`HECKE_SAT_PERF`	per-cell memos of the saturation certificates
`HECKE_FIELD_DIET`	the memory diet of a field cell (dense programs in parts, early release, `malloc_trim`)
`HECKE_FORMS_FASTMUL`, `HECKE_FORMS_NTT_CACHE`, `HECKE_FORMS_NTT3`	the forms algebra's q-series products by number-theoretic transforms, their cache, and the three-prime route for $\ell \geq 2^{16}$; `HECKE_FORMS_FASTMUL=0` retired (S25 LEVERS-25)
`HECKE_LEVEL_N_GEN_ORDER`	`sorted` (default) or `as_built`: the generator order of the level-N cokernel (any other value is an error)

9.10 The orbifold census routes (`L23_*`)

The orbifold census at $\ell = 2, 3$ (Section 8.6) prints one `EFFECTIVE l23_...=on|off` line per route (see the example there). Each is on by default; `0` restores the earlier route, except for the two retired arms (`L23_TERNARY_F3=0`, `L23_BIN3O1=0`, S25 LEVERS-25: refused in a default build, the earlier route in a legacy build).

Variable	Effect
L23_TERNARY_F3	the degree-1 amalgam reader's $\mathbb{F}_3$ route (bit-sliced eliminations); 0 retired
L23_RING2K, L23_MOD3K	the Smith readings of an $\ell = 2$ (resp. 3) cell over $\mathbb{Z}/2^{31}$ (resp. $\mathbb{Z}/3^{19}$) on the native ring engines
L23_F3L	ρ_n by the $O(d^2)$ column recurrence and the $\mathbb{F}_3$ eliminations of an $\ell = 3$ cell
L23_BIN3O1	the amalgam reader's $O(d^2)$ relation matrix, parity split and vectorised Taylor shifts; 0 retired

9.11 Diagnostics and test-only variables

Variable	Effect
ENGINE_TRACE_NODES, ENGINE_TRACE_RSS, ENGINE_TRACE_ELIM, ENGINE_TRACE_LEVELS, ENGINE_TRACE_PASSES, ENGINE_TRACE_DEVICE, ENGINE_TRACE_DEVICE_RELEASE, ENGINE_TAPE_TRACE	diagnostic traces on standard output
ENGINE_TRACE_VRAM=<ms>	samples the device memory every <ms> and prints a summary at exit
ENGINE_TRACE_IMMA_ELIM, ENGINE_KERNEL_TIMING	per-call phase times of the device elimination; the timing table of KernelBitIdentityValidation
ENGINE_GPU_ELIM_SIMULATE_CUDA_ERROR	test-only error injection for the counted host fallback
ENGINE_SCALING_REPS, ENGINE_TRAP_REPS	repetitions of two engine validations

9.12 Owner-gated emitters, interlocks and strict mode

Owner-gated: writes files

The variables in this box make an otherwise read-only command **write files**. Two of them rewrite tracked reference files of the repository (a *re-baseline*, which only the project owner performs, explaining every changed line). Never set one of them to "make a validation pass".

- `GOLDEN_EMIT=1 ./hacs GoldenEmit` (optionally `GOLDEN_OUT=<path>`) regenerates the golden reference `Golden/GoldenReference.inc`.
- `CONSOLE_EMIT_EXPECTED=1 ./hacs ConsoleScriptEmit` regenerates the frozen `.expected` transcripts of `Console/scripts/`.
- `GOLDEN_DUMP=<dir> ./hacs GoldenDump` writes the before/after object dump of the re-baseline protocol into `<dir>`.
- `HECKE_DUMP_TAU=<dir>` (`HeckeTauDumpValidation`) and `HECKE_DUMP_SAT_CERT=<dir>` (`HeckeSatCertDumpValidation`) write data files for the Lean checkers into a scratch directory, which must exist and lie outside the source tree. Their sub-knobs select what is written: `HECKE_DUMP_TAU_RHO_CELLS="n:ell,..."` (the ρ_n cells, default $(2,5),(10,691),(16,13)$), `HECKE_DUMP_TAU_PAIR` (the τ-table pairs) and `HECKE_DUMP_SAT_CERT_CELLS="n:ell:kmax,..."` (default `168:13:2,220:17:2,258:37:2`); a set-but-empty or malformed value is refused (exit 1).
- `CLI_MANUAL_EMIT=<dir>` (`CliManualCoverageValidation`) writes this manual's generated tables into a scratch directory outside the tree (Appendix E).
- `ORBIFOLD_MP_DUMP=<dir>` (`OrbifoldMPRulesValidation`) writes six `mp_*.tsv` data files of the Moerdijk–Pronk model (orbits, points, top simplices, arrows, predicates, $|S_0|,|S_1|,|S_2|$) for the Lean cross-checks, and prints `EFFECTIVE OrbifoldMPRulesValidation arm=dump dir=<dir> files=6 fnv=0x...`. The directory must exist and lie outside the source tree, otherwise the validation prints `REFUSED` and fails; unset, nothing is written. It changes no value.
- `HECKE_ATLAS_OUT=<dir>` lets `--atlas` write its store; `HECKE_EXT_JSON_OUT=<file>` makes `HeckeExtRamanujanValidation` write the $(10,691)$ extension record to a file.

- `GOLDEN_STRICT=1` is not an emitter: it makes `GoldenRegressionValidation` gate its tier B (bit-exact objects) as well as tier A.

- `HECKE_LOCALITY_PROBE=1` and `HECKE_HIDA_HEAVY=1` are *interlocks on a run*: without them `HeckeLocalityProbe` and `HeckeHidaHeavyCellProbe` print `[SKIP]` even under `--all-full`. The second can need tens of GiB of memory – predict the peak RSS before enabling it.

9.13 Read-only validation hooks

Many validations accept a variable that widens or replaces the list of cells an arm runs over; none of them writes or changes a value. They share a naming scheme:

Pattern	Meaning
`HECKE_<ARM>_ALL=1`	run an arm over its whole published grid (e.g. `HECKE_CENSUS_SPLIT_ALL`, `HECKE_THETA_IMAGE_ALL`)
`HECKE_<ARM>_CELLS="n:ell,..."`	run an arm over the listed cells instead of its default list (e.g. `HECKE_RING_LOEWY_CELLS`, `HECKE_SAT_DEFECT_CELLS`, `HECKE_NEBENTYPUS_CELLS`, `HECKE_LEVEL_N_SPARSE_CELLS`, `HECKE_HIDA_COMPONENTS_CELLS`); a malformed cell is refused by its position

Pattern	Meaning
`..._U_ELL=<mb>`, `..._RECORDS=<dirs>`, `..._TSV=<file>`	the U_ℓ budget, the record roots or the census table an arm reads
`..._LARGE=1`	add a few expensive cells to a coverage validation
`ORBIFOLD_MP_CELLS=4,6` (or `4`, `6`, `all`)	adds the nerve-H^2 cells $n \in \{4, 6\}$ to `OrbifoldMPNerveValidation` (12 cells, about 5 s); prints `EFFECTIVE OrbifoldMPNerveValidation arm=h2 cells=<k> n=<list>`; any other token is refused (exit 1)
`HECKE_TAMEFAMILY_HEAVY`, `HECKE_TAME_FAMILY_CELLS="p:l0:l1,..."`	the heavy cell, respectively a survey list, of `HeckeTameFamilyValidation` (the $\Gamma_H(N)$ tame-family check)
`HECKE_LAMBDA_TOWER_CELLS="l:a:r[:K],..."`	the cell list of `HeckeLambdaTowerValidation`; the $r = 3$ cells,
`HECKE_LAMBDA_TOWER_R3=1`	refused by name above the caps `HECKE_LAMBDA_TOWER_RSS_CAP_GIB` (default 4) and `HECKE_LAMBDA_TOWER_OPS_CAP`
`HECKE_NEB_WILD_ALL=1`	adds the three-class cell $(7, r = 3)$ to `HeckeNebentypusWildValidation` (about $2.6 \cdot 10^{11}$ operations)
`HECKE_SHAPIRO_CELLS="N:n:ell:m,..."`	extra cells of `HeckeLValueShapiroValidation` (ranks on $\Gamma_0(N)$ by Shapiro against Eichler–Shimura)
`HECKE_LN23_STURM=1`	`HeckeLevelNCurveValidation` runs only the Sturm separating-family arms
`NF_PRANK_CE=CE1,CE2,CE1C,CE2C|all`, `NF_PRANK_CELLS="N:p,..."`, `NF_PRANK_DENSE`	cells and dense bound of `NumberFieldPRankValidation`
`CUDA_IMMA_ELIM_BENCH=m,n[,p]`	times one random and one sparse $m \times n$ device elimination in `CudaImmaElimValidation` (arm `bench`)
`HECKE_NEBENTYPUS_NB3_FULL=1`	the full 132-cell nebentypus grid (run at every slot close)
`HECKE_LEVEL_N_HOOK_RSS_CAP_GIB`	the pre-run RSS cap of the level-N hooks (default 8)
`CONSOLE_DETERMINISM_ARMS=threads`	restrict `ConsoleDeterminismValidation` to its thread arms

The descent validations take the same kind of cell lists: `HECKE_DESCENT_CELLS`, `HECKE_DESCENT_REPEATS`, `HECKE_DESCENT_LEVELN_CELLS`, `HECKE_DESCENT_N_CELLS`, `HECKE_DESCENT_PAIR_CELLS`, `HECKE_DESCENT_BDRY_CELLS`, `HECKE_DESCENT_TREE_CELLS`, `HECKE_DESCENT_TREEPAIR_CELLS`, `HECKE_DESCENT_HECKE_CELLS`, `HECKE_DESCENT_PARLN_CELLS` replace the cost cells of the descent's slow validations, and `HECKE_DESCENT_SMITH_MT_BENCH`, `HECKE_DESCENT_SMITH_MT_CELLS`, `HECKE_DESCENT_SMITH_MT_THREADS`, `HECKE_DESCENT_SMITH_MT_REPEATS`, `HECKE_DESCENT_SMITH_MT_PANEL`, `HECKE_DESCENT_SMITH_GPU_CELLS` and `HECKE_DESCENT_SMITH_GPU_ARMS` add timing arms to `DescentSmithMtValidation` and `DescentSmithGpuValidation`.

Every such arm prints its `EFFECTIVE` line. The complete table, with the cost and memory of each hook, is in `docs/validation-catalogue.md`.

9.14 Hooks of the level-N curve and the Λ-adic tower

Two validations take a family of read-only variables. As for every hook of this section they only choose which cells run (or how far a certificate climbs), never write, and never change a value; each arm prints an `EFFECTIVE` line, which is the proof that the variable was read.

`HeckeLevelNCurveValidation` (LN23). The validation computes, for the weight-2 Eisenstein block at prime level N and $p = 2, 3$, the Hecke module W of the *curve* $X_0(N)$ (its dimension, Loewy length and layers) and compares it with the published results of Calegari–Emerton (CE) on the Eisenstein ideal. It is a `standard` validation (about 19 s, 1177 checks). A reading is taken with the Hecke primes $q \le Q$ and is *certified* when the family contains a Mazur good prime, the

reading stays the same under the next two widenings, and the layers have the curvilinear shape $d, d-2, \ldots, 3, 1$ (or $\dim W = 1$); the validation then compares the certified reading, and also prints the one at $q \le 13$. Its lines are `LN23_CELL`, `LN23_THM11_SUMMARY`, `LN23_THM12I_SUMMARY`, `LN23_STURM_SUMMARY` and so on; the last line is `HeckeLevelNCurveValidation: <c> checks, <f> failing, <t> s`.

Variable	Effect
`HECKE_LN23_QMAX=<q>`	the first rung of the Hecke primes $q \le Q$ every reading uses (default 13; an integer in $[2, 997]$); the certificate climbs along $13, 19, 31, 47, 71, 101, \ldots$ until it holds
`HECKE_LN23_Z1_NMAX=<n>`	the torsion gate over all primes $N \le n$, plus every composite $N \le 100$ (default 600; at most 20000; `2000`: 377 levels, about 1 s)
`HECKE_LN23_THM11_NMAX=<n>`	the grid of CE's Theorem 1.1 (primes $N \equiv 9 \bmod 16$; default 2000; at most 100000; `30000`: 397 cells, 72 min, 0.77 GiB per cell at `ENGINE_THREADS=4`)
`HECKE_LN23_THM12I_NMAX=<n>`	the grid of CE's Theorem 1.2(i) (primes $N \equiv 10, 19 \bmod 27$; default 2500; at most 100000; `10000`: 133 cells, about 3.5 min)
`HECKE_LEVEL_N_R3_SMALLP_CELLS="N:p,..."`	replaces all arms by readings at the listed cells: N prime, $p \in \{2, 3\}$, $p \mid \mathrm{num}((N-1)/12)$; any other cell is refused by its position
`HECKE_LN23_STURM=1`	only the Sturm separating-family arm (Hecke primes $q \le \lfloor (N+1)/6 \rfloor$); 76 cells, 228 checks, about 27 min, 0.45 GiB. Combined with the previous variable it runs on those cells. Any value other than `1` is an error
`HECKE_LN23_HEAVY=1`	adds the cell $(21929, 2)$ ($\dim W = 255$, Loewy length 128; about 34 s, 0.34 GiB at `ENGINE_THREADS=4`)

A value that is not a positive integer, or lies outside its range, is never ignored: the validation stops with an `[EXCEPTION]` line naming the variable and counts as `[FAIL]`. The same holds for every token "refused by its position" in this section. At the default $q \le 13$ the Theorem 1.1 grid to $3 \cdot 10^4$ reads 369 of 397 cells without the certificate; with it, 397 of 397 agree — which is why the certified reading is the one compared. Example:

```
HECKE_LN23_STURM=1 HECKE_LEVEL_N_R3_SMALLP_CELLS="41:2,73:2" ENGINE_THREADS=4 ./hacs
↪  HeckeLevelNCurveValidation
```

prints `EFFECTIVE HeckeLevelNCurveValidation arm=STURM cells=2 (HECKE_LN23_STURM=1; every other arm skipped)` and, in a fraction of a second, `HeckeLevelNCurveValidation: 6 checks, 0 failing`. Read the `cells=` count: it shows which cells were selected.

`HeckeLambdaTowerValidation` (Λ-adic tower). The validation reads the Hecke modules of the Λ-adic tower for $\ell = 5, 7, 11, 13$ ([U], `sec:u-tower`) on the cells (ℓ, a, r): a even with $a < \ell - 1$ is the weight class and r the tower level ($\Gamma_1(\ell^r)$). There are 23 default cells: $\ell \in \{5, 7\}$ with all even a and $r = 1, 2, 3$, and $(11, 2), (11, 4), (13, 0), (13, 4)$ for $r = 1, 2$ (`EFFECTIVE HeckeLambdaTowerValidation arm=DEFAULT cells=23`).

Variable	Effect
HECKE_LAMBDA_TOWER_CELLS="l:a:r[:K],..."	replaces the cell list (K, default 3, is the precision exponent, $1 \le K \le 3$; $5 \le \ell$, a even, $a < \ell - 1$, $1 \le r \le 3$). A malformed or out-of-range token is refused by its position, an empty list fails, and $\ell > 13$ is refused with tower_edge_not_built. Prints EFFECTIVE HeckeLambdaTowerValidation arm=CELLS cells=<k>
HECKE_LAMBDA_TOWER_R3=1	adds the two $r = 3$ cells $(11, 2, 3)$ and $(13, 0, 3)$, subject to the two caps below (any other value is an error)
HECKE_LAMBDA_TOWER_RSS_CAP_GIB=<g>	the memory cap of an $r = 3$ cell in GiB (default 4)
HECKE_LAMBDA_TOWER_OPS_CAP=<x>	the cap on the predicted number of Krylov operations (default 10^{12})

How the caps work. Before an $r = 3$ cell allocates anything, the validation predicts its memory and its Krylov work from the $r = 2$ cell of the same (ℓ, a) measured in the same run (with HECKE_LAMBDA_TOWER_CELLS set, list $(11, 2, 2)$ and $(13, 0, 2)$ too, or both $r = 3$ cells are refused as tower_rss_cap) and prints EFFECTIVE HeckeLambdaTowerValidation arm=R3 cell=<l:a:r> predicted_bytes=... cap_bytes=... predicted_krylov_ops=... ops_cap=.... If the prediction exceeds a cap, the cell is *refused*, not run: REFUSED tower_rss_cap <cell> or REFUSED tower_cost_cap <cell>, each marked "counted, not a failure". At the default caps both $r = 3$ cells are refused (predicted 33.5 GB / $1.5 \cdot 10^{14}$ operations and 86 GB / $6.4 \cdot 10^{14}$). A cell runs only when it is under *both* caps; raising them is the user's decision about the machine: read the predicted_bytes line first, and do not raise the memory cap above the free memory. The second example raises only the memory cap: it prints both predictions and still refuses both cells ($(11, 2, 3)$ as tower_cost_cap, $(13, 0, 3)$ as tower_rss_cap), which is the safe way to read them. Example:

```
HECKE_LAMBDA_TOWER_CELLS="5:0:2,7:2:3:3" ./hacs HeckeLambdaTowerValidation
HECKE_LAMBDA_TOWER_R3=1 HECKE_LAMBDA_TOWER_RSS_CAP_GIB=40 ./hacs
↪  HeckeLambdaTowerValidation   # predictions only
```

The first line reports arm=CELLS cells=2; the record per cell is HECKE_LAMBDA_TOWER ell= a= r= K= ... digest=0x... (schema hecke_lambda_tower/v1, not pinned) and the last line is HECKE_LAMBDA_TOWER checks=<c> failing=<f> wall_s=<t> peak_rss_mib=<m> verdict=OK (50 checks, about 10 s and 0.6 GiB for this example).

Saturation defect. HECKE_SAT_DEFECT_CELLS="n:ell:kmax,..." selects the cells of HeckeSaturationDefectValidation; HECKE_SAT_DEFECT_STRICT=1 additionally fails every not_stable_at_KMAX block, so that a too-small kmax is loud. A census driver sets both for each case (Section 8.7).

Chapter 10

Glossary and troubleshooting

10.1 Glossary

cell (n, ℓ)
: a weight n (the symmetric power Sym^n) together with a coefficient prime ℓ; the unit of every census and campaign. The *flagship cell* is $(10, 691)$.

golden reference / point / object
: the committed file `Golden/GoldenReference.inc`: for each *point* (a cell and operator) the hashed *objects* (matrices, polynomials, decompositions) that every build must reproduce bit for bit; `golden(...)` compares against it. A change to it is a *re-baseline* and is *owner-gated*: only the project owner performs one, explaining each changed line.

fingerprint
: a hash of the mathematical settings in force; a non-default setting moves it, so results of different meaning never compare equal.

handle
: a lazy pipeline object, `declared`, `computed` or `failed` (Section 7.2.4).

arm, `EFFECTIVE` line
: an *arm* is one environment-selected variant of a validation; the `EFFECTIVE` line says which arm actually ran and over how many cells.

tier `fast`, `standard` or `slow`: which of `--all` / `--all-full` runs a validation.

lean
: the memory-lean mode of a computation (smaller peak RSS, same values).

report levels (a)/(b)/(c)
: the three layers of a campaign job's report: (a) the flat cochain matrix $\Theta^{(q)}$, (b) the matrix $A^{(q)}(T_p)$ of the Hecke operator, (c) the certified decomposition (`level_a = auto|always|never` controls (a)).

shape group
: the campaign jobs with the same matrix shapes (same n, ℓ, degrees), batched together.

`BENCH` line
: the one machine-readable timing line a `--bench` run prints.

transcript, annotation
: a frozen `.expected` output of a script; an annotation is a `#~` line that comparisons ignore (Section 7.1).

S_0, $S_{\mathrm{sep}}(\ell)$, "anemic"
: atlas words: $S_{\mathrm{sep}}(\ell)$ is the finite set of primes $q \neq \ell$ whose T_q separate the mod-ℓ eigensystems of the atlas ([E]); S_0 the small base prime set, and "anemic" the eigenvalue vector $(a_q)_q$ taken over primes outside the level only (Section 6.6).

10.2 Troubleshooting

Everything is zero.
An odd n: $H^\bullet(\Gamma; \mathrm{Sym}^n) = 0$ for odd n on this model (`[C, rmk:negI]`). Use an even n.

`[FAIL]` for a name I did not mean to run.
The first argument was not a recognised token, so it and everything after it were taken as validation names – typically a console modifier given before the console form (`--set k=v --script f.oc`). Put the form first.

`golden(...)`: the mathematical settings are not at their defaults.
The frozen reference pins the defaults only. `reset_settings()`, or start a fresh session; `fingerprint()` must be `0x0000000000000000`.

Exit code 5.
A legal setting this build cannot execute: `field_polynomial = "conway"`, `coefficient_ring = "Z/<m>"`, or a verb that does not honour the coefficient ring in force (the atlas and the extension record are $\mathbb{F}_\ell$ objects; `coefficient_ring = "Z"` is legal and is honoured by `descent` only, every other verb refuses it by name). Not a bad value – the value is recorded, never silently replaced by the default.

`CliManualCoverageValidation` fails with "STALE".
A verb, type, setting, validation or command-line token changed after this manual was generated. Regenerate (Appendix E); a "no row" failure means a parser gained a flag: add a row to the token table in `Console/CliManualCoverageValidation.cpp` and regenerate.

`--bench micro ... --backend gpu`: not available, exit 2.
No usable CUDA device, or an op (`rref`, `kernel`) that has no device implementation.

The process was killed or the machine became unresponsive.
Memory. The large cells of the extension layer, the census hooks and the level-N hooks can need many GiB; several hooks refuse cells whose predicted peak exceeds a cap. Estimate the peak RSS first (the `--bench` line reports `rss_mb`; `rss()` in the console), use `--lean` or `lean = true` for sweeps, and run one heavy job at a time.

An env hook seems to have no effect.
Look for its `EFFECTIVE` line. No line: the variable was not read (wrong name, or the validation was not run). `cells=0`: the list was empty.

A run is slower than the figures in the documentation.
Timings depend on the build type (use `RelWithDebInfo` or `Bench`, not `Debug`), on `ENGINE_THREADS` and on the load of the machine; they are never part of a result.

`cmake` fails under WSL with a missing `/run/user/...`.
The snap wrapper needs `XDG_RUNTIME_DIR`; call `/snap/cmake/current/bin/cmake` directly.

A configuration change has no effect.
The build directory's `CMakeCache.txt` pins the first configuration; remove it and `CMakeFiles/`, or use a fresh build directory.

`cmake .` fails with "In-source builds are not supported".
Configure into a separate directory (`cmake -B build`) and remove the `CMakeCache.txt` and `CMakeFiles/` the attempt left in the repository root.

AddressSanitizer reports a leak of 50 960 bytes.
The WSL CUDA driver's one-time probe; run sanitizer builds with `ENGINE_CUDA_DISABLE=1`. ThreadSanitizer additionally needs `setarch x86_64 -R` on this kernel.

A validation reads the wrong files, or fails after a checkout was removed.

The binary reads tracked files through the source path baked in at configure time (Section 1.3).

Appendix A

Verb reference

All 178 overloads of the 165 verbs in 7 groups, in registration order, which is also overload-resolution order. `x: Kind = v` is an optional parameter with its default; `...` a variadic tail.

A.1 Group `core`

`print(value: any, ...)` [served]
prints its arguments on one line, separated by spaces (a String prints bare)

`show(value: any)` [served]
prints the full canonical form, ignoring print_max_entries

`describe(value: any)` [served]
prints type, state, settings snapshot, creation site and members (same as ?x)

`str(value: any)` [served]
the display form as a String

`repr(value: any)` [served]
the canonical form as a String (strings quoted)

`json(value: any)` [served]
JSON for Int / Bool / String / List / Record, with a proper escaper

`annotate(text: String)` [served]
prints a '#~ ' line, which every transcript comparison ignores

`len(value: any)` [served]
length of a List, String or Record

`keys(record: Record)` [served]
the keys of a Record, in order

`range(from: Int, to: Int)` [served]
the half-open range [from, to) as a List of Int

`range(from: Int, to: Int, step: Int)` [served]
the half-open range [from, to) with the given step

`abs(value: Int)` [served]
absolute value

`min(a: Int, b: Int)` [served]
the smaller of two Ints

`max(a: Int, b: Int)` [served]
the larger of two Ints

`type(value: any)` [served]
the type name as a String

`assert(condition: Bool)` [served]
fails the script (exit code 4) when the condition is false

`assert(condition: Bool, message: String)` [served]
fails the script (exit code 4) with the given message

`exit()` [not served]
ends the script with exit code 0

`exit(code: Int)` [not served]
ends the script with the given exit code

`vars()` [served]
every bound variable as a Record

`verbs()` [served]
every registered verb name

`types()` [served]
every registered type name

`settings()` [served]
every setting with its value, source (session / env) and family (engine / mathematical)

`reset_settings()` [served]
re-applies the settings snapshot taken when the session started

`script_args()` [served]
the script parameters, same as `args`

`compute(object: Object)` [served]
forces a lazily declared handle and returns it

`state(object: Object)` [served]
state, creation site, settings snapshot and resumable state of a handle

`resume(object: Object)` [served]
continues a computation that a budget cut off, with the same budget

`resume(object: Object, budget: any = nil)` [served]
continues a budget-cut computation with a new budget, recorded in the resumed handle's settings snapshot

`display(value: any, ...)` [served]
forces each declared handle, then shows each argument: in a served notebook as a rendered value (one value frame each, like a bare expression), elsewhere as the full canonical form (like show)

A.2 Group `arithmetic`

`Fp(p: Int, a: Int)` [served]
the element a of F_p, as a canonical residue

`poly(p: Int, coeffs: List)` [served]
the polynomial with the given coefficients LOW -> HIGH over F_p

`X(p: Int)` [served]
the indeterminate X over F_p

`coeffs(f: Poly)` [served]
the coefficients LOW -> HIGH (the machine order; the print form is HIGH -> LOW)

`degree(f: Poly)` [served]
the degree, -1 for the zero polynomial

`factor(f: Poly)` [served]
the certified factorisation of f (always passed through CertifyFactorization)

`gcd(f: Poly, g: Poly)` [served]
the monic gcd

`powmod(g: Poly, e: Int, f: Poly)` [served]
g^e mod f

`eval(f: Poly, x: Int)` [served]
f(x) in F_ell

`matrix(p: Int, rows: List)` [served]
a matrix over F_p from a list of equal-length rows

`identity(p: Int, n: Int)` [served]
the n x n identity over F_p

`zero(p: Int, rows: Int, cols: Int)` [served]
the zero matrix over F_p

`mul(a: any, b: any)` [served]
the product of two matrices or two polynomials

`add(a: any, b: any)` [served]
the sum of two matrices or two polynomials

`sub(a: any, b: any)` [served]
the difference of two matrices or two polynomials

`scale(A: Matrix, c: Int)` [served]
c * A

`rank(A: Matrix)` [served]
the rank of A

`nullity(A: Matrix)` [served]
cols(A) - rank(A)

`rref(A: Matrix)` [served]
the reduced row echelon form of A

`kernel(A: Matrix)` [served]
a basis of ker A, one column per basis vector

`image(A: Matrix)` [served]
a basis of the column space of A, one column per basis vector

`column_space(A: Matrix)` [served]
a basis of the column space of A (the name plan Sec. 3.5 uses)

`inverse(A: Matrix)` [served]
A^{-1}

`transpose(A: Matrix)` [served]
A^T

`pow(A: Matrix, e: Int)` [served]
A^e for a square A and e >= 0 (A^0 = I)

`solve(A: Matrix, B: Matrix)` [served]
the exact X with A*X = B (A of full column rank; typed error otherwise)

`hstack(parts: List)` [served]
the parts side by side

`vstack(parts: List)` [served]
the parts stacked

`submatrix(A: Matrix, r0: Int, r1: Int, c0: Int, c1: Int)` [served]
the half-open block A[r0:r1, c0:c1]

`columns(A: Matrix, idx: List)` [served]
the listed columns of A, in the order given

`vec(A: Matrix)` [served]
A as one column, ROW-MAJOR (SubspaceOps' convention)

`unvec(v: Matrix, rows: Int, cols: Int)` [served]
the inverse of vec

`sum_spaces(U: Matrix, V: Matrix)` [served]
a basis of the column span of [U | V]

`intersect(U: Matrix, V: Matrix)` [served]
a basis of the intersection of the two column spans

`contains(U: Matrix, v: Matrix)` [served]
whether v lies in the column span of U

`in_span(list: List, A: Matrix)` [served]
whether A lies in the F_ell-span of the listed matrices

`restrict(A: Matrix, basis: Matrix)` [served]
the matrix of A on the invariant subspace spanned by the basis columns

`algebra(ops: List)` [served]
the unital F_ell-algebra generated by pairwise-commuting operators: {dim, basis}

`commutant(ops: List)` [served]
{X : X A = A X for every A in ops}: {dim, basis}

`is_cyclic(ops: List, v: Matrix)` [served]
whether F_ell[ops] . v is the whole space

`cyclic_vector(ops: List, seed: Int)` [served]
a cyclic vector for F_ell[ops], from a MANDATORY seed

`random_vector(p: Int, n: Int, seed: Int)` [served]
a SplitMix64 vector; the seed is mandatory, so it is reproducible

`random_matrix(p: Int, rows: Int, cols: Int, seed: Int)` [served]
a SplitMix64 matrix; the seed is mandatory

`jordan_type(A: Matrix)` [served]
the Jordan partition of a NILPOTENT A, from the nullities of its powers

`charpoly(A: Matrix)` [served]
the characteristic polynomial (Berkowitz; division-free)

`minpoly(A: Matrix)` [served]
the minimal polynomial (Krylov)

`trace(A: Matrix)` [served]
the trace in F_p

`eval_at(f: Poly, A: Matrix)` [served]
f(A)

A.3 Group `pipeline`

`sl2z_model()` [served]
the 7-vertex SL(2,Z) orbifold model with its exact tessellation tables

`sl2z_model(name: String)` [served]
the named model descriptor (this build ships 'sl2z_7vertex')

`models()` [served]
the model names this build can build

`sphere(n: Int)` [served]
the simplicial n-sphere (n = 1, 2, 3)

`cylinder()` [served]
the simplicial cylinder

`moebius()` [served]
the simplicial Moebius band

`projective_plane()` [served]
the simplicial projective plane

`complex(simplices: List, name: String)` [served]
a complex from its simplex lists, grouped by dimension

`symmetric_power(model: Model, n: Int, ell: Int)` [served]
the local system Sym^n(F_ell^2) on the model

`cohomology(L: LocalSystem)` [served]
H^*_eq(K, R; L) – lazy: the work runs on the first computed access

`dickson_exceptional(n: Int, ell: Int)` [served]
whether (n, ell) is on the Dickson-invariant exceptional locus

`dickson_count(n: Int, ell: Int)` [served]
#{(a, b) >= 0 : a(ell+1) + b(ell^2-ell) = n}

`hecke(H: Cohomology, p: Int)` [served]
T_p on H^* (this IS U_ell when p == ell – the same code path)

`hecke(H: Cohomology, p: Int, r: Int = 1)` [served]
the primitive double-coset pipeline for N = p^r

`family(H: Cohomology)` [served]
the Hecke operator family on H^* (32-bit only)

`dual(L: LocalSystem)` [served]
the contragredient D_n of a model local system (E7; lazy)

`pairing(L: LocalSystem, D: ContragredientLocalSystem)` [served]
the Poincare-duality pairing between H_c(L) and H(D), D = dual(L) (E7; lazy)

`s1_scalar(scalar: Int, ell: Int)` [served]
the rank-1 local system on S^1 with scalar monodromy (a preset: no Hecke action)

`s1_unipotent(ell: Int)` [served]
the rank-2 unipotent local system on S^1 (a preset: no Hecke action)

`cylinder_scalar(scalar: Int, ell: Int)` [served]
the scalar-monodromy local system on the cylinder

`cylinder_unipotent(ell: Int)` [served]
the rank-2 unipotent local system on the cylinder

`moebius_scalar(scalar: Int, ell: Int)` [served]
the scalar-twist local system on the Moebius band

`moebius_unipotent(ell: Int)` [served]
the rank-2 unipotent local system on the Moebius band

`decompose(ops: any, q: Int)` [served]
the certified primary decomposition of H^q over a commuting Hecke family (one operator or a list of them); q is mandatory

`report(D: Decomposition, level_a: Bool = false)` [served]
the hecke_report/v2 record of the decomposition's degree

`report(D: Decomposition, level_a: Bool = false, path: String = "")` [not served]
the hecke_report/v2 record of the decomposition's degree, also written to a file

`crosschecks(T: HeckeOperator)` [served]
the classical alpha/beta/omega factorization checks (i)/(ii)/(iii) against the fused pipeline – expensive in p

A.4 Group `reproducibility`

`hash(x: any)` [served]
the per-kind FNV-1a-64 of the golden harness, as canonical 0x... text

`hash_int(x: any)` [served]
the same hash as an Int, when it fits one

`golden(point: String, object: String)` [served]
the frozen GoldenReference.inc entry {tier, rows, cols, value, hash} – READ-ONLY

`golden_has(point: String, object: String)` [served]
whether the reference has that (point, object)

`golden_size()` [served]
the number of entries in the reference

`fingerprint()` [served]
the mathematical-settings fingerprint (0x0... at the defaults)

`math_settings()` [served]
the mathematical settings in force, with their sources and the fingerprint (read-only in C1a; `set` on them is phase C2)

`save_state(x: any)` [served]
the R3 resumable-state object of a handle, as JSON text

`save_state(x: any, path: String)` [not served]

the same, written to a file (a state over 64 KiB goes to a sidecar referenced by relative path and FNV-1a-64)

A.5 Group harness

`validations()` [served]

every registered validation as [name, tier], in registration order

`validations(tier: String)` [served]

the registered validations of one tier ("fast" / "standard" / "slow")

`run_validation(name: String, inherit_settings: Bool = false)` [not served]

runs one registered validation in process and returns its code (0 = pass); the session's settings are lifted to its baseline for the run unless inherit_settings, and the validation's own output is captured unless `verbose`

`bench(mode: String, n: Int, ell: Int, p: Int = 0, r: Int = 1, family: List = List, repeat: Int = 1, lean: Bool = false, cross_checks: Bool = false, count_allocs: Bool = false, inherit_settings: Bool = false)` [not served]

one `--bench` run (cohom / hecke / family / decomp / ext) through bench::RunMode: a record {mode, params, dims, total_s, total_us, median_s, repeats, rss_mb, phases, extra, bench_line}; times are Strings

`micro(op: String, a: any = nil, b: any = nil, c: any = nil, d: any = nil, backend: String = "", repeat: Int = 1, inherit_settings: Bool = false)` [not served]

one `--bench micro` op (gemm / gemv / rref / kernel / transpose / suite) with up to four integer arguments; backend "" = the ENGINE_BACKEND default

`campaign_spec(path: String)` [not served]

parses a campaign spec file (relative to the running script)

`campaign_spec_text(text: String)` [served]

parses campaign spec text

`campaign_plan(spec: CampaignSpec)` [served]

the plan: one {signature, jobs} record per shape group, in execution order

`run_campaign(spec: CampaignSpec, output: String = "", inherit_settings: Bool = false)` [not served]

runs the campaign (writes one JSON per job, index.json and summary.tsv under `output`, default the spec's output_dir): {output_dir, ran, skipped, failed, jobs, summary_tsv, index_json, wall_s}

`golden_subset(cells: List, inherit_settings: Bool = false)` [served]

recomputes every golden object of the [n, ell] cells and compares both tiers: {mismatches, digest, records}

`timed(x: any)` [served]

forces a declared handle and times it: {value, wall_s, wall_us, peak_rss_mb, peak_rss_kb, allocs, alloc_bytes} (peak RSS is the PROCESS high-water mark)

`bench_line(record: Record)` [served]

prints the record's `BENCH ...` line as a `#~` annotation

`bench_line(record: Record, mode: String)` [served]

the same, naming the mode of a timed() record

`rss()` [served]

the process's resident set: {peak_rss_mb, rss_mb, peak_rss_kb, rss_kb, scope} – the high-water mark of the PROCESS, not of any one verb

A.6 Group `reference`

`help_markdown()` [served]

the console reference manual (docs/console/console.md) as Markdown, generated from this registry – [P]'s `help --markdown`

A.7 Group `extension`

`sturm_bound(k: Int, ell: Int)` [served]

the computed Sturm bound B'(k, ell) at the default cap 12*B(k)

`sturm_bound(k: Int, ell: Int, cap: Int = 0)` [served]

the same with the cap pinned

`u_ell_decision(n: Int, ell: Int)` [served]

whether U_ell fits the `u_ell` budget in force, by the frozen RSS model

`u_ell_decision(n: Int, ell: Int, budget_mb: Int)` [served]

the same against an explicit budget in MB

`bernoulli_mod(w: Int, ell: Int)` [served]

B_w in F_ell by the F_ell recursion (von Staudt-Clausen: defined exactly for 0 <= w < ell - 1, refused outside it)

`glover_factors(n: Int, ell: Int)` [served]

the Glover / Ash-Stevens composition factors Sym^{n_i} (x) det^{j_i} of Sym^n(F_ell^2) over F_ell[GL_2(F_ell)], with multiplicities, in the PIPELINE's convention: Sym^n of the STANDARD module (not its dual), the one rho_n acts on – the dual would shift every det_twist by -n

`canonical_field_polynomial(ell: Int, r: Int)` [served]

C_{ell,r}, the defining polynomial of the degree-r split field, as the `field_polynomial` setting in force selects it: `least` (the default) gives the CanonicalLess-least monic irreducible of [E] def:canon; `conway` is a legal value this build cannot execute (exit code 5)

`nilpotent_part(A: Matrix)` [served]

the nilpotent part n of A = s + n (Jordan-Chevalley over F_ell: s = A^(ell^N))

`nilpotent_part(ops: List)` [served]

the nilpotent parts of a COMMUTING family (the generators of its radical ideal)

`radical(alg: any)` [served]

rad A = the nilpotent elements of A = F_ell[ops] (ker Frob^e), {dim, basis}; `alg` is a list of commuting operators, the Record of algebra(ops) or a HeckeAlgebra

`tangent_space(alg: any)` [served]

m/m^2 = rad/rad^2: {dim (over F_ell), local, f, embdim, basis}

`socle(alg: any)` [served]

soc A = Ann_A(rad A): {dim, local, f, gorenstein, basis}

`nilpotency_index(alg: any)` [served]

the least nu with rad^nu = 0 (1 iff A is reduced)

`is_local(alg: any)` [served]

whether A has exactly one maximal ideal (Berlekamp: dim{x : x^ell - x in rad} - dim rad == 1) – decided, never searched

`algebra_invariants(alg: any)` [served]

every invariant above in one Record

`loewy(alg: any, V: Matrix = nil)` [served]

the Loewy (radical) series rad^j V of an A-stable subspace V (columns; the whole space when omitted): {dims, layers, f, layers_over_f, length}

`socle_series(alg: any, V: Matrix = nil)` [served]

the socle series soc^j V, same shape as loewy

`check_identities(H: Cohomology, primes: List = nil, items: List = nil)` [served]

every item HeckeIdentityCertificate defines, at H's cell, as {items: [{item, name, status, detail}], code, k_max, ...}; primes = [p, p'] (default [2, 3]); items 6 and 7 are assembled only when listed in `items`, and an item whose inputs were not assembled reports `unavailable`

`L_map(H_lower: Cohomology, H: Cohomology)` [served]

the Dickson map L_* : H^*(M_m) -> H^*(M_n), m = n - ell - 1: {L, certificate, induced, ranks}

`ext_record(n: Int, ell: Int, cone: Bool = false, sign_split: Bool = false, presentation: Bool = false, sturm: Bool = true, commutant: Bool = true)` [served]

the cell's hecke_ext/v5 record (Stages 1-5) under the mathematical settings in force (lazy); cone / sign_split add the E4b-E5 / E3 objects, sturm / commutant select the Stage-4 routes

`local_algebra(m: MaximalIdeal)` [served]

the local Hecke algebra T_m of a certified-local maximal ideal

`module_structure(m: MaximalIdeal, group: String = nil)` [served]

the LoewyProfile of m in `group` (default: m's own group)

`presentation(m: MaximalIdeal)` [served]

generators and a minimal relation set of H_m over T_m

`atlas(ell: Int)` [served]

Sigma(ell) and S_sep(ell), in process at ell in {5, 7, 11, 13} (lazy)

`GF(ell: Int, r: Int)` [served]

the canonical field F_ell[Y]/(C_{ell,r}) of degree r

`canonical_field(ell: Int, r: Int)` [served]

the same as GF(ell, r) ([X] Sec. 7's name)

`gf_add(x: GFElt, y: GFElt)` [served]

x + y in GF

`gf_sub(x: GFElt, y: GFElt)` [served]

x - y in GF

`gf_mul(x: GFElt, y: GFElt)` [served]

x * y in GF

`gf_inv(x: GFElt)` [served]

x^-1 in GF (x != 0)

`gf_pow(x: GFElt, e: Int)` [served]

x^e in GF

`gf_frobenius(x: GFElt)` [served]

x^ell

`gf_eval(f: Poly, x: GFElt)` [served]

f(x) for f over F_ell and x in GF

`settings_snapshot()` [served]

the mathematical settings in force, as a Settings value (math_settings() is the C1a Record form)

`iota(H: Cohomology)` [served]

the production involution iota^* on H and its +/- split (lazy)

`boundary(H: Cohomology)` [served]

H^*_bd on the boundary circle with r and T_p^bd (lazy)

`cone(H: Cohomology)` [served]

H^*_c = H^*(Cone(r)) with its LES and A_c(T_p) (lazy)

`dickson_les(n: Int, ell: Int)` [served]

the long exact sequence of 0 -> Sym^n (x) det -> Sym^{n+ell+1} -> Q_n -> 0 over F_ell, ell >= 5, as a DicksonLES handle (lazy)

`hecke_ring_algebra(n: Int, ell: Int, kmax: Int = 3)` [served]

the Z/ell^K Hecke-algebra route ([C] cor:stablecert; HeckeRingHeckeAlgebra), ell >= 5, n even, kmax = 3 by default, ell^max(kmax, K_cls) < 2^64, U_ell within the `u_ell` budget: {schema = hecke_ring_algebra/v1, n, ell, kmax, primes, e_max, K_cls, K_run, Lv_run, level_mode, blocks, lines} (K_run = the levels read, Lv_run = the levels at which the module route ran, level_mode = reuse:<policy> | per_K:<reason>; [C] obl:precisionreuse); per block {system, D, free_rank, torsion, rank_T, A1_hilbert, AK_hilbert, K_stab, eps, verdict, stable_cert, stable, cf9, hidden}; a ring exit 5, ell < 5 exit 3

`theta(H: Cohomology, H_up: Cohomology)` [served]

Dickson theta = L_* : H^q(M_n (x) det) -> H^q(M_{n+ell+1}), source-indexed (the same map as L_map(H, H_up)): {n, n_up, ell, induced, ranks, ker, image, twist_check, u_ell_zero}

`lfunction(n: Int, ell: Int, a: Int, M: Int = 2, R: Int = 4, sign: String = "minus", sep: List = nil)` [served]

the record lfunction/v1 of branch a: L_a(T) in Lambda/(ell^R, omega_{M-1}(T)) of the ell-stabilised MTT measure ([E] def:lvmtt) of the ordinary eigensymbol picked by sep (List of [p, c]; nil = T_2+3T_3+7T_5; the Eisenstein root if it occurs, else the first root mod ell), sign "minus" (r^-) or "plus" (r^+); ell >= 5 prime, n even, M >= 2, ell^M <= 4*10^6: {schema, n, k, ell, R, M, G, u, sign, geometric, form, normalisation, a, vanishes, mu, lambda, readable, lambda_mod_ell, coefficients, precision, values_modulus, critical_values, hash, line} ([E] lem:lvlfunprec: prec(0) = R, prec(t) = min(R, M-1-floor(log_ell t)))

`kummer_unit(ell: Int, i: Int)` [served]

the cyclotomic unit eta_i of layer 1 ([E] sec:lvnf) and its Kummer l-th-power test, ell prime, 5 <= ell <= 65521, i even, 2 <= i <= ell - 3: {schema, ell, i, exponents, coefficients, max_digits, v_pi, unramified, lambda_split, not_lth_power, eigen_ok, unit_ok, ell_divides_Bi, oracle_agrees, split_primes}

`hida_tower(n: Int, ell: Int, m: Int)` [served]

the Hida weight-tower cell at class n ([C] def:hidatower), ell >= 5 prime, n even <= 200, m <= 3: both readings of the ordinary part at levels e = 1 .. m, {schema, n, ell, m, e_max, K, h, exps, levels: [{e, ord_rank, image_free, charpoly, free_ord_rank, free_charpoly, readings_agree}], lines}

`level_n(N: Int, n: Int, ell: Int, primes: List = nil)` [served]

H^1(Gamma_0(N); Sym^n F_ell) by Shapiro's lemma ([C] sec:levelN; the sparse cokernel of the induced module) and the Heilbronn operators T_p (p not dividing N) / U_p (p | N) on it, primes = List of at most 6 primes <= 97, none equal to ell (nil = 2, 3, 5 without ell); ell >= 5 prime, n even, N <= 100, index(N) * (n + 1) <= 1000: {schema, N, n, ell, index, D, dim_S, dim_ST, h0, h1, cusps, dim_cusp_forms, ops: [{label, p, bad, charpoly}]}

`nebentypus(N: Int, m: Int, n: Int, ell: Int, primes: List = nil)` [served]

H^1(Gamma_0(N); Sym^n F_ell (x) chi) for the Dirichlet character of Conrey label N.m reduced into F_ell ([C] sec:nebentypus), and the Heilbronn operators on it; ell prime not dividing N with order(chi) | ell - 1, n >= 0, N <= 100, index(N) * (n + 1) <= 1000; primes = List of at most 4 primes <= 13 (nil = 2, 3, 5 without ell): {schema, label, N, m, order, parity_ok, n, ell, D, h0, h1, ops: [{label, p, bad, charpoly}]} (h0 = h1 = 0 and no ops when chi(-1) != (-1)^n)

`forms_algebra(k: Int, ell: Int, primes: List = nil, terms: Int = 8)` [served]

the forms side mod ell: S_k over F_ell with its Miller q-expansion basis and T_p on it ([E] sec:forms; the characteristic polynomials), ell >= 5 prime, k even, 12 <= k <= 400, primes (at most 6, <= 97, none equal to ell; nil = 2, 3, 5 without ell), terms <= 64: {schema, k, ell, d, k_prime, precision, basis, ops: [{p, charpoly}]}

`hida_lambda(ell: Int, k: Int, W: Int, primes: List = nil)` [served]

the forms side of the Lambda-adic Hecke algebra at one weight ([U] sec:u-lambda): the ordinary parts e M_k and e S_k over Z/ell^W and the characteristic polynomials of T_p and U = T_ell on e S_k; ell >= 5 prime <= 101, k even with 4 <= k <= 200 and k - 1 >= W, W <= 6, ell^W < 2^62; primes as for forms_algebra (nil = 2, 3, 5 without ell): {schema, ell, k, W, dim_M, dim_S, rank_M, rank_S, comm_ok, stable_ok, eisenstein, ops: [{name, charpoly}]}

`descent(N: Int, n: Int, primes: List = nil)` [served]

the integral descent record of H^q(Gamma_0(N); Sym^n Z^2) over Z ([C] thm:descentcert, sec:descentpair; slot-24 D6), a lazy DescentRecord: groups to q_max = 3, bad primes, K per prime, char_Z(T_p), the torsion Hecke modules, the boundary pair; only under coefficient_ring = "Z"; primes = List of 1 .. 6 primes <= 97 not dividing N, sum <= 100 (nil = the first three of 2, 3, 5, 7, 11, 13 not dividing N); N <= 300, n <= 60, index(N) * (n + 1) <= 300 (at the bound about 40 s and 1.4 GiB)

`hecke_structure(N: Int, n: Int, algebra: String = "full")` [served]

the record integral_hecke_structure/v1 of the cell (N, n, algebra) ([C] sec:recordschema item 8; slot-25 HZ-REC): status, record_fnv1a64, cert_sha256, the status signature and the hashed body (G0 .. G8) as a Record; only under coefficient_ring = "Z"; algebra = "full" | "anemic" (N > 1); N <= 300, n even <= 60, index(N) * (n + 1) <= 300 (the descent verb's bound; beyond it a terminal REPL asks first, elsewhere it refuses unless confirm = "yes")

Appendix B

Type reference

Computed members force the handle's computation; *structural* members are available while it is still `declared`. Members with arguments are methods: `H.H(1)`, `T.A(1)`.

`Fp`

an element of F_p (plan Sec. 3.4)

Member	Kind	Description
`.p`	structural	the modulus
`.value`	structural	the canonical residue in [0, p)

`Poly`

a polynomial over F_ell; prints HIGH -> LOW degree, while coeffs / hash use the machine order LOW -> HIGH (plan Sec. 2 C5)

Member	Kind	Description
`.ell`	structural	the modulus
`.deg`	structural	the degree (-1 for the zero polynomial)
`.monic`	structural	whether the leading coefficient is 1

`Matrix`

a dense matrix over F_p (FlatMatrixT<uint32_t>)

Member	Kind	Description
`.rows`	structural	the row count
`.cols`	structural	the column count
`.p`	structural	the modulus
`.entry(row, col)`	structural	the (row, col) entry as a canonical residue

`Model`

the SL(2,Z) orbifold model: its triangulation and the exact tessellation tables

Member	Kind	Description
`.name`	structural	the descriptor name
`.dims`	structural	the simplex counts by dimension
`.vertices`	structural	the vertex count

`Complex`

a simplicial complex (the presets of SimplicialCohomology.h and complex(...))

Member	Kind	Description
`.name`	structural	the complex's name
`.dim`	structural	the top dimension
`.euler`	structural	the Euler characteristic
`.dims`	structural	the simplex counts by dimension
`.simplices(q)`	structural	the q-simplices, each as a List of vertex indices

`LocalSystem`

Sym^n(F_ell^2) on a model, with its rho_n(g_r) relations (lazy)

Member	Kind	Description
`.n`	structural	the symmetric power
`.p`	structural	the coefficient prime ell
`.d_V`	structural	dim Sym^n = n + 1
`.coefficient_ring`	structural	the ring's canonical text: F_ell, or Z/ell^<k> ([E] sec:ring (a))
`.relations`	computed	the number of local-system relations

`Cohomology`

H^q_eq(K, R; L) with its C / Z / B / H bases and projectors (lazy); over Z/ell^k (coefficient_ring, ell >= 5) its lengths, invariant factors and K-mode

Member	Kind	Description
`.computed`	structural	whether compute_cohomology() has run
`.dims`	computed	dim H^q for every degree q
`.ell`	computed	the coefficient prime
`.d_V`	computed	the representation dimension
`.q_max`	computed	the top degree of THIS complex (derived)
`.truncated`	computed	false on a finite model
`.q_max_source`	computed	'derived' unless truncated (E9)
`.degrees`	computed	[0 .. q_max], derived
`.coefficient_ring`	structural	the ring's canonical text: F_ell, or Z/ell^<k> ([E] sec:ring (a))
`.lengths`	computed	the composition length of H^q over Z/ell^k, q = 0, 1, 2 (lengths_H^q)
`.ladic_type_certified`	computed	whether the K-mode certifies the Z_(ell)-type at K = k ([E] sec:ring (d))
`.precision`	computed	on a digit handle (ell^k >= 2^64): the certified precision bound – {precision_requested, precision_working, e_max, word_level, precision_certified, digit_width, digits_L}

Member	Kind	Description
.bockstein_rank	computed	bockstein_rank^0: the rank of beta^0 on H^0(F_ell), from the Z/ell^2 and F_ell objects ([E] sec:ring (d), (g))
.higher_torsion_count	computed	dim H^0(F_ell) - bockstein_rank^0 ([E] sec:ring (d))
.C(q)	computed	a basis of C^q_eq, one N_q x d_V cochain per element
.Z(q)	computed	a basis of Z^q_eq
.B(q)	computed	a basis of B^q_eq
.H(q)	computed	a basis of H^q_eq
.P(q)	computed	the projector C^q_flat -> H^q coordinates (legacy shape conventions)
.dim_C(q)	computed	dim C^q_eq (available even in the memory-lean mode)
.invariant_factors(q)	computed	H^q = (+)_i Z/ell^{e_i}: the exponents e_i, descending (invariant_factors_H^q)
.r_K(q)	computed	r_K(d^q) for q in {0, 1}: the K-mode's rank of the coboundary
.ladic_type(q)	computed	{free_rank, torsion} of H^q(C)_(ell); defined only at a certified level (else the object's named std::logic_error)
.red(q)	computed	red_H^q: H^q(Z/ell^k) -> H^q(F_ell), the (c5) matrix from the cyclic summands to the F_ell object's D5b basis, over F_ell

HeckeOperator

T_p (or U_ell, the same path) on H^*: the coset tables, the certified tau datum and the assembled Theta / A^{(q)}

Member	Kind	Description
.p	structural	the Hecke prime
.r	structural	the exponent of N = p^r
.cosets	computed	the number of cosets m
.certificates	computed	the in-process certificate bundle compute() already ran (O12(a) and the counters)
.json	computed	the operator's own report JSON (levels (a) + (b))
.dims	computed	dim H^q of the underlying complex
.coefficient_ring	structural	the ring's canonical text: F_ell, or Z/ell^<k> ([E] sec:ring (a))
.charpoly_free	computed	over Z/ell^k: {precision, coeffs} – the charpoly of A^{(1)}(T_p) on the free quotient, over Z/ell^precision, coefficients low -> high; defined only at a certified level
.A(q)	computed	A^{(q)}(T_p), the cohomology-level matrix in degree q (over Z/ell^k: the (c5) matrix on the cyclic summands, a List of rows, entry (i, j) in [0, ell^{e_i}))
.Theta(q)	computed	Theta^{(q)}_flat on the flat cochains C^q(K; V)
.Theta_eq(q)	computed	Theta^{(q)} restricted to C^q_eq in the stored C_eq basis
.apply(q, v)	computed	Theta^{(q)} applied to a flat cochain (the fused route)
.apply_route2(q, v)	computed	the INDEPENDENT second route of [C, eq:heckechain] on the same cochain (matrix-free; it does not read the fused tables)

Family

the Hecke operator family on one cohomology: T_p, T_{p^r} (recursion and the independent direct route), T_n and the diamond T(p,p)

Member	Kind	Description
`.T(p)`	computed	T_p per degree (this IS U_ell at p = ell)
`.T_pow(p, r)`	computed	the FULL T_{p^r} by the algebra recursion of [C, thm:relations]
`.T_pow_direct(p, r)`	computed	the same operator by the INDEPENDENT double-coset route – what T_pow is cross-checked against
`.T_pow_primitive(p, r)`	computed	the PRIMITIVE double-coset operator T'_{p^r} (equal to T_p at r = 1; it differs from the standard T_{p^r} for r >= 2)
`.T_n(n)`	computed	T_n for a composite index n
`.T_pp(p)`	computed	the diamond T(p,p) = p^n I ([C, prop:diamond])
`.theta(p, q)`	computed	Theta^{(q)}_flat of the family's r = 1 pipeline for p

`ContragredientLocalSystem`

D_n = Sym^n(F_ell^2)^vee on the 7-vertex model (rho_n^vee(h) = rho_n(adj h)^T), its cohomology, Phi_n : M_n -> D_n and the Hecke action A(D_n)(T_p) (E7; lazy)

Member	Kind	Description
`.n`	structural	the symmetric power
`.p`	structural	the coefficient prime ell
`.d_V`	structural	dim D_n = n + 1
`.coefficient_ring`	structural	F_ell (E7 is a field object)
`.phi_is_isomorphism`	structural	whether Phi_n (x) F_ell is an isomorphism, i.e. n < ell ([C] lem:symdual)
`.dims`	computed	dim H^q(Gamma; D_n) for every degree q (golden dim_H_Dn^q)
`.dims_M`	computed	dim H^q(Gamma; M_n), the standard module, for comparison
`.Phi`	computed	Phi_n : M_n -> D_n, (n+1) x (n+1) over F_ell
`.induced_Phi(q)`	computed	P^q = H^q(Phi_n) : H^q(M_n) -> H^q(D_n)
`.A(p, q)`	computed	A(D_n)(T_p)^q, the Hecke action on H^q(D_n) (U_ell at p = ell); charpoly(D.A(p, q)) is the golden charpoly_A_Dn(T_p)^q
`.A_M(p, q)`	computed	A(M_n)(T_p)^q on the standard module, built from the SAME coset / tau data
`.intertwines(p)`	computed	{q0, q1, q2}: whether A(D_n)(T_p) P^q == P^q A(M_n)(T_p) exactly (UNTWISTED, factor 1; [C] prop:pairing)

`Pairing`

the cone-level pairing H^t_c(M_n) x H^{2-t}(D_n) -> F_ell and the circle pairing: Grams, perfectness, the PD adjointness gate of [C] cl:pd (E7; lazy)

Member	Kind	Description
`.coefficient_ring`	structural	F_ell (E7 is a field object)
`.certificate`	computed	the pairing certificate re-derived: {failing, failing_items}
`.perfect(t)`	computed	whether the cone pairing in H_c degree t is perfect (golden pairing_perfect^t)
`.gram(t)`	computed	G^t: dim H^t_c x dim H^{2-t}(D_n) (golden pairing_matrix^t)
`.boundary_perfect(s)`	computed	whether the circle pairing in degree s is perfect

Member	Kind	Description
`.boundary_gram(s)`	computed	the circle Gram in degree s
`.pd_gate(primes)`	computed	{t0, t1, t2}: A_c^{(t)T} G^t == G^t A(D_n)^{(2-t)} for EVERY p in `primes` (factor 1; t = 0 vacuous and true); at primes [2, 3, 5] it is the golden pd_adjoint^t. A TEST of [C] cl:pd, which is open

`Block`

one primary block of a decomposition: its dimension, residue degree, the per-prime minimal / characteristic polynomials and generalized-Jordan partition, and the E0b locality certificate

Member	Kind	Description
`.dim`	computed	dim_F of the block
`.residue_degree`	computed	f = lcm_j deg(pi_j)
`.certified`	computed	whether f is CERTIFIED (a witness proves it), not merely a lower bound
`.local`	computed	the E0b locality certificate: this block is ONE maximal ideal
`.phase`	computed	which phase reached the verdict (linear / berlekamp / residue_search)
`.w_hat_exponent`	computed	N: the Teichmueller exponent of the witness
`.witness_poly`	computed	pi_w, the certifying witness's minimal polynomial
`.basis`	computed	the d x dim matrix whose columns span the block
`.primes`	computed	the family's primes, ascending
`.witness`	computed	the linear residue witness as [[p, c], ...] (empty in strict mode, where the witness is a matrix)
`.min_poly(p)`	computed	the minimal polynomial of T_p on this block
`.char_poly(p)`	computed	the characteristic polynomial of T_p on this block
`.character(p)`	computed	P_p, the character polynomial of the locality certificate
`.jordan(p)`	computed	the generalized-Jordan partition of T_p on this block, parts descending

`Decomposition`

the certified primary decomposition of H^q as a module over the commutative F_ell-algebra generated by a commuting family of Hecke matrices

Member	Kind	Description
`.q`	structural	the cohomological degree
`.ell`	structural	the coefficient prime
`.n`	structural	n_sym of the coefficient system
`.primes`	structural	the family's primes, in the order given
`.blocks`	computed	the primary blocks, in canonical order
`.num_blocks`	computed	how many primary blocks there are
`.U`	computed	the change of basis: the block bases, in block order
`.witness_hash`	computed	the golden `decomp_*_witness` hash (the residue-witness data), as 0x... text
`.locality`	computed	the E0b locality loop's state: mode, budget, splits, completeness
`.json`	computed	the decomposition's own JSON record
`.certify()`	computed	the UNCONDITIONAL certificate: 0 iff every claim re-derives (never trusting the randomized factor search)

Member	Kind	Description
`.conjugated(p)`	computed	U^{-1} A^{(q)}(T_p) U, verified exactly block-diagonal

Report

the hecke_report/v2 record of one degree: level (b) A^{(q)}(T_p), level (c) the decomposition, and optionally level (a) Theta^{(q)}_flat

Member	Kind	Description
`.schema`	structural	the frozen schema name
`.level_a`	structural	whether level (a) was included
`.path`	structural	the file it was written to, or ""
`.degree`	computed	the degree q this record is about
`.q_max`	computed	the top degree of THIS complex
`.truncated`	computed	false on a finite model
`.q_max_source`	computed	"derived" unless a setting pinned q_max
`.degrees`	computed	[0 .. q_max]
`.json`	computed	the report text
`.write(path)`	computed	write the report to a file (an empty path writes to stdout)
`.has_key(key)`	computed	whether the record carries a given JSON key – the schema assertion a transcript can make without freezing the whole text

CampaignSpec

a parsed campaign spec (docs/matrix-engine/campaign.md) – campaign_spec(path), campaign_spec_text(text)

Member	Kind	Description
`.name`	structural	the spec's `name`
`.output_dir`	structural	where run_campaign writes by default
`.jobs`	structural	the number of jobs the plan has

Sturm

the computed Sturm bound B'(k, ell) of [E] def:bprime: the least prime bound at which the F_ell-ALGEBRA generated by T_q, q <= B, reaches dim M_k

Member	Kind	Description
`.k`	structural	the weight k = n + 2
`.ell`	structural	the coefficient prime
`.outcome`	computed	found / none_at_cap / in_progress
`.found`	computed	whether B' exists at or below the cap
`.B_prime`	computed	B', meaningful iff `found`
`.cap`	computed	the cap in force (12*B(k) by default)
`.dim_M_k`	computed	dim M_k(F_ell)
`.closure_dim`	computed	the dimension the algebra closure reached
`.primes_done`	computed	the primes processed, ascending
`.certified`	computed	SturmBoundCertificate::verify() == 0, re-derived from the reported data

Member	Kind	Description
`.verify`	computed	the certificate's code (0 = every clause re-derives)
`.json`	computed	the certificate's JSON, `resumable` object included

`Atlas`

Sigma(ell): the anemic level-1 eigensystems over weights <= ell + 1, their theta-twists, and the separating set S_sep(ell) (HeckeAtlas, in process at ell in {5, 7, 11, 13}); lazy

Member	Kind	Description
`.ell`	structural	the prime
`.n_max`	computed	the largest even weight cell n <= ell - 1
`.primes`	computed	the anemic pool T
`.num_systems`	computed	|Sigma(ell)|
`.chi_count`	computed	distinct chi_{a,b} as T-systems
`.chi_in_sigma`	computed	of which in Sigma
`.separating_primes`	computed	the greedy S_sep(ell)
`.separates`	computed	S_sep separates Sigma u chi
`.s0_separates`	computed	{2,3,5,7} \ {ell} already separates
`.cells`	computed	the weight cells {n, dims, ideals, local_ideals}
`.json`	computed	the hecke_atlas/v1 store text
`.system(i)`	computed	the i-th system {id, r, type, base, twist, chi, eigenvalues, collected, in_H1}
`.lookup(eigenvalues)`	computed	the id of the system with these eigenvalues on `primes` (Ints at r = 1, coefficient lists at r > 1; any Frobenius conjugate), or -1

`GF`

the project-canonical field F_ell[Y]/(C_{ell,r}) ([E] def:canon; HeckeCanonicalField), C chosen by the `field_polynomial` setting

Member	Kind	Description
`.ell`	structural	the prime
`.degree`	structural	r = [F : F_ell]
`.polynomial`	structural	C_{ell,r}
`.canonical`	structural	C is the CanonicalLess-least monic irreducible
`.zero`	structural	0
`.one`	structural	1
`.gen`	structural	Y, the class of the variable
`.elt(coeffs)`	structural	the element sum c_i Y^i (coefficients low -> high, reduced mod C)
`.element_at(i)`	structural	the i-th element of the field's fixed enumeration

`GFElt`

an element of a GF field (coefficient vector in the basis 1, Y, ..., Y^{r-1})

Member	Kind	Description
`.coeffs`	structural	the coefficients, low -> high, width r
`.is_zero`	structural	x == 0

Member	Kind	Description
.field	structural	the GF handle
.poly	structural	the representative polynomial in Y

Settings

a snapshot of the MATHEMATICAL settings (hecke::MathSettings, [X] Sec. 1.5): every registered key with its value and source, and the fingerprint

Member	Kind	Description
.fingerprint	structural	FNV-1a-64 of the non-default entries (0x0 at the defaults)
.is_default	structural	every key at its default
.keys	structural	the registered keys, in registry order
.values	structural	{key: value text}
.text	structural	the canonical text
.json	structural	the settings JSON (values and sources)
.get(key)	structural	the value text of one key

ExtRecord

one cell's hecke_ext/v5 record (HeckeExtDriver, Stages 1-5): its maximal ideals, groups, u_ell verdict and settings; lazy

Member	Kind	Description
.n	structural	the symmetric power n
.ell	structural	the coefficient prime
.schema	computed	the record schema
.weight	computed	k = n + 2
.coefficient_ring	computed	the record's ring (F_ell)
.dims	computed	dim H^q of the bulk, every degree
.degrees_run	computed	the bulk degrees Stages 2-5 ran on
.operators	computed	the primes whose operators Stage 1 computed
.anemic_pool	computed	the anemic pool {q <= 31} \ {ell}
.s_sep	computed	S_sep(ell) as Stage 2(3) ran it (empty if not computed)
.dim_M_k	computed	dim M_k
.dim_S_k	computed	dim S_k
.dim_dickson	computed	the Dickson count
.H1_shriek	computed	dim H^1_! (cone = true), else nil
.les_exact	computed	the cone's LES exact (cone = true), else nil
.groups	computed	the record's groups, {name, available, dim}
.u_ell	computed	the cell's U_ell verdict and its resumable state
.ideals	computed	the maximal ideals, as MaximalIdeal handles
.num_ideals	computed	the number of maximal ideals
.has_sign_records	computed	whether sign_split = true built H^+ / H^-
.ideals_plus	computed	the ideals of the H^+ record (sign_split = true)
.ideals_minus	computed	the ideals of the H^- record (sign_split = true)
.settings	computed	the MathSettings the record was computed under
.verify	computed	HeckeExtRecord::verify() (0 = consistent)
.json	computed	the hecke_ext/v5 record text (carries the build's commit)
.digest	computed	FNV-1a-64 of the record text with commit redacted (NOT the golden ext_json_* hash)
.ideal(i)	computed	the i-th maximal ideal

MaximalIdeal

one maximal ideal m of a cell's Hecke algebra (an ideal of the ExtRecord)

Member	Kind	Description
.id	computed	the ideal's id in its record
.sign	computed	all / plus / minus
.dim	computed	dim_F_ell of the block H_m
.group	computed	the ambient group of the block
.local	computed	certified local
.residue_degree	computed	f = [T/m : F_ell]
.residue_degree_certified	computed	whether f is certified
.witness_poly	computed	pi_w, the witness's minimal polynomial
.w_hat_exponent	computed	N of the Teichmueller lift (local only)
.final	computed	true / false / heuristic
.final_reason	computed	why not final (" when final)
.certificate	computed	cyclic / commutant / sturm(B') / dickson / boundary / unavailable
.certificate_routes	computed	the routes' outcomes {a, b, c, d0, d1} (d0 / d1: the finality certificates, hecke_ext/v4)
.generators_certified	computed	the certified generating primes
.boundary_reps	computed	R(ell), the class representatives (d1) tested (empty unless it reached them)
.operator_primes	computed	the primes of the restricted operators
.tangent_active	computed	the primes whose n_p is active in m/m^2
.embdim	computed	dim_K m/m^2 (local only)
.nilpotency	computed	the nilpotency index nu (local only)
.gorenstein	computed	dim soc = f (local only)
.label	computed	the atlas label {type, system, theta_twist, ...} or nil
.predicted	computed	the predictions attached to the ideal
.unpredicted	computed	what was found and not predicted
.modules	computed	the LoewyProfile of every group the ideal lives in
.algebra	computed	the local Hecke algebra T_m (HeckeAlgebra)
.t(p)	computed	the restricted operator t_p = T_p|_{H_m}
.character(p)	computed	P_p, the eigenvalue of T_p as a polynomial in the witness (local only)

HeckeAlgebra

the local Hecke algebra T_m = A_S of a certified-local ideal (HeckeLocalAlgebra; [X] Sec. 7's LocalAlgebra)

Member	Kind	Description
.ell	computed	the prime
.D	computed	dim_F_ell of the block
.f	computed	the residue degree
.dim	computed	dim_F_ell A_S
.dim_m	computed	dim_F_ell m_S
.nilpotency	computed	nu: m^nu = 0 != m^{nu-1}
.hilbert	computed	dim(m^i/m^{i+1})/f
.socle_dim	computed	dim_F_ell soc A_S
.gorenstein	computed	dim soc A_S == f
.embdim	computed	dim_K m/m^2
.primes	computed	S, ascending
.active_set	computed	the primes whose n_p-bar is nonzero

Member	Kind	Description
.minimal_active	computed	a K-spanning subset, greedy in ascending p
.min_generators	computed	dim(V/m V)/f
.cyclic	computed	certificate (a): V cyclic over A_S
.verify	computed	certificate (a)'s verify() (0 = re-derives)
.basis	computed	an F_ell-basis of A_S (canonical RREF)
.radical	computed	an F_ell-basis of m_S
.w_hat	computed	the Teichmueller lift of the witness
.invariants	computed	the SAME invariants by the certificate-free route of algebra_invariants(t_p) – a cross-check
.json	computed	the LocalAlgebra JSON
.t(p)	computed	t_p = T_p restricted to the block
.n(p)	computed	n_p = t_p - P_p(w_hat), the nilpotent generator

LoewyProfile

the Stage-5 module structure of a block at m (HeckeModuleStructure; [X] Sec. 7's ModuleStructure): Loewy / socle series, Jordan types, structure maps

Member	Kind	Description
.group	structural	the ambient group
.D	computed	dim_F_ell V
.f	computed	the residue degree
.loewy	computed	Loewy layer dimensions in units of f
.socle	computed	socle layer dimensions in units of f
.m_dims	computed	dim_F_ell m^j V, j = 0 .. nu_V
.socle_dims	computed	dim_F_ell soc^j V
.primes	computed	the generators' primes
.cyclic	computed	V cyclic over T_m
.free	computed	V free of rank one
.min_generators	computed	dim(V/mV)/f
.semisimple	computed	mV = 0 (no extension at m)
.verify	computed	ModuleStructure::verify() (0 = re-derives)
.json	computed	the module structure JSON
.structure_maps	computed	the ExtensionClass of every consecutive layer pair
.jordan(p)	computed	the Jordan type of n_p on V, in units of f
.layer(j)	computed	the canonical basis of the Loewy layer m^j V / m^{j+1} V (columns)
.structure_map(j)	computed	the ExtensionClass of the layer pair (j, j+1)

ExtensionClass

the structure map theta_j between two consecutive Loewy layers ([E] D4): the extension class the generators n_p glue the layers by

Member	Kind	Description
.top_layer	computed	j
.bottom_layer	computed	j + 1
.top_dim	computed	the top layer's K-dimension
.bottom_dim	computed	the bottom layer's K-dimension
.rank	computed	F_ell rank of the stacked map
.rank_over_f	computed	its K-rank
.active	computed	the primes whose own map is nonzero

Member	Kind	Description
.rank_p	computed	the F_ell rank per generator
.split	computed	rank == 0: the two layers are not glued
.matrix(p)	computed	the matrix of n_p between the layers (rows bottom, columns top)

Presentation

generators and a minimal relation set of a certified-local block over T_m (HeckePresentation, [X] Sec. 2.2 Stage 5, OQ11)

Member	Kind	Description
.available	structural	whether the block was local
.min_generators	structural	t, over A_S
.min_relations	structural	over A_S
.dim_relation_module	structural	t dim A_S - D
.free	structural	no relations
.generator_indices	structural	the generators' standard indices
.generators	structural	the generators as columns
.relations	structural	the relations as t-tuples over the A_S basis
.verify	structural	Presentation::verify() (0 = re-derives)
.json	structural	the presentation JSON

Involution

the production involution iota^* = -iota^*_geo on H ([C] def:iota(iv); HeckeInvolution) with the +/- split and, per degree, the sign split of the Hecke action and the Eisenstein line (HeckeSignSplit); lazy

Member	Kind	Description
.n	computed	the symmetric power
.ell	computed	the prime
.status	computed	the split's status
.coefficient_ring	computed	F_ell, or Z/ell^k (slot 20: iota honours the ring)
.lengths_plus	computed	length H^q_+ per degree (= dims_plus over F_ell)
.lengths_minus	computed	length H^q_- per degree (= dims_minus over F_ell)
.dims	computed	dim H^q
.dims_plus	computed	dim H^q_+
.dims_minus	computed	dim H^q_-
.certificate	computed	InvolutionCertificate::verify() (0 = obl:iota discharged; over Z/ell^k: iota^2 = id and H = H^+ (+) H^- were ENGINE_CHECKed)
.json	computed	hecke_involution/v1 (over Z/ell^k: hecke_involution_ring/v1)
.invariant_factors_plus(q)	computed	H^q_+ = (+)_i Z/ell^{e_i}: the e_i, descending (F_ell: all 1)
.invariant_factors_minus(q)	computed	H^q_- = (+)_i Z/ell^{e_i}: the e_i, descending (F_ell: all 1)
.matrix(q)	computed	iota^* on H^q
.plus(q)	computed	a basis of H^q_+ (columns, in H^q coordinates)
.minus(q)	computed	a basis of H^q_- (columns, in H^q coordinates)
.sign_split(q)	computed	the Hecke action's sign split at {2,3,5,7} \ {ell} and the Eisenstein line

Member	Kind	Description
`.eisenstein_line(q)`	computed	the Eisenstein line in the stored H^q basis (defined only)

BoundaryCohomology

H^*_bd on the boundary circle (SL2ZBoundaryPair), with the restriction r and T_p^bd (HeckeBoundaryOperator); lazy

Member	Kind	Description
`.n`	computed	the symmetric power
`.ell`	computed	the prime
`.model`	computed	the boundary model
`.is_group_cohomology`	computed	2 in R^x (the circle computes group cohomology)
`.dims`	computed	dim H^q_bd
`.bulk_dims`	computed	dim H^q of the bulk
`.q_max`	computed	the top degree of THIS complex (derived)
`.truncated`	computed	false on a finite model
`.q_max_source`	computed	'derived' unless truncated (E9)
`.degrees`	computed	[0 .. q_max], derived
`.coefficient_ring`	computed	the complex's ring (the extension layer is F_ell)
`.r(q)`	computed	the restriction H^q -> H^q_bd
`.A(p, q)`	computed	T_p^bd on H^q_bd

ConeCohomology

H^*_c = H^*(Cone(r)) with its long exact sequence (EquivariantMappingCone) and, per prime, the cone action A_c(T_p) with h DERIVED from the certified pair homotopy eta over Z (HeckeConeAction); lazy

Member	Kind	Description
`.n`	computed	the symmetric power
`.ell`	computed	the prime
`.name`	computed	the cone's name
`.dims`	computed	dim H^q_c
`.bulk_dims`	computed	dim H^q of the bulk
`.boundary_dims`	computed	dim H^q_bd
`.q_max`	computed	the top degree of THIS complex (derived)
`.truncated`	computed	false on a finite model
`.q_max_source`	computed	'derived' unless truncated (E9)
`.degrees`	computed	[0 .. q_max], derived
`.coefficient_ring`	computed	the complex's ring (the extension layer is F_ell)
`.certificate`	computed	ConeCertificate::verify() (0 = re-derives)
`.H1_shriek`	computed	dim H^1_! = dim ker r^1 (nil if not derivable)
`.les`	computed	the long exact sequence spots {term, q, dim, rank_in, rank_out, asserted, exact}
`.les_exact`	computed	every asserted spot of the LES is exact
`.pi(q)`	computed	H^q_c -> H^q
`.r(q)`	computed	H^q -> H^q_bd
`.iota(q)`	computed	H^q_bd -> H^{q+1}_c (the connecting map)
`.A(p, q)`	computed	A_c(T_p) on H^q_c
`.h(p, q)`	computed	the cochain homotopy h at p in degree q, DERIVED from eta (not stored)

Member	Kind	Description
`.pair_homotopy(p)`	computed	the certified pair homotopy eta at p: {eta_zero, slots, certify, solve_regime, ...}
`.action_certificate(p)`	computed	ConeActionCertificate::verify() of the action at p (0 = re-derives)

DicksonLES

the long exact sequence of 0 -> Sym^n (x) det -> Sym^{n+ell+1} -> Q_n -> 0 on the model over F_ell, ell >= 5 ([E] sec:thetaobj; HeckeDicksonTheta): the six terms, the five maps in the stored bases (theta = L_* source-indexed, delta by the snake construction), exactness at all six spots as subspaces, H^0 the Dickson ring; lazy

Member	Kind	Description
`.n`	structural	the symmetric power of the source term
`.ell`	structural	the prime
`.n_up`	structural	n + ell + 1, the middle term's symmetric power
`.dims`	computed	{H0_n, H0_up, H0_Q, H1_n, H1_up, H1_Q}
`.ranks`	computed	{theta0, pi0, delta, theta1, pi1}
`.maps`	computed	{theta0, pi0, delta, theta1, pi1} as Matrix values in the stored bases (column convention)
`.exact`	computed	exactness certified at all six spots as subspaces
`.ker_theta`	computed	a basis of ker theta^1 (columns, in H1_n's stored coordinates)
`.im_delta`	computed	a basis of im delta (columns, in H1_n's stored coordinates)
`.ker_equals_im`	computed	ker theta^1 = im delta as subspaces of H1_n
`.dickson_count`	structural	#{(a, b) : a(ell + 1) + b(ell^2 - ell) = n}
`.certificate`	computed	DicksonThetaCertificate code (0 = every check of [E] (6a)-(6d) held)
`.coefficient_ring`	structural	the complex's ring (theta is F_ell only)

DescentRecord

the integral descent record integral_descent_record/v1 of H^q(Gamma_0(N); Sym^n Z^2) over Z ([C] thm:descentcert; slot-24 D6): the groups to q_max = 3 (object = group, truncated, periodic from q = 2), the bad primes, the certified precision K per prime, char_Z(T_p | H^1/tors), the torsion Hecke modules by their invariants, the boundary pair (sec:descentpair); only under coefficient_ring = Z; lazy

Member	Kind	Description
`.N`	structural	the level
`.n`	structural	the symmetric power
`.primes`	structural	the Hecke primes P (none dividing N)
`.coefficient_ring`	structural	Z (the record exists only under coefficient_ring = Z)
`.object`	structural	group (group cohomology, never Bredon; OD-10 (a))
`.q_max`	structural	3, derived from thm:descentcert(6)
`.truncated`	structural	true: degrees above q_max are not reported (they are periodic, thm:descentcert(6))
`.q_max_source`	structural	derived
`.degrees`	structural	[0, 1, 2, 3]
`.groups`	computed	H^0 .. H^3 as abelian groups ("Z^3 + Z/2 + Z/4"); empty when refused

Member	Kind	Description
`.bad_primes`	computed	B(M_N) of thm:descentcert(1): List of [ell, count] (prop:badprimesN(1)), ell >= 5
`.torsion_primes`	computed	the primes whose torsion is read: 2, 3 (n even) and B(M_N)
`.K`	computed	per prime of the groups' run: {ell, route, K, W, certified} (certification provenance, not hashed)
`.charpoly`	computed	per p: {p, r, coefficients (high -> low, String: an integer, "[sign,D digits]" beyond 2^63, "?" undetermined), exact (high -> low, String: every coefficient as an exact integer, "?" undetermined), determined, digest, refused} – char_Z(T_p | H^1/tors), prop:crtcharpolyN's output rule
`.charpoly_factored`	computed	per p: {p, status (ok | unavailable), expanded, factored ("(X + 24)^2 (X - 2049)"), factors: [{coefficients (high -> low, String), degree, multiplicity, irreducible}], complete, verified, note} – char_Z(T_p | H^1/tors) factored over Z (Yun + Zassenhaus, exact; the product of the factors is checked against char_Z); presentation only, not in the record's JSON or digest
`.torsion`	computed	per (q, ell): {q, ell, exponents, module_certified, ops: [{p, invariants}], refused} – the torsion Hecke module by its choice-free invariants (prop:torsionhecke(1))
`.pair`	computed	the boundary pair (sec:descentpair): {bulk, boundary, compact} each {q_max, truncated, q_max_source, degrees, groups}, q = 0 .. 4 (compact, the cone, also derived_from = [bulk, boundary]); refused
`.groups_inv`	computed	H^0 .. H^3 as structures: per q {free_rank, torsion: [[p, k, mult], ...]} = Z^free_rank + (+) (Z/p^k)^mult (the same groups as `groups`); empty when refused
`.pair_inv`	computed	the boundary pair's groups as structures: {bulk, boundary, compact}, each a List over q = 0 .. 4 of {free_rank, torsion: [[p, k, mult], ...]} (the same groups as `pair`)
`.status`	computed	ok | refused
`.refusal`	computed	the first refused part's reason ("" when ok)
`.digest`	computed	record_fnv1a64: FNV-1a-64 of the record's invariants and the settings' canonical text
`.json`	computed	the record as one JSON line (schema integral_descent_record/v1)
`.module(ell, K)`	structural	H^1(Gamma_0(N); Sym^n Z^2) (x) Z/ell^K with T_p (p in primes) by cochain-level lifts (slot-25 EXPORT; [C] prop:torsionhecke, prop:strictliftN): {N, n, ell, K, K_max, route, carrier, status (ok | blocked(export) | refused), refusal, free_rank, torsion (exponents at K), invariant_factors, generators: [{kind, exponent, cocycle (a cocycle mod ell^K)}], ops: [{p, matrix (column convention, row i mod ell^exponent_i), charpoly_free (high -> low, mod ell^K)}]}; one run at the route's width serves every K <= K_max (computed on the first call per ell; it does not force the record); ell in {2, 3}: the tree model, ell >= 5: the cokernel presentation (K_max <= 1 for ell >= 46341)

Member	Kind	Description
`.module_Z(mode)`	computed	H^1(Gamma_0(N); Sym^n Z^2) over Z as a presented Z[T_p]-module ([C] prop:torsionhecke): T_p = [[tau, M], [0, F]] in generators torsion first, then free; module_Z() = EXPORT-Z step 2 (F exact over Z), module_Z("local") = step 1 (slot-25 EXPORT-Z, F pending). {N, n, primes, status (ok | blocked(export)), refusal, splitting (export_retraction/v1), retractions: [{ell, route, K_run, K, fnv, [step 2: pi_B (t_ell x r, row a mod d_a: the retraction on B, the torsion coordinates of the classes [b_j]), pi_B_fnv]}], torsion_primes, largest_torsion_prime, invariant_factors (the elementary divisors d_i = ell^sigma_i, ell ascending; key name kept), torsion_generators: [{ell, exponent, d}], free_rank (a blocked value prints these four as "blocked"), free_block (step 2: "exact", or "refused" with free_refusal; local: "pending(step 2)"), M_basis (step 2: global, M in the basis B; local: M's ell-block is against module(ell, K)'s free generators), entry_encoding (step 2: exact, F as decimal integer strings; local: word), split (unknown), presentation, [step 2: basis: {rule (iso_dixon/v3), q0, saturation (det_pm1), basis_fnv, phi_fnv, bits_B, bits_Phi, bound (phi0 | phi_cert by the Auto size rule; -at rank 0)}, free_refusal], ops: [{p, tau (row i mod d_i, block-diagonal over ell), M: [{ell, K, matrix}] (step 2: against the classes [b_j] of B, T_p b_j = sum_i F_ij b_i + sum_a M_aj g_a, i.e. M = M_local C - pi_B F + tau pi_B; local: against module(ell, K)'s free generators), F (step 2: r x r decimal strings; local: "pending(step 2)"), F_local: [{ell, modulus, matrix}] (the export's free block mod ell^K, not F), charpoly_Z (the record's char_Z(T_p | H^1/tors), high -> low), [step 2: F_fnv, F_proof (apriori | apost | rank0), F_primes, M_fnv (FNV-1a-64 of M over every ell, span convention), C: [{ell, modulus, matrix}] (B in the export's local free generators: C F = F_local C mod ell^K)]}], torsion_canon: [{ell, ops: [{p, invariants}]}] (the record's)}; unique up to [[Aut T, Hom(L, T)], [0, GL_r(Z)]]

Appendix C

Settings reference

`math.` marks a mathematical setting (may change a value, moves the fingerprint); `engine` settings never change a value, and their default is named by the environment variable that supplies it. *env-only* keys are shown by `settings()` but cannot be `set`. The eleven mathematical rows share one registry boilerplate sentence; their values and meanings are in Section 7.4.1.

Key	Family	Kind	Default / source	Description
`print_max_entries`	engine	Int	session	entries printed before a value prints its summary form
`verbose`	engine	Bool	session	let the pipeline's own stdout/stderr chatter through (default: true in the REPL, false in scripts)
`locality`	**math.**	String	`"budget=64"`	mathematical setting `locality` (hecke::MathSettings – it changes what a result MEANS, so a non-default value moves the fingerprint and `golden(...)` refuses)
`u_ell`	**math.**	String	`"auto"`	mathematical setting `u_ell` (hecke::MathSettings – it changes what a result MEANS, so a non-default value moves the fingerprint and `golden(...)` refuses)
`sturm`	**math.**	String	`"auto"`	mathematical setting `sturm` (hecke::MathSettings – it changes what a result MEANS, so a non-default value moves the fingerprint and `golden(...)` refuses)
`sturm_unavailable`	**math.**	String	`"nonfinal"`	mathematical setting `sturm_unavailable` (hecke::MathSettings – it changes what a result MEANS, so a non-default value moves the fingerprint and `golden(...)` refuses)
`commutant_budget`	**math.**	String	`"20000000000"`	mathematical setting `commutant_budget` (hecke::MathSettings – it changes what a result MEANS, so a non-default value moves the fingerprint and `golden(...)` refuses)
`final_mode`	**math.**	String	`"closure"`	mathematical setting `final_mode` (hecke::MathSettings – it changes what a result MEANS, so a non-default value moves the fingerprint and `golden(...)` refuses)

Key	Family	Kind	Default / source	Description
`field_polynomial`	**math.**	String	`"least"`	mathematical setting `field_polynomial` (hecke::MathSettings – it changes what a result MEANS, so a non-default value moves the fingerprint and `golden(...)` refuses)
`split_field_report`	**math.**	String	`"canonical"`	mathematical setting `split_field_report` (hecke::MathSettings – it changes what a result MEANS, so a non-default value moves the fingerprint and `golden(...)` refuses)
`presentation`	**math.**	String	`"off"`	mathematical setting `presentation` (hecke::MathSettings – it changes what a result MEANS, so a non-default value moves the fingerprint and `golden(...)` refuses)
`predictions`	**math.**	String	`"annotate"`	mathematical setting `predictions` (hecke::MathSettings – it changes what a result MEANS, so a non-default value moves the fingerprint and `golden(...)` refuses)
`coefficient_ring`	**math.**	String	`"F_ell"`	mathematical setting `coefficient_ring` (hecke::MathSettings – it changes what a result MEANS, so a non-default value moves the fingerprint and `golden(...)` refuses)
`threads`	engine	Int	env `ENGINE_THREADS`	the engine consumer pool's worker count (engine::tape::consumer_pool().size()); thread-local, propagated onto run_concurrently tasks
`backend`	engine	String	env `ENGINE_BACKEND`	engine | legacy – the Phase 2/3 rollback; `legacy` holds engine::elim::ScopedLegacy and forces the legacy cohomology and Hecke-operator arms (thread-local; OC_ENABLE_LEGACY=ON builds only – an OFF build refuses `legacy`, exit 5) (one of: engine, legacy)
`device`	engine	String	env `ENGINE_DEVICE`	cpu | gpu | hybrid – pass 6's device policy (PassOptions); process-global. `cpu` is the golden path; the GPU is never the default (one of: cpu, gpu, hybrid)
`batch`	engine	Bool	env `ENGINE_BATCH`	pass 5, batching (PassOptions; process-global)
`ksplit`	engine	Bool	env `ENGINE_KSPLIT`	pass 4, k-splitting of GEMMs with k > k_max (PassOptions; process-global)
`arena`	engine	Bool	env `ENGINE_ARENA`	the temp arena (false: every Temp owns its storage) (PassOptions; process-global)
`cse`	engine	Bool	env `ENGINE_CSE`	pass 2, common subexpressions (PassOptions; process-global)
`fuse`	engine	Bool	env `ENGINE_FUSE`	pass 2b, product-into-AXPY fusion (PassOptions; process-global)
`layout`	engine	Bool	env `ENGINE_LAYOUT`	pass 3, layout (PassOptions; process-global)

Key	Family	Kind	Default / source	Description
`lean`	engine	Bool	session	the memory-lean configuration of `--bench --lean`: drop the cohomology objects no production consumer reads and skip the Theta_eq solve (A^(q) unchanged); thread-local, captured at a handle's CREATION like every setting
`program_cache`	engine	Bool	session	whether compiled tape programs are served from the program cache (false: a fresh compile every time); process-global
`confirm`	engine	String	session	a call outside a verb's silent cost envelope (NBCONFIRM): `ask` = on a terminal REPL show the estimate and ask, in a served session answer confirm_required with a per-cell token, elsewhere refuse as before (exit 3); `yes` = run it (hard memory / disk refusals still refuse); `no` = always refuse as before; not settable in a served session (one of: ask, yes, no)
`confirm_wall_s`	engine	Int	session	seconds: on a terminal REPL or in a served session, a call outside its silent envelope whose estimated wall time (upper edge) is above this is confirmed before it runs (NBCONFIRM; default 60; serve-settable)
`host_threads`	engine	String	env `ENGINE_HOST_THREADS` (env-only)	environment-only: read once by the engine from ENGINE_HOST_THREADS (empty = unset); there is no in-process override
`tile_mult`	engine	String	env `ENGINE_TILE_MULT` (env-only)	environment-only: read once by the engine from ENGINE_TILE_MULT (empty = unset); there is no in-process override
`tile_cols`	engine	String	env `ENGINE_TILE_COLS` (env-only)	environment-only: read once by the engine from ENGINE_TILE_COLS (empty = unset); there is no in-process override
`spin`	engine	String	env `ENGINE_SPIN` (env-only)	environment-only: read once by the engine from ENGINE_SPIN (empty = unset); there is no in-process override
`elim_base`	engine	String	env `ENGINE_ELIM_BASE` (env-only)	environment-only: read once by the engine from ENGINE_ELIM_BASE (empty = unset); there is no in-process override
`cuda_disable`	engine	String	env `ENGINE_CUDA_DISABLE` (env-only)	environment-only: read once by the engine from ENGINE_CUDA_DISABLE (empty = unset); there is no in-process override
`cuda_device`	engine	String	env `ENGINE_CUDA_DEVICE` (env-only)	environment-only: read once by the engine from ENGINE_CUDA_DEVICE (empty = unset); there is no in-process override
`cuda_program_cache`	engine	String	env `ENGINE_CUDA_PROGRAM_CACHE` (env-only)	environment-only: read once by the engine from ENGINE_CUDA_PROGRAM_CACHE (empty = unset); there is no in-process override
`cuda_graph`	engine	String	env `ENGINE_CUDA_GRAPH` (env-only)	environment-only: read once by the engine from ENGINE_CUDA_GRAPH (empty = unset); there is no in-process override

Key	Family	Kind	Default / source	Description
cuda_merge_levels	engine	String	env ENGINE_CUDA_MERGE_LEVELS (env-only)	environment-only: read once by the engine from ENGINE_CUDA_MERGE_LEVELS (empty = unset); there is no in-process override
cuda_pinned_mb	engine	String	env ENGINE_CUDA_PINNED_MB (env-only)	environment-only: read once by the engine from ENGINE_CUDA_PINNED_MB (empty = unset); there is no in-process override
cuda_copy_threads	engine	String	env ENGINE_CUDA_COPY_THREADS (env-only)	environment-only: read once by the engine from ENGINE_CUDA_COPY_THREADS (empty = unset); there is no in-process override
trace_nodes	engine	String	env ENGINE_TRACE_NODES (env-only)	environment-only: read once by the engine from ENGINE_TRACE_NODES (empty = unset); there is no in-process override
trace_rss	engine	String	env ENGINE_TRACE_RSS (env-only)	environment-only: read once by the engine from ENGINE_TRACE_RSS (empty = unset); there is no in-process override
trace_elim	engine	String	env ENGINE_TRACE_ELIM (env-only)	environment-only: read once by the engine from ENGINE_TRACE_ELIM (empty = unset); there is no in-process override
trace_levels	engine	String	env ENGINE_TRACE_LEVELS (env-only)	environment-only: read once by the engine from ENGINE_TRACE_LEVELS (empty = unset); there is no in-process override
trace_passes	engine	String	env ENGINE_TRACE_PASSES (env-only)	environment-only: read once by the engine from ENGINE_TRACE_PASSES (empty = unset); there is no in-process override

Appendix D

Validation roster

The 289 registered validations of this build, sorted by name, with their tiers (75 `fast`, 166 `standard`, 48 `slow`).

BarCocycleSolverValidation	fast	EngineCudaMemoryValidation	standard
BinaryF2Validation	standard	EngineDeterminismCoreValidation	standard
BlockEliminationSlowValidation	slow	EngineDeterminismValidation	standard
BlockEliminationValidation	standard	EngineDeviceTrapValidation	slow
BlockFlatMatrixValidation	standard	EngineElimPassOptionsValidation	fast
BlockMatValidation	standard	EngineElimSessionValidation	standard
BlockModelPipelineValidation	standard	EngineFingerprintDeviceBitValidation	fast
CampaignCertificateValidation	fast	EngineLadicInductionValidation	standard
CampaignRingValidation	standard	EngineTapeConsumerValidation	standard
CampaignSmokeValidation	standard	EngineTapeEliminationValidation	standard
CliManualCoverageValidation	fast	EngineTapeValidation	standard
CoefficientRingKthRootValidation	fast	EngineU128CompositeValidation	standard
CohomologyBlockModelValidation	standard	FlatMatrixValidation	fast
ConsoleC0Validation	fast	GoldenDump	fast
ConsoleCancelValidation	standard	GoldenEmit	fast
ConsoleCliPreservationValidation	fast	GoldenRegressionValidation	standard
ConsoleCostModelValidation	fast	GpuLeanSessionValidation	standard
ConsoleDeterminismValidation	standard	HeckeAtlasSkeletonValidation	fast
ConsoleErrorValidation	fast	HeckeAtlasValidation	slow
ConsoleReadOnlyTagValidation	fast	HeckeBadPrimeValidation	fast
ConsoleScriptEmit	fast	HeckeBenchmarkSmoke	slow
ConsoleScriptValidation	fast	HeckeBlockFormValidation	standard
ConsoleServeAtlasValidation	standard	HeckeBocksteinCuspValidation	fast
ConsoleServeValidation	standard	HeckeBoundaryFinalityValidation	standard
ConsoleSettingsValidation	fast	HeckeBoundaryOperatorValidation	standard
ConsumerMigrationValidation	fast	HeckeBoundaryParabolicValidation	standard
ContractCensusValidation	fast	HeckeBoundaryTauValidation	standard
CudaBitIdentityValidation	standard	HeckeBredonTreeRingValidation	standard
CudaImmaBitIdentityValidation	standard	HeckeBredonTreeValidation	fast
CudaImmaElimValidation	standard	HeckeBredonU64FlatRouteValidation	fast
CudaTransferValidation	standard	HeckeCanonicalFieldValidation	fast
DenseElimPipelineValidation	standard	HeckeCensusCellListValidation	fast
DenseElimSlowValidation	slow	HeckeCensusLatticeValidation	standard
DenseElimValidation	standard	HeckeCensusSplitValidation	standard
DescentSmithGpuValidation	standard	HeckeConeActionValidation	standard
DescentSmithMtValidation	standard	HeckeConeCrossRouteRingValidation	standard
EngineArenaFirstFitValidation	fast	HeckeConeValidation	standard
EngineArenaValidation	standard	HeckeCosetTablesSelfCheckValidation	fast
EngineCoreScalingValidation	standard	HeckeCosetTablesValidation	fast
EngineCrossDeviceValidation	standard	HeckeCrossCheckValidation	standard

HeckeDecompositionSmokeValidation	fast	HeckeLambdaModSymValidation	standard
HeckeDicksonThetaCoverageSlowValidation	slow	HeckeLambdaQExpValidation	standard
HeckeDicksonThetaHeckeCoverageSlowValidation	slow	HeckeLambdaTowerValidation	standard
HeckeDicksonThetaHeckeValidation	standard	HeckeLambdaTwoVarValidation	standard
HeckeDicksonThetaValidation	standard	HeckeLambdaValidation	standard
HeckeDualityCoverageSlowValidation	slow	HeckeLatticeModuleValidation	slow
HeckeDualityValidation	standard	HeckeLevelConeValidation	standard
HeckeEndToEndValidation	standard	HeckeLevelInfResValidation	standard
HeckeExactIntSmokeValidation	fast	HeckeLevelNCurveValidation	standard
HeckeExtAlgebraInvariantsValidation	standard	HeckeLevelNHeilbronnValidation	standard
HeckeExtAlgebraValidation	standard	HeckeLevelNSparseValidation	standard
HeckeExtArchitectureValidation	fast	HeckeLevelNValidation	standard
HeckeExtCampaignValidation	standard	HeckeLevelNormalityValidation	standard
HeckeExtDeterminismValidation	standard	HeckeLocalityProbe	slow
HeckeExtEndToEndValidation	slow	HeckeLocalitySlowValidation	slow
HeckeExtKummerValidation	slow	HeckeLocalityValidation	standard
HeckeExtRamanujanValidation	standard	HeckeMazurEisValidation	standard
HeckeExtRecordValidation	fast	HeckeMixedFinalityValidation	slow
HeckeExtSeparationValidation	standard	HeckeModuleMtValidation	slow
HeckeExtSmallEllValidation	standard	HeckeModuleStreamSlowValidation	slow
HeckeExtSturmSweepValidation	standard	HeckeModuleStreamValidation	standard
HeckeExtendedRegimeValidation	fast	HeckeMorphismGeneralizationValidation	fast
HeckeFamilyConcurrentBuildValidation	fast	HeckeMorphismRegressionValidation	standard
HeckeFormsFastMulValidation	standard	HeckeNebentypusValidation	standard
HeckeFormsHeckeAlgebraValidation	standard	HeckeNebentypusWildValidation	standard
HeckeFormsNtt3Validation	standard	HeckeOperatorFamilySmokeValidation	fast
HeckeFpFilterValidation	standard	HeckeOperatorSmokeValidation	fast
HeckeFreeModelValidation	standard	HeckeOracleValidation	standard
HeckeHidaCMValidation	standard	HeckeOrdinaryPrecisionValidation	standard
HeckeHidaComponentsValidation	standard	HeckePairHomotopyValidation	standard
HeckeHidaControlValidation	standard	HeckePeriodicityClassValidation	standard
HeckeHidaDiscLambdaValidation	standard	HeckePrecisionReuseSlowValidation	slow
HeckeHidaEisLambdaValidation	standard	HeckePrecisionReuseValidation	standard
HeckeHidaHeavyCellProbe	slow	HeckePredictionsValidation	fast
HeckeHidaLevelEllValidation	standard	HeckePresentationValidation	fast
HeckeHidaTowerSlowValidation	slow	HeckeReductionCoverageSlowValidation	slow
HeckeHidaTowerValidation	standard	HeckeReductionValidation	standard
HeckeIdentityValidation	fast	HeckeRelationsValidation	standard
HeckeIndependenceValidation	standard	HeckeRho2StreamValidation	standard
HeckeInducedLocalSystemSmokeValidation	fast	HeckeRingLoewyDigitSlowValidation	slow
HeckeIntegralActionValidation	standard	HeckeRingLoewyValidation	standard
HeckeIntegralCharpolyValidation	standard	HeckeSatCertDumpValidation	fast
HeckeInvolutionValidation	standard	HeckeSatPerfValidation	slow
HeckeIotaSignValidation	standard	HeckeSaturationDefectValidation	standard
HeckeIotaStreamValidation	standard	HeckeShapiroValidation	fast
HeckeIotaTauSymmetryValidation	fast	HeckeSimultaneousSmokeValidation	fast
HeckeIsotropyModelEllValidation	standard	HeckeSimultaneousValidation	standard
HeckeIsotropyModelTreeSlowValidation	slow	HeckeSmallCarriesValidation	fast
HeckeIsotropyModelTreeValidation	standard	HeckeStrictLiftRelationsValidation	standard
HeckeIsotropyModelValidation	standard	HeckeStrictLiftValidation	standard
HeckeLValueCoefficientRingValidation	standard	HeckeSturmBoundValidation	fast
HeckeLValueEisensteinMeasureValidation	standard	HeckeTameFamilyValidation	standard
HeckeLValueIwasawaInvariantsValidation	standard	HeckeTauDumpValidation	fast
HeckeLValueMeasureValidation	standard	HeckeTauTableSelfCheckValidation	fast
HeckeLValueShapiroValidation	standard	HeckeTauTableValidation	fast
HeckeLValueValidation	standard	HeckeTessellationSmokeValidation	fast
HeckeLValueWildCharacterValidation	standard	HeckeTessellationValidation	fast
HeckeLadicCellValidation	standard	HeckeThetaBlockValidation	standard
HeckeLadicSplitValidation	slow	HeckeThetaCensusRingRefusalValidation	fast

HeckeThetaRssDietValidation	slow
HeckeUEllBudgetValidation	fast
HeckeUEllLowRankValidation	slow
HeckeUEllReachSlow	slow
HeckeUEllReachValidation	standard
HeckeUEllStreamValidation	slow
HeckeWideTauSlowValidation	slow
HeckeWideTauValidation	standard
HeckeZCertDumpValidation	slow
HeckeZEisUpValidation	standard
HeckeZExportValidation	standard
HeckeZExportZ2SlowValidation	slow
HeckeZExportZ2Validation	standard
HeckeZExportZValidation	standard
HeckeZFreeBasisPairsValidation	standard
HeckeZFreeBasisSatValidation	standard
HeckeZFreeBlockCrtValidation	standard
HeckeZLocalCensusXcheck	slow
HeckeZLocalParValidation	standard
HeckeZLocalValidation	standard
HeckeZRationalValidation	fast
HeckeZRecordValidation	standard
HeckeZTorValidation	standard
HeckeZUTagValidation	fast
HeckeZUpValidation	standard
HostElimThreadDeterminismValidation	standard
IntegralDescentGroupsSlowValidation	slow
IntegralDescentGroupsValidation	standard
IntegralDescentHeckeSlowValidation	slow
IntegralDescentHeckeValidation	standard
IntegralDescentLevelNSlowValidation	slow
IntegralDescentLevelNValidation	standard
IntegralDescentMapsSlowValidation	slow
IntegralDescentMapsValidation	standard
IntegralDescentPairSlowValidation	slow
IntegralDescentPairValidation	standard
IntegralDescentParAsmValidation	slow
IntegralDescentParLnValidation	slow
IntegralDescentRecordValidation	standard
IntegralDescentSmithBlockedValidation	standard
IntegralDescentTreePairSlowValidation	slow
IntegralDescentTreePairValidation	standard
IsotropyModelDeltaSquaredValidation	fast
KernelAvx2BitIdentityValidation	fast
KernelBitIdentityValidation	standard
L23AmalgamReaderValidation	standard
L23Bin3o1Validation	slow
LeanCensusFileValidation	fast
LicenceGuardValidation	fast
LocalSystemCohomologyValidation	fast
MatrixRetirementValidation	fast
MatrixValidation	fast
ModContextValidation	standard
ModCtxMontgomeryPairValidation	fast
NumberFieldCyclicCubicValidation	standard
NumberFieldLllValidation	fast
NumberFieldPRankValidation	standard
NumberFieldRibetFieldValidation	slow
NumberFieldValidation	fast
OrbifoldCampaignL23Validation	standard
OrbifoldCensusL23SlowValidation	slow
OrbifoldCensusL23Validation	standard
OrbifoldF3lValidation	slow
OrbifoldMPAllDegreesValidation	standard
OrbifoldMPBoundaryValidation	standard
OrbifoldMPComparisonValidation	standard
OrbifoldMPHeckeValidation	standard
OrbifoldMPLevelConeValidation	standard
OrbifoldMPNerveH3Slow	slow
OrbifoldMPNerveValidation	standard
OrbifoldMPRefinementValidation	standard
OrbifoldMPRulesValidation	fast
OrbifoldMPTeardropValidation	fast
OrbifoldMod3kValidation	slow
OrbifoldRing2kValidation	slow
PerformanceSmokeValidation	standard
ProgramCacheReleaseValidation	fast
Ring2kLadderValidation	standard
Ring2kR5SimdValidation	standard
SharedHelpersValidation	fast
SimplicialCohomologyValidation	fast
SmallEllPolicyScopeValidation	fast
SubstrateDifferentialValidation	standard
SymmetricPowerValidation	standard
SymmetricPowerValidationSlow	slow
TernaryF3Validation	standard
ThetaEqSolveValidation	standard
VacuousRelationValidation	standard
ValidationHarnessCatchValidation	fast
VnniGemmBitIdentityValidation	standard
mod_arithmetic_validation	fast

Appendix E

Regenerating this manual

The files under `docs/latex/cli_manual/` are generated from a built binary:

```
docs/latex/tools/gen_cli_manual_tables.sh <path/to/hacs>           # rewrite the
↪  tables
docs/latex/tools/gen_cli_manual_tables.sh <path/to/hacs> --check   # diff only, exit 1
↪  if stale
docs/latex/build_cli_manual.sh <outdir>       # pdflatex + makeindex; outdir in
↪  scratch
```

The generator runs `CLI_MANUAL_EMIT=<scratch> <binary> CliManualCoverageValidation`, which writes the tables into a scratch directory, and copies them into the tree. The fast validation `CliManualCoverageValidation`, part of every `--all`, checks that (1) every committed table equals what the binary generates; (2) every registered verb, type, setting, validation and command-line token appears in them; (3) this file inputs each of them; (4) the token table equals the set of strings the parsers of `main.cpp`, `Bench/`, `Campaign/`, `Console/Core/ConsoleCli.cpp`, `HeckeExt/HeckeAtlasCli.cpp` and the `--backend` parser compare against; (5) the top-level usage equals the frozen `cli_frozen/help.stdout`; (6) every excerpt line of the manual occurs in its script and every quoted transcript exists; (7) every registered verb is called in a numbered script that the manual names (by an excerpt, a transcript or a `\file{}` reference), or is on the allow-list `docs/latex/cli_manual_gate.tsv` with a reason (an allow-list entry for a verb that has an example, or is not registered, is stale and fails); (8) every `ENGINE_*`, `HECKE_DESCENT_*` and `L23_*` name read in the sources, and every `lever` row of that file, appears as `\env{NAME}` in the environment chapter (Sections 9.3–9.14); (9) every `\runoutput{<name>}` has its `docs/latex/cli_manual_runs/<name>.txt`. The run outputs are regenerated by `docs/latex/tools/gen_cli_manual_runs.sh <binary>` (`--check` compares them with timings masked). Prose outside the generated files is not checked beyond (6)–(9): when a front end changes, revise the matching section too.

Appendix F

Versions

1.5.13 (2026-11-09, release audit fixes; not binding). No change of mathematical content and no table, count, verb, type, setting or token changes. The program is called `hacs` throughout (title, quick start, examples, the quoted run outputs; `cohomology_app` remains a link to it, Section 1.2). The two sections on running a census with the project's own batch scripts on its own machines (formerly "Running a census on one machine" and "A second machine") are replaced by one neutral section, Section 8.7 ("Long census runs"): the scripts are not part of the release, and the general rules they implemented are kept. The presets paragraph of Section 1.3 names the `termux` / `aarch64-cpu` presets and points to `BUILDING.md`.

1.5.12 (2026-11-09, slot 26 LAYOUT; not binding). Paths only, no change of content (repository layout, owner ruling 2026-11-09): the sources moved under `src/` (`src/hacs/`, `src/ModArithmetic/`, `src/Engine/`, `src/Cuda/`); the script folder is `src/hacs/Console/scripts/`; builds are out-of-source only (Section 1.3, and one troubleshooting item). No table, count, verb, type, setting or token changes.

1.5.11 (2026-11-09, slot 26; not binding). No change of mathematical content: machine names and private paths removed for the public release (owner ruling 2026-11-09); the section on the second machine is now written without its host name (label `sec:second-machine`).

1.5.10 (2026-11-08, slot-25 PERF-VF; not binding). The environment row `ENGINE_VNNI2_FLOOR` (the two-limb VNNI route's work-per-thread floor, Section 9.5): its default, its parsing (digits only) and when its EFFECTIVE line prints. No other table, count, verb, type, setting or token changes.

1.5.9 (2026-11-07, slot-25 LEVERS-25; not binding). Wording only: the env arms `=0` of `ENGINE_GPU_LEAN`, `HECKE_RHO2_STREAM`, `HECKE_MODULE_STREAM`, `HECKE_MODULE_EVAL`, `HECKE_MODULE_PARALLEL`, `HECKE_ROUTE2_SHARED_MEMO`, `HECKE_RHO_SIMD` (also `scalar`), `HECKE_PRECISION_REUSE`, `HECKE_FORMS_FASTMUL`, `L23_TERNARY_F3`, `L23_BIN3O1` (and `HECKE_IOTA_STREAM`, not listed in the tables) are described as retired: refused in a default build, the earlier route in a legacy build (Sections 9.6, 9.9, 9.10). No table, count, verb, type, setting or token changes.

1.5.8 (2026-11-07, slot-25 NBCONFIRM S5; not binding). Interactive mode: `Ctrl-C` cancels the running statement at its next cancellation point (the old sentence "a running verb cannot be interrupted" is withdrawn); the served `interrupt` likewise. One validation added to the generated tables (`ConsoleCancelValidation`); no verb, type, setting or token changes.

1.5.7 (2026-11-06, LaTeX polish stage 1; not binding). Uniform author block (the former author line was "hecke-extensions project") and the dedication "In memoriam Prof. Dr. Günter Harder (1938–2025)." at the head of the abstract. No table, count, verb, type, setting or token changes.

1.5.6 (2026-11-06, slot 25 HZ-REC; not binding). The verb `hecke_structure` and the campaign keys `structure`, `algebra`, `hz_cert` (paragraph in the chapter on integral cohomology; script `060_s25_hecke_structure.oc`).

1.5.5 (2026-11-05, slot 24 close; CL-R2 audit finding M6; not binding). Wording only: the rollback arms `ENGINE_ELIM_SESSION=0`, `ENGINE_GPU_IMMA=0` and `ENGINE_GPU_ELIM=0` are described as

retired (refused in a default build, comparison arms in a legacy build), and the four U_ℓ memory levers as retired by CL-R1 (Sections 9.6, 9.7, 9.9). No table, count, verb, type, setting or token changes.

1.5.4 (2026-11-03, slot 24 FMTMP; not binding). The descent record's `.charpoly` entries gain `exact`, every coefficient as an exact integer; `.charpoly_factored` reads the coefficients beyond 2^{63} from the record's exact values (`[C, sec:recordschema]` item 7 (l)); the generated type table is regenerated.

1.5.3 (2026-10-31, slot 24 CL-7; owner ruling 4 of 2026-10-30; not binding). With no argument the program prints the top-level usage and exits 1; the legacy benchmark is gone (Section 1.4). The frozen usage text now lists `--help`, `--atlas`, the console forms and `--serve`; the dispatch lives in `main.cpp`.

1.5.2 (2026-10-29, slot 24 NB-R34; not binding). The verb `display` (Section 7.2.6, with an example; [P] 3.14.0-pre.2) and what the notebook page renders (Section 7.8); the generated tables count one more verb and overload.

1.5.1 (2026-10-29, slot 24 DOCS-FIX; not binding). Referee fixes C1–C12 (`docs/implementation/slot24/DOCS_referee_1029.md`): $5 \equiv 1 + 2^{11} \pmod 7$ in the level-1 example (C1); one Eisenstein class at level 11, weight 2 (C2); the golden reference pins nine descent cells, 156 tier-A entries (C3); the invariants string has `ker=`/`cok=` only when certified with at most 400 summands, and `B=` blocks above 300 summands read `-` (C4); the `shortcut` route at 3 and the rollback routes listed (C5); `D digits` are mixed-radix digits (C6); every piece is certified or refused (C7); the boundary pair is certified by `[C, prop:descentpair]` (C8); the orbifold census introduced as the second route at level 1, with its cross-check (C9); §8.6 opens with the mathematics, the `orbifold_cell` v1→v2 history in a footnote (C10); stabiliser orders $2, 4, 6$ (C11); `Z` for rank one (C12). No generated table or run output changed.

1.5.0 (2026-10-29, slot 24 DOCS-CLI; not binding). *Restructured by subject* (owner direction 2026-10-29): getting started; the mathematical objects (new: the model, Sym^n, the coefficient rings, levels and nebentypus); cohomology over a field (with $\mathbb{Z}/\ell^k$ and the Moerdijk–Pronk model); Hecke operators; *integral cohomology by descent* (new chapter: what is computed, the algorithm step by step, reading the `DescentRecord` including the torsion-invariants grammar, worked examples, limits); extensions, Hida theory and the forms algebra (new prose for `pow`, `kummer_unit`, `hida_tower`, `level_n`, `nebentypus`, `forms_algebra`, `hida_lambda`, `lfunction`); the console and its language (new: a tour of the small verbs, the scripts by subject); campaigns (new: campaigns over $\mathbb{Z}$, a sample orbifold run); performance and engine switches (new: a complete environment chapter, among them `ENGINE_KERNEL`, `ENGINE_AVX2_PAIR`, `ENGINE_GPU_SMITH2K*`, `ENGINE_GPU_LEAN`, `ENGINE_TERNARY_F3`, the `HECKE_DESCENT_*` levers with the panel rule of `dca97bb8`, the module and forms levers, the `L23_*` levers). New console scripts `054_s24_descent.oc` and `055_s24_language_tour.oc` (frozen transcripts); real command-line outputs generated by the new `docs/latex/tools/gen_cli_manual_runs.sh` into `docs/latex/cli_manual_runs/`. Corrected: the orbifold record is `orbifold_cell/v2`; the `ENGINE_DEVICE` row (the GPU is the fast route of the descent's 2-adic Smith form); the counts (289 validations). `CliManualCoverageValidation` now also requires a worked example per verb (allow-list `docs/latex/cli_manual_gate.tsv`), the environment chapter to name every engine, descent and `L23` lever and every listed rollback lever, and every quoted run output to exist. Version history and all section labels of 1.4.1 are kept.

1.4.1 (2026-10-26, slot 24 D6; wording only; not binding). `coefficient_ring = Z` is no longer described as refused: it is stored (fingerprint non-zero) and honoured by the integral descent only (the verb `descent`, the campaign's descent cells, the record `integral_descent_record/v1`; `[C, sec:recordschema]` item 7, `[C, def:ringsemantics]`(3)); every other consumer refuses it by name with exit 5; `Z/<m>` stays reserved. Corrected: the exit-code table (exit 5 row), the `coefficient_ring` row of the settings table, the troubleshooting item on exit code 5. No generated table moves.

1.4.0 (2026-10-12, slot 22 CLOSE-T). The release of 1.4.0-pre.2: only the version changes (no table,

count, verb, type, setting or token; not binding).

1.4.0-pre.2 (2026-10-11, slot 22; pre-release). The generated tables checked against the slot-22 census binary (`gen_cli_manual_tables.sh --check`: current, 22 files; 289 validations, no new verb, type, setting or token). Prose only. New text: Section 8.6 (the orbifold census at $\ell = 2, 3$: the keys `orbifold`, `orbifold_primes`, `orbifold_m_target`, `orbifold_k_cap`; the `orbifold_cell/v1` record (`v2` since slot 24); reading the `EFFECTIVE campaign arm=orbifold` line; resume, which stops on a failed record; measured memory and time); Section 8.7 (the second machine, a second workstation: the CPU-only build, the anchor gate, the bulk command, PAUSE and resume, the gate); Section 9.14 (the LN23 hooks of `HeckeLevelNCurveValidation`, the Λ-tower cells and caps of `HeckeLambdaTowerValidation`, the saturation-defect hooks); one row in the engine-switch table for the GPU tuning knobs `ENGINE_CUDA_GRAPH`, `ENGINE_CUDA_MERGE_LEVELS`, `ENGINE_CUDA_PROGRAM_CACHE`, `ENGINE_ELIM_SESSION` and the trace `ENGINE_TRACE_DEVICE`. The paragraph on the orbifold section in Section 8.1 is replaced by Section 8.6. Corrected the same day after the audit `docs/implementation/slot22/CLI_pre2_audit.md` (the second machine's build commit and roster count, the Λ-tower caps, seven clarifications).

1.4.0-pre.1 (2026-10-10, slot 22; pre-release). The prose brought up to the slot-22 tables (regenerated in the tree): 289 validations (75 `fast`, 166 `standard`, 48 `slow`), 241 run by `--all`; verbs, types and settings unchanged. New text: Section 9.7 (`ENGINE_BINARY_F2`, `ENGINE_GPU_IMMA`, `ENGINE_GPU_ELIM*`, the `EFFECTIVE DEVICE_ELIM` line); Section 3.4 (the ten Moerdijk–Pronk validations, `ORBIFOLD_MP_*`); Section 8.7 (`run_local_census.sh`, the GPU lane); the `--atlas --query` paragraph; new rows in the engine-switch and read-only-hook tables.

1.3.1-pre.1 (2026-10-08, slot 22 DOC22; pre-release). One paragraph added to Section 7.8 (the notebook's page themes, client-side only, [P] Section 18.17). No table, count, verb or token changes.

1.3.0 (2026-10-06, the slot-21 close). The tables regenerated from the binary of the release commit (`gen_cli_manual_tables.sh --check`: current); 289 validations (75 `fast`, 166 `standard`), 241 run by `--all`; 165 verbs with 178 overloads, 169 of them served. New since 1.2.0, all served (`read_only`): the verbs `pow`, `kummer_unit` and `hida_tower` (slot 21) and `level_n`, `nebentypus`, `forms_algebra`, `hida_lambda` (early slot 22, WP-VERB-2, landed before the slot-21 close); eleven new validations — `LicenceGuardValidation`, `NumberFieldLllValidation` (`fast`), `HeckeLValueCoefficientRingValidation`, `HeckeLValueEisensteinMeasureValidation`, `HeckeLValueIwasawaInvariantsValidation`, `HeckeLValueWildCharacterValidation`, `HeckeLambdaModSymVa` `HeckeLambdaQExpValidation`, `HeckeLambdaTwoVarValidation`, `HeckeLevelNCurveValidation`, `NumberFieldRibetFieldValidation` (`standard`). Appendix F's entries for 1.2.0 and earlier now give their counts literally. No front-end section changes.

1.2.0 (2026-10-05, the slot-20 close). The tables regenerated from the close binary (`gen_cli_manual_tables.sh --check`: current); 182 validations, 165 run by `--all`; 155 verbs with 168 overloads, 159 of them served. New since 1.1.0: the verb `lfunction` (slot-20 S1; the record `lfunction/v1`, the one-variable ℓ-adic series of a linear route, `[E, sec:lvlfunction]`; served); the involution `iota` honours `coefficient_ring Z/ell^<k>` (slot-20 WP-LV3, `[E, sec:ring]`; no longer exit 5); two new validations, `HeckeLValueMeasureValidation` (WP-MEAS, `standard`; the 691 arm `HECKE_LVALUE_MEASURE_691=1`) and `ConsoleServeAtlasValidation` (NB-4, `standard`); three token rows made precise (slot-20 WP-8: the spec value `off` of `level_a`, the range $5 \le \ell \le 100000$ of `--atlas`, `-` reads the script from stdin); and, from 1.2.0-pre.1, the atlas server `--serve-atlas` (Section 7.9; protocol `oc-serve-atlas/1`, [P] v3.11 Section 18.16; its tokens `--store`, `--index`, `--expect-fnv`, `--max-reply-bytes`, `--rss-cap-mib`) and the sidecar writer `--atlas --index` / `--out`; one validation, `ConsoleServeAtlasValidation` (`standard`).

1.1.0 (2026-10-04, the slot-19 close). The tables regenerated from the close binary (`gen_cli_manual_tables.sh --check`: current). New since 1.0.0: the served console `--serve` (Section 7.8; protocol `oc-serve/1`, [P] v3.10 Section 18; its tokens `--rss-cap-mib`, `--max-reply-bytes`; built only with `OC_BUILD_NOTEBOOK=ON`); the per-overload `read_only` tag, shown in Appendix A as [served] / [not served], with its twin `cli_manual/read_only_verbs.json` and the counts

158 of 167 (at 1.1.0); the verb `report` split into a served pathless overload and the file-writing overload (167 overloads at 1.1.0, one more than 1.0.0); the build switches and presets of Section 1.3; six new validations — `NumberFieldValidation`, `ConsoleReadOnlyTagValidation` (`fast`), `HeckeLValueValidation`, `ConsoleServeValidation`, `HeckeLambdaValidation` (`standard`), `HeckeLatticeModuleValidation` (`slow`) — so 180 in all at 1.1.0, 163 run by `--all`.

1.0.0 (2026-10-03, the slot-18 close). The first release; refereed as 1.0.0-pre.1 (Fable F-M, ACCEPT WITH REPAIRS), repairs at 1.0.0-pre.2.

Index

- (command line), 9
- (console), 32

abs (verb), 71
add (verb), 73
algebra (verb), 74
algebra (campaign spec key), 46
algebra_invariants (verb), 78
--all (command line), 8
--all-full (command line), 8
annotate (verb), 71
arena (setting), 98
assert (verb), 72
Atlas (type), 88
atlas (verb), 79
--atlas (command line), 9

--backend (--bench), 55
backend (setting), 98
batch (setting), 98
bench (verb), 77
--bench (command line), 9
bench_line (verb), 77
bernoulli_mod (verb), 78
Block (type), 86
boundary (verb), 80
BoundaryCohomology (type), 93

-c (command line), 9
-c (console), 32
--calibrate (command line), 9
--calibrate=thorough (command line), 9
--campaign (command line), 9
--campaign-cell (--bench), 55
campaign_plan (verb), 77
campaign_spec (verb), 77
campaign_spec_text (verb), 77
CampaignSpec (type), 87
canonical_field (verb), 79
canonical_field_polynomial (verb), 78
certify (campaign spec key), 47
charpoly (verb), 75
check_identities (verb), 79
CLI_MANUAL_* (environment), 42
CLI_MANUAL_EMIT (environment), 64
--close (--atlas), 30
coefficient_ring (setting), 98
coeffs (verb), 73
cohom (--bench), 55
Cohomology (type), 83
cohomology (verb), 75
--collect-only (--atlas), 30
column_space (verb), 73
columns (verb), 74
commutant (verb), 74
commutant_budget (setting), 97
Complex (type), 83
complex (verb), 75
compute (verb), 72
concurrent_jobs (campaign spec key), 47
cone (verb), 80
ConeCohomology (type), 93
confirm (setting), 99
confirm_wall_s (setting), 99
--console (command line), 9
--console (console), 32
CONSOLE_* (environment), 42
CONSOLE_DETERMINISM_ARMS (environment), 65
CONSOLE_EMIT_EXPECTED (environment), 64
contains (verb), 74
ContragredientLocalSystem (type), 85
--count-allocs (--bench), 55
--cross-checks (--bench), 55
crosschecks (verb), 76
cse (setting), 98
cuda (--bench micro), 57
cuda_copy_threads (setting), 100
cuda_device (setting), 99
cuda_disable (setting), 99
cuda_graph (setting), 99
CUDA_IMMA_ELIM_BENCH (environment), 65
cuda_merge_levels (setting), 100
cuda_pinned_mb (setting), 100
cuda_program_cache (setting), 99
cyclic_vector (verb), 74
cylinder (verb), 75
cylinder_scalar (verb), 76
cylinder_unipotent (verb), 76

decomp (--bench), 55

decompose (verb), 76
Decomposition (type), 86
degree (verb), 73
degrees (campaign spec key), 46
descent (verb), 81
DescentRecord (type), 94
describe (verb), 71
device (setting), 98
dickson_count (verb), 75
dickson_exceptional (verb), 75
dickson_les (verb), 80
DicksonLES (type), 94
display (verb), 72
--dry-run (--campaign), 45
dual (verb), 75

elim_base (setting), 99
ell (campaign spec key), 46
engine (--backend), 56
ENGINE_* (environment), 42
ENGINE_ARENA (environment), 57
ENGINE_AVX2_PAIR (environment), 58
ENGINE_BACKEND (environment), 57
ENGINE_BATCH (environment), 57
ENGINE_BINARY_F2 (environment), 58, 60
ENGINE_CSE (environment), 57
ENGINE_CUDA_ARENA_SLACK_MB (environment), 59
ENGINE_CUDA_COPY_THREADS (environment), 59
ENGINE_CUDA_DEVICE (environment), 42, 59
ENGINE_CUDA_DISABLE (environment), 7, 42, 59, 69
ENGINE_CUDA_GRAPH (environment), 59
ENGINE_CUDA_MERGE_LEVELS (environment), 59
ENGINE_CUDA_PINNED_CHUNK_MB (environment), 59
ENGINE_CUDA_PINNED_MB (environment), 59
ENGINE_CUDA_PROBE (environment), 10, 59
ENGINE_CUDA_PROGRAM_CACHE (environment), 59
ENGINE_DEVICE (environment), 59
ENGINE_ELIM_BASE (environment), 58
ENGINE_ELIM_PANEL (environment), 58
ENGINE_ELIM_SESSION (environment), 59
ENGINE_ELIM_TRSM_BASE (environment), 58
ENGINE_FUSE (environment), 57
ENGINE_GPU_ELIM (environment), 59, 60
ENGINE_GPU_ELIM_MIN (environment), 59, 60
ENGINE_GPU_ELIM_SIMULATE_CUDA_ERROR (environment), 60, 63
ENGINE_GPU_ELIM_VERIFY (environment), 59, 60
ENGINE_GPU_IMMA (environment), 59, 60
ENGINE_GPU_LEAN (environment), 59
ENGINE_GPU_SMITH2K (environment), 59
ENGINE_GPU_SMITH2K_HOST_MB (environment), 59
ENGINE_GPU_SMITH2K_MIN (environment), 59
ENGINE_GPU_SMITH2K_VERIFY (environment), 59
ENGINE_GPU_SMITH2K_VRAM_MB (environment), 59
ENGINE_HOST_ELIM_HOOK (environment), 58
ENGINE_HOST_THREADS (environment), 42, 57
ENGINE_KERNEL (environment), 58
ENGINE_KERNEL_TIMING (environment), 63
ENGINE_KSPLIT (environment), 57
ENGINE_LADIC_WIDTH (environment), 58
ENGINE_LAYOUT (environment), 57
ENGINE_SCALING_REPS (environment), 63
ENGINE_SPIN (environment), 58
ENGINE_TAPE_TRACE (environment), 63
ENGINE_TERNARY_F3 (environment), 58
ENGINE_THREADS (environment), 42, 53, 57, 69
ENGINE_TILE_COLS (environment), 58
ENGINE_TILE_MULT (environment), 58
ENGINE_TRACE_DEVICE (environment), 63
ENGINE_TRACE_DEVICE_RELEASE (environment), 63
ENGINE_TRACE_ELIM (environment), 63
ENGINE_TRACE_IMMA_ELIM (environment), 60, 63
ENGINE_TRACE_LEVELS (environment), 63
ENGINE_TRACE_NODES (environment), 63
ENGINE_TRACE_PASSES (environment), 63
ENGINE_TRACE_RSS (environment), 63
ENGINE_TRACE_VRAM (environment), 63
ENGINE_TRAP_REPS (environment), 63
ENGINE_VNNI (environment), 58
ENGINE_VNNI2_FLOOR (environment), 58
ENGINE_VNNI_SHADOW (environment), 58
eval (verb), 73
eval_at (verb), 75
exit (verb), 72
--expect-fnv (--serve-atlas), 43
ext (--bench), 55
ext (campaign spec key), 47
ext_gpu (campaign spec key), 47
ext_gpu_cells (campaign spec key), 47
ext_groups (campaign spec key), 47
ext_jobs (campaign spec key), 47
ext_memory_mb (campaign spec key), 47
ext_record (verb), 79
ExtensionClass (type), 91
ExtRecord (type), 89

factor (verb), 73
Family (type), 84

family (verb), 75
family (--bench), 55
family (campaign spec key), 47
field_polynomial (setting), 98
final_mode (setting), 97
fingerprint (verb), 76
forms_algebra (verb), 81
Fp (type), 82
Fp (verb), 72
fuse (setting), 98

gcd (verb), 73
gemm (--bench micro), 57
gemv (--bench micro), 57
GF (type), 88
GF (verb), 79
gf_add (verb), 79
gf_eval (verb), 79
gf_frobenius (verb), 79
gf_inv (verb), 79
gf_mul (verb), 79
gf_pow (verb), 79
gf_sub (verb), 79
GFElt (type), 88
glover_factors (verb), 78
golden (verb), 76
GOLDEN_* (environment), 42
GOLDEN_DUMP (environment), 64
GOLDEN_EMIT (environment), 64
golden_has (verb), 76
GOLDEN_OUT (environment), 64
golden_size (verb), 76
GOLDEN_STRICT (environment), 64
golden_subset (verb), 77
gpu (--backend), 56

-h (--atlas), 31
-h (--bench micro), 57
-h (--bench), 55
-h (--campaign), 45
-h (command line), 9
-h (console), 32
hash (verb), 76
hash_int (verb), 76
hecke (verb), 75
hecke (--bench), 55
HECKE_* (environment), 42
HECKE_ATLAS_OUT (environment), 31, 64
HECKE_CENSUS_SPLIT_ALL (environment), 64
HECKE_COSET_STREAM (environment), 62
HECKE_DESCENT_BDRY_CELLS (environment), 65
HECKE_DESCENT_CELLS (environment), 65
HECKE_DESCENT_EMBED_MAP (environment), 61
HECKE_DESCENT_GOOD_DENSE (environment), 61
HECKE_DESCENT_GOOD_SPARSE (environment), 61
HECKE_DESCENT_HECKE_CELLS (environment), 65
HECKE_DESCENT_LEVELN_CELLS (environment), 65
HECKE_DESCENT_N_CELLS (environment), 65
HECKE_DESCENT_PAIR_CELLS (environment), 65
HECKE_DESCENT_PAR_ASM (environment), 61
HECKE_DESCENT_PAR_ELL (environment), 61
HECKE_DESCENT_PAR_LN (environment), 61
HECKE_DESCENT_PARLN_CELLS (environment), 65
HECKE_DESCENT_REPEATS (environment), 65
HECKE_DESCENT_SMITH_BLOCKED (environment), 61
HECKE_DESCENT_SMITH_F2_MT (environment), 61
HECKE_DESCENT_SMITH_GPU_ARMS (environment), 65
HECKE_DESCENT_SMITH_GPU_CELLS (environment), 65
HECKE_DESCENT_SMITH_MT (environment), 61
HECKE_DESCENT_SMITH_MT_BENCH (environment), 65
HECKE_DESCENT_SMITH_MT_CELLS (environment), 65
HECKE_DESCENT_SMITH_MT_PANEL (environment), 65
HECKE_DESCENT_SMITH_MT_REPEATS (environment), 65
HECKE_DESCENT_SMITH_MT_THREADS (environment), 65
HECKE_DESCENT_SMITH_PANEL (environment), 61
HECKE_DESCENT_SMITH_SIMD (environment), 61
HECKE_DESCENT_TREE (environment), 61
HECKE_DESCENT_TREE_CELLS (environment), 65
HECKE_DESCENT_TREE_PAIR (environment), 61
HECKE_DESCENT_TREEPAIR_CELLS (environment), 65
HECKE_DUMP_SAT_CERT (environment), 64
HECKE_DUMP_SAT_CERT_CELLS (environment), 64
HECKE_DUMP_TAU (environment), 64
HECKE_DUMP_TAU_PAIR (environment), 64
HECKE_DUMP_TAU_RHO_CELLS (environment), 64
HECKE_EXT_JSON_OUT (environment), 64
HECKE_FIELD_DIET (environment), 62
HECKE_FORMS_FASTMUL (environment), 62
HECKE_FORMS_NTT3 (environment), 62

HECKE_FORMS_NTT_CACHE (environment), 62
HECKE_HIDA_COMPONENTS_CELLS (environment), 64
HECKE_HIDA_HEAVY (environment), 64
HECKE_LAMBDA_TOWER_CELLS (environment), 65, 67
HECKE_LAMBDA_TOWER_OPS_CAP (environment), 65, 67
HECKE_LAMBDA_TOWER_R3 (environment), 65, 67
HECKE_LAMBDA_TOWER_RSS_CAP_GIB (environment), 65, 67
HECKE_LEVEL_N_GEN_ORDER (environment), 62
HECKE_LEVEL_N_HOOK_RSS_CAP_GIB (environment), 65
HECKE_LEVEL_N_R3_SMALLP_CELLS (environment), 66
HECKE_LEVEL_N_SPARSE_CELLS (environment), 64
HECKE_LN23_HEAVY (environment), 66
HECKE_LN23_QMAX (environment), 66
HECKE_LN23_STURM (environment), 65, 66
HECKE_LN23_THM11_NMAX (environment), 66
HECKE_LN23_THM12I_NMAX (environment), 66
HECKE_LN23_Z1_NMAX (environment), 66
HECKE_LOCALITY_PROBE (environment), 64
HECKE_MODULE_EVAL (environment), 62
HECKE_MODULE_LEAN_PEAK (environment), 62
HECKE_MODULE_MT (environment), 62
HECKE_MODULE_PARALLEL (environment), 62
HECKE_MODULE_STREAM (environment), 62
HECKE_MODULE_TILE_WORK (environment), 62
HECKE_NEB_WILD_ALL (environment), 65
HECKE_NEBENTYPUS_CELLS (environment), 64
HECKE_NEBENTYPUS_NB3_FULL (environment), 65
HECKE_PARALLEL_SETUP (environment), 62
HECKE_PRECISION_REUSE (environment), 62
HECKE_RANK1_CLOSED_FORM (environment), 62
HECKE_RHO2_STREAM (environment), 62
HECKE_RHO_SIMD (environment), 62
hecke_ring_algebra (verb), 80
HECKE_RING_LOEWY_CELLS (environment), 64
HECKE_ROUTE2_SHARED_MEMO (environment), 62
HECKE_SAT_DEFECT_CELLS (environment), 64, 67
HECKE_SAT_DEFECT_STRICT (environment), 67
HECKE_SAT_PERF (environment), 62
HECKE_SHAPIRO_CELLS (environment), 65
hecke_structure (verb), 81
HECKE_TAME_FAMILY_CELLS (environment), 65
HECKE_TAMEFAMILY_HEAVY (environment), 65
HECKE_TAU_WIDE (environment), 62
HECKE_THETA_IMAGE_ALL (environment), 64
HECKE_U_ELL_LOW_RANK (environment), 62
HeckeAlgebra (type), 90
HeckeOperator (type), 84
--help (--atlas), 31
--help (--bench micro), 57
--help (--bench), 55
--help (--campaign), 45
--help (command line), 9
--help (console), 32
help (--bench micro), 57
help_markdown (verb), 78
hida_lambda (verb), 81
hida_tower (verb), 80
host_threads (setting), 99
hstack (verb), 74
hz_cert (campaign spec key), 46
hz_g8 (campaign spec key), 47

-i (command line), 9
-i (console), 32
identity (verb), 73
image (verb), 73
in_span (verb), 74
--index (--atlas), 31
--index (--serve-atlas), 43
intersect (verb), 74
inverse (verb), 73
Involution (type), 92
iota (verb), 80
is_cyclic (verb), 74
is_local (verb), 78

--jobs (--campaign), 45
jobs (campaign spec key), 47
jordan_type (verb), 74
json (verb), 71

kernel (verb), 73
kernel (--bench micro), 57
keys (verb), 71
ksplit (setting), 98
kummer_unit (verb), 80

L23_BIN3O1 (environment), 63
L23_F3L (environment), 63
L23_MOD3K (environment), 63
L23_RING2K (environment), 63
L23_TERNARY_F3 (environment), 63
L_map (verb), 79
layout (setting), 98
--lean (--bench), 55
lean (campaign spec key), 47
lean (setting), 99
legacy (--backend), 56

len (verb), 71
level (campaign spec key), 46
level_a (campaign spec key), 47
level_a_max_entries (campaign spec key), 47
level_n (verb), 80
lfunction (verb), 80
--list (command line), 8
local_algebra (verb), 79
locality (setting), 97
LocalSystem (type), 83
loewy (verb), 79
LoewyProfile (type), 91
--log (console), 32

math_settings (verb), 76
Matrix (type), 82
matrix (verb), 73
max (verb), 72
--max-reply-bytes (--serve-atlas), 43
--max-reply-bytes (--serve), 41
MaximalIdeal (type), 90
memory_lean (campaign spec key), 47
micro (verb), 77
micro (--bench), 55
min (verb), 71
minpoly (verb), 75
Model (type), 82
model (campaign spec key), 47
models (verb), 75
module_structure (verb), 79
moebius (verb), 75
moebius_scalar (verb), 76
moebius_unipotent (verb), 76
mul (verb), 73

n (campaign spec key), 46
--n-max (--atlas), 30
n_sym (campaign spec key), 46
name (campaign spec key), 47
nebentypus (verb), 80
NF_PRANK_CE (environment), 65
NF_PRANK_CELLS (environment), 65
NF_PRANK_DENSE (environment), 65
nilpotency_index (verb), 78
nilpotent_part (verb), 78
--no-cross-checks (--bench), 55
nullity (verb), 73

OC_* (environment), 42
orbifold (campaign spec key), 47
orbifold_k_cap (campaign spec key), 48
orbifold_m_target (campaign spec key), 47
ORBIFOLD_MP_CELLS (environment), 17, 65
ORBIFOLD_MP_DUMP (environment), 17, 64
orbifold_primes (campaign spec key), 47
--out (--atlas), 31
--output (--campaign), 45
output (campaign spec key), 47
output_dir (campaign spec key), 47

p (campaign spec key), 46
p_hecke (campaign spec key), 46
Pairing (type), 85
pairing (verb), 76
parts (campaign spec key), 46
Poly (type), 82
poly (verb), 72
pow (verb), 74
powmod (verb), 73
predictions (setting), 98
Presentation (type), 92
presentation (verb), 79
presentation (setting), 98
print (verb), 71
print_max_entries (setting), 97
program_cache (setting), 99
projective_plane (verb), 75

q (campaign spec key), 46
--query (--atlas), 31
--quiet (--atlas), 30
--quiet (--campaign), 45
--quiet (console), 32

r (campaign spec key), 46
radical (verb), 78
random_matrix (verb), 74
random_vector (verb), 74
range (verb), 71
rank (verb), 73
--repeat (--bench), 55
Report (type), 87
report (verb), 76
repr (verb), 71
reset_settings (verb), 72
restrict (verb), 74
resume (verb), 72
resume (campaign spec key), 47
rref (verb), 73
rref (--bench micro), 57
rss (verb), 77
--rss-cap-mib (--serve-atlas), 43
--rss-cap-mib (--serve), 41
run_campaign (verb), 77
run_validation (verb), 77

s1_scalar (verb), 76

s1_unipotent (verb), 76
save_state (verb), 76
scale (verb), 73
--script (command line), 9
--script (console), 32
script_args (verb), 72
--serve (command line), 9
--serve-atlas (command line), 10
--set (--serve), 41
--set (console), 32
Settings (type), 89
settings (verb), 72
settings_snapshot (verb), 79
--shard (command line), 9
show (verb), 71
sign (campaign spec key), 47
sl2z_model (verb), 75
socle (verb), 78
socle_series (verb), 79
solve (verb), 74
sphere (verb), 75
spin (setting), 99
split_field_report (setting), 98
state (verb), 72
--store (--serve-atlas), 43
str (verb), 71
structure (campaign spec key), 46
Sturm (type), 87
sturm (setting), 97
sturm_bound (verb), 78
sturm_unavailable (setting), 97
sub (verb), 73
submatrix (verb), 74
suite (--bench micro), 57
sum_spaces (verb), 74
symmetric_power (verb), 75

tangent_space (verb), 78
theta (verb), 80
theta_rank_max_dim (campaign spec key), 47
--threads (--campaign), 45
threads (campaign spec key), 47
threads (setting), 98
tile_cols (setting), 99
tile_mult (setting), 99
timed (verb), 77
trace (verb), 75
trace_elim (setting), 100
trace_levels (setting), 100
trace_nodes (setting), 100
trace_passes (setting), 100
trace_rss (setting), 100
transpose (verb), 74
transpose (--bench micro), 57
type (verb), 72
types (verb), 72

--u-ell (--bench), 55
u_ell (setting), 97
u_ell_decision (verb), 78
unvec (verb), 74

validations (verb), 77
vars (verb), 72
vec (verb), 74
--verbose (console), 32
verbose (setting), 97
verbs (verb), 72
--verify (--atlas), 30
--version (command line), 8
vstack (verb), 74

X (verb), 73

zero (verb), 73